

原版技术性开发系列教程

第 1 册

命令系统 Ciallo!

徐木弦 主编
(急招：编写组成员)

第一版序言

Minecraft 拥有多种多样的玩法，生存、PVP、PVE、模组、建筑、红石电路这些为众多玩家所熟知的玩法自成体系，玩家可以自由选择其中的某一方面深入研究。在这些玩法中，比较默默无闻的一种玩法可能便是广义上的命令，即包含了 MC-CMD（命令）、资源包和数据包的系统。

在没有系统地接触命令系统前，玩家对于这个系统不了解、甚至完全没有意识到这个系统的存在都是有可能的。对于一些仅对命令系统做出更新的快照版本，一些玩家可能认为它们是“无用的”，即使这些更新在大部分情况下丰富了命令系统，使得服务器的运营、冒险地图及原版模组的制作更方便。在较为早期的版本中，命令更多依靠命令方块运行，这使得一些玩家错误地认为冒险地图或原版模组中的机关是由纯粹的红石电路运行的，并将这些机关的巧妙之处完全归结于红石电路的运作而完全忽略命令的作用。

这是因为命令系统更多地呈现一种“在后台运行”的状态，玩家无法在明面上查看它们的运行逻辑，更多地只是感受它们带来的效果。其次，命令系统是一个“入门难、精通容易”的板块，学习的门槛较高，一些基本的概念，如权限等级、命名空间 ID、加载等级、坐标、目标选择器、JSON、NBT、记分板等，都是难于理解而对命令学习至关重要的知识点。再者，普通玩家在接触到命令系统时，更多地是熟悉诸如 `/kill`、`/gamemode` 这类带有简单参数的命令写法，而不会深究命令中各参数的意义，浅尝辄止的行为让玩家不大可能形成命令系统是一个完备体系的认识。最后，玩家在游戏时一般不会启用命令，而大部分命令所需的权限等级较高（即需要启用命令才能使用），因此一般玩家很少会与命令产生接触。

不过，命令系统应用范围极广，它对游戏的作用也是显而易见的，服务器的运行、冒险地图的制作、原版模组的制作均需要使用命令系统。对于游戏原有内容的修改和新内容的制作，如游戏外观的修改、粒子动画效果、自定义战利品、自定义物品等，均可以由命令系统来完成。命令系统提供了一个在不修改源代码的情况下的开发环境，使得 Minecraft 几乎成为了一款“无限”的游戏，唯一有限的也许便是玩家的想象力。同时，命令系统也极大地丰富了游戏社区、论坛。

在历代版本更新中，命令系统也随之有过大大小小的改善。在早期的版本中，命令一般只能借由聊天栏和命令方块执行，而命令方块是一种红石元件，因此彼时命令的自动化执行对于红石电路的依赖性较强，只能将红石信号转化为命令执行的信号或条件。然而在后续的版本中，函数及数据包的问世使得命令能够脱离红石电路单独运行，并随着数据包的不断完善，命令系统的研

究和应用则更加方便。一些冒险地图真正做到了零命令方块化,命令的执行全部转交给数据包完成。因此命令方块及红石电路的教程则放在了附录中,不使用数据包仅使用命令方块时可以参阅这些内容。

对于这样一个较为完备的体系,笔者在前人既有教程的基础上,理清思路,加以整理,为之编写完整的命令教程。在内容安排上,优先讲述基本概念,如权限等级、命令执行上下文、命名空间 ID、数据包标签、参数、区块及加载等级等,这些概念是能够执行命令的最基本条件。在绪论之后是命令系统的五大重要基础板块:坐标、目标选择器、JSON、NBT 和记分板,这些内容是需要熟练掌握的基本功。在第七章,我们主要学习命令 `/execute`,这条命令能够修改命令执行上下文,一般可以认为是整个命令系统中最强大的命令。一些零散的命令则放到了第八章中讲解。资源包和数据包则被安排到了其他教程中,这些内容是命令系统的进阶部分,一定程度上脱离了狭义的命令体系,却也是命令系统的精华部分。学有余力的读者可进行《资源包》和《数据包》两本教程的学习。

在系统学习命令系统前,读者需要了解到的一点是,其他玩法领域可能更多地追求稳定的游戏环境,因此可能会选择旧的游戏版本。而命令系统随着版本更新不断完善,使得命令玩家不自觉地追求新版本,因此有关命令的讨论一般都基于最新版游戏。一些问题,诸如“在 1.8 版本中这个命令怎么写”,则可能得不到答复,因为命令系统在扁平化时经历了巨大的改动,使得改动前后几乎是两个完全不同的系统。同时,不同平台的命令系统也是不互通的,本教程仅适用于 Java 版,部分内容与基岩版不同,因此无法作为基岩版命令教学的参考。

那么在学习命令之前需要掌握哪些预备知识呢?首先需要对原版的 Minecraft 有足够的了解,因为命令基本上涉及了游戏中的方方面面。其次可以适当学习一些英语,命令的参数中有大量的英语单词,如果熟悉它们的意思将会极大提升命令编写的效率。本教程在论述的过程中,会经常使用高等数学、线性代数和概率论与数理统计的语言,读者可先行了解与命令系统紧密相关的一些数学知识,如向量代数与空间解析几何、矩阵与变换、随机事件及概率分布等。计算机图形学的一些内容也包括在内。此外,大家还可以提前学习一些编程思想,建立严密的逻辑思维能力,懂得如何指挥电脑(游戏)进行工作。没有相关编程基础并不意味着完全不能学好命令,编程思维也可以在命令学习的过程中逐步建立。

本教程仅作为基础教程使用,即仅介绍命令系统中所有的基本概念,并在此基础上进行一定的应用、扩展,其中应用层面主要面向冒险地图和原版模组的制作。一些较为高级的算法则不在本教程范围之内。其中带“*”号的是选择性学习内容,它们所述的一些是不常用的内容,一些则是《资源包》或《数据包》两本教程的相关内容,想进修《资源包》和《数据包》的读者需留意这些带“*”号知识点。

限于笔者的知识水平,本教程必有一定疏漏之处,望广大读者斧正!

编者

2024 年 1 月 24 日

第二版序言

自 2023 年发布的 1.19.4 起，Minecraft 的技术性开发板块进入了又一个新时代，这可谓是自 2012 年骇人更新加入命令方块、2018 年水域更新扁平化以来第三次革命性的巨变。这次“巨变”的主要体现在于：逐步开放了很多 API，即将很多固有注册表转为可写注册表，使得原先需要大量命令模拟的效果用数据包自定义即可，这无疑大大省去了技术性开发的成本。从这两年 Minecraft Java 版的更新迭代可以看出，Minecraft 的开发重点逐渐地由加入新的实质性游戏内容转为对数据包、资源包的支持。新游戏内容更新的占比下降，技术性更新的占比上升。数据包、资源包的版本号更迭已由先前的一个大版本一次变为一个快照一次，迭代速度越来越快。以数据包为例，2022 年数据包的版本号为 10，彼时距数据包的加入仅过了四年多；而截止至 2024 年底，数据包的版本号已达到了 61。

许多玩家对自 1.17 以来的 Minecraft 开发极不满意，有些人认为 Minecraft 近几年并没有革命性的更新出现，每个版本的更新内容也极少，对游戏深度的挖掘不够，“敷衍了事”。对于每个快照更新介绍中大篇幅的技术性更新内容，大量玩家则视若无睹。这个现象直接反映了普通玩家对技术性开发这一板块认识的欠缺，正如第一版序言所说，“玩家对于这个系统不了解、甚至完全没有意识到这个系统的存在”。为此，对玩家进行技术性开发板块知识的普及很有必要。

不得不承认的是，技术性开发确实是一种门槛较高的玩法，其学习成本较高，容错率低，开发周期长，且开发过程不可见，因此难于在社区中引起较大的讨论度。目前，各大社区平台仍然缺乏专门的技术性开发论坛，现有的教程也零碎且不完整。社区的讨论主要集中于原版游戏内容、Mods 玩法等，鲜有技术性开发有关的资料，这使其搜索难度较大。技术性开发玩家通常“单打独斗”，缺少交流，进一步限制了有开发想法的玩家学习进步。为此，为已入门的技术性开发玩家编写系统性的、完整的教程很有必要。

原版技术性开发的命令、资源包、数据包三个板块在本系列教程中分作三部分别讲述。对于其中的一些基本概念，如注册表、坐标、目标选择器、文本组件、NBT、记分板、属性、存档格式等，为了便于教程编纂，将其放在《命令系统》一书中与命令语法一起说明。命令是数据包的一个子集，用于函数中，数据包为它提供了程序化运行的环境。在早先的版本中，命令通常是红石电路的一个子集，用于命令方块中，由红石信号控制命令的执行。虽然现在主流的开发环境是数据包，但仍有玩家使用命令方块红石电路进行开发，因此本系列教程中仍保留对命令方块红石电

路的讲解,《命令系统》书中的一些例子则提供了使用数据包和使用命令方块红石电路的两种解法,但侧重于使用数据包,而在《数据包》中则仅提供使用数据包的解法。资源包是相对较独立的一个板块,其内容中仅少部分与数据包有关联,对美术设计的要求较高,部分开发资源包的玩家专注于资源包,不会去开发数据包。然而在开发实践中数据包与资源包的联系却越来越紧密,仅依靠数据包能够实现的效果有限,与资源包配合开发能得到更好的效果。

这一版的《命令系统》教程相较上一版而言,对调了一些章节使得教程整体的逻辑更合理。同时新增了一些章节,新增的内容大多拆分自原本的章节,但对这些被拆分的内容有了更详尽的叙述。原本教程中零散的内容在这一版中均尽可能地归到一类下。经过这些变动后,教程的整体章节较先前的版本有了很大的不同。产生这些变动的主要原因是文本组件的存储格式由 JSON 变为了 SNBT,即使在数据包中文本组件仍使用 JSON 格式。从教程逻辑上讲,文本组件应合并至“NBT 格式”一章,但上一版的“NBT 格式”一章已有 11 个小节,内容量极其庞大,若再塞入文本组件的全部内容,只会使“NBT 格式”一章显得更臃肿。况且属性、状态效果这些内容依照存储格式也归到了实体格式或物品堆叠组件格式之下,将这些归类的内容全部放入“NBT 格式”一章更不适宜。于是,原本经典的“绪论——坐标——目标选择器——原始 JSON 文本——NBT 格式——记分板——命令 `/execute`——杂项命令”的教程章节顺序已不适用。经过考虑,这一版的教程章节变动安排如下:

1. “绪论”、“坐标与区块”、“UUID 与目标选择器”三章的顺序不变,仅调整这三章内部小节的顺序及编排。
2. 删去原本的“NBT 格式”一章,并将其重命名为“NBT 与 JSON 格式”,新的章节保留原本“NBT 格式”一章的概述、NBT 路径、命令 `/data` 的语法。同时将原本“原始 JSON 文本”一章的“JS 对象表示法”一节移入新的“NBT 与 JSON 格式”。随后将概述一节中涉及 SNBT 和 JSON 相互转换的内容移出,单独设置一个小节。
3. 将新的“NBT 与 JSON 格式”提前至第四章,将“原始 JSON 文本”改名为“文本组件”并后移至第五章。“文本组件”一章将同时讲解其 SNBT 和 JSON 形式。
4. 对于从原本“NBT 格式”一章中移出的其他内容,结合属性、状态效果,单独开设新的一章“存档格式”作为第六章,同时合并原本附录 II “游戏文件”的内容至这一章的概述小节。新的“存档格式”包含命令存储格式、区域文件格式、方块实体格式、实体格式、属性格式、状态效果格式、标记格式、展示实体格式、交互实体格式、玩家格式和物品堆叠组件格式。
5. “记分板”、“命令的执行”、“其他分类的命令”分别顺延至第七、八、九章。“命令的执行”更名为“命令 `/execute`”。
6. 由于章节合并,“其他分类的命令”中“状态效果”、“属性”两节被删除。
7. 删除原本的附录 II “游戏文件”。

受限于编者的知识水平,本教程一定有不足之处,望广大读者斧正!

编者

2025 年 2 月 7 日

目 录

第一章 绪论	1
1.1 注册表与数据值	2
1.1.1 注册表 *	2
1.1.2 扁平化 *	8
1.1.3 命名空间 ID	9
1.1.4 数据包标签	12
1.2 命令	12
1.2.1 命令参数	13
1.2.2 命令的输入	15
1.2.3 权限等级与限制条件	16
1.2.4 命令的解析 *	18
1.2.5 命令上下文	19
1.3 JSON 格式	21
1.3.1 JSON 数据类型	21
1.3.2 JSON 的转义序列	24
1.4 游戏文件	26
1.4.1 常用文件格式	26
1.4.2 .minecraft 文件夹 *	27
1.5 数据包	33
1.5.1 数据包的基本结构	36
1.5.2 实验性内容 *	43
1.5.3 数据包标签定义格式	44
1.6 资源包	47
1.6.1 资源包的基本结构	48
1.6.2 GPU 警告列表 *	53
1.6.3 地区合规性警告 *	54

1.7	游戏机制	55
1.7.1	端	55
1.7.2	游戏刻	57
1.7.3	区块加载	60
1.7.4	区块刻 随机刻 计划刻 红石刻	65
1.7.5	方块	66
1.7.6	实体	68
1.7.7	游戏规则	70
1.8	服务器管理	71
1.8.1	server.properties *	71
第二章	坐标	72
2.1	坐标系与坐标	73
2.1.1	坐标的命令参数类型	73
2.2	区块	76
2.2.1	命令/forceload	76
第三章	文本组件	77
3.1	文本组件内容	77
3.1.1	翻译文本	77
第四章	存档格式	78
4.1	存档文件夹的结构	78
4.2	方块实体	78
4.3	技术性实体	78
附录 I	数据库	79
I.1	数据包和资源包版本号	79
I.2	方块状态	90
I.3	数据包标签	90
索引		91
参考文献		95

第一章 绪论

原版技术性开发, **Minecraft Wiki** 称为“Java 版可自定义内容”, 是由命令、资源包、数据包及相关的组件附件组合成的一个板块。技术性开发成果丰富, 这些成果即是社区玩家常用的 **Mods**、冒险地图、数据包、资源包、服务器等。**Minecraft** 的技术性开发大致分为 **Mods** 开发和原版开发, 其区别在于是否对游戏的源代码进行了修改。

本系列教程针对的是原版技术性开发, 这一部分玩家的工作方向通常为制作冒险地图、开发原版模组、制作资源包或者管理服务器。

1.1 注册表与数据值

Minecraft 有许多不同的游戏资源，如草方块、石头、箭、铁锹、猪等，对这些游戏资源进行分类，可以将草方块、石头分为方块，箭、铁锹分为物品，猪划分至实体。方块、物品、实体显然是区分这些游戏资源的“大类”。**注册表 (Registry)** 就是对不同资源进行分类管理的机制。除分类管理外，还需要给予每种资源一个独特的“身份证”，目的是与别的资源区分开来，唯一地映射到注册表内的给定值。这些“身份证”被称为游戏资源的数据值，或称 ID。

1.1.1 注册表 *

注册表可分为以下两类：**固有注册表 (Built-in registry)** 和 **可写注册表 (Writable registry)**。无论资源位于什么类型的注册表，**游戏都只会识别已被注册的资源**。

下面列举了 Minecraft 中所有的资源类型：

一、固有注册表

固有注册表存储硬编码的游戏内容，除非修改源代码，否则其中的内容不可更改。

表 1.1 固有注册表

注册表	资源类型	数据包路径	默认值
ACTIVITY	生物 AI	activity	
ATTRIBUTE	属性	attribute	
ATTRIBUTE_TYPE	环境属性类型	attribute_type	
BIOME_SOURCE	生物群系源	worldgen/biome_source	
BLOCK	方块	block	air
BLOCK_ENTITY_TYPE	方块实体类型	block_entity_type	furnace
BLOCK_PREDICATE_TYPE	方块谓词类型	block_predicate_type	
BLOCK_STATE_PROVIDER_TYPE	方块状态提供者类型	worldgen/block_state_provider_type	
BLOCK_TYPE	方块类型	block_type	
CARVER	雕刻器类型	worldgen/carver	
CHUNK_GENERATOR	区块生成器类型	worldgen/chunk_generator	
CHUNK_STATUS	区块状态	chunk_state	empty
COMMAND_ARGUMENT_TYPE	命令参数类型	command_argument_type	
CONSUME_EFFECT_TYPE	消耗使用效果类型	consume_effect_type	
CREATIVE_MODE_TAB	创造标签页	creative_mode_tab	

(续表)

注册表	资源类型	数据包路径	默认值
CUSTOM_STAT	统计信息	custom_stat	
DATA_COMPONENT_PREDICATE_TYPE	数据组件谓词类型	data_component_predicate_type	
DATA_COMPONENT_TYPE	数据组件类型	data_component_type	
DEBUG_SUBSCRIPTION	调试订阅	debug_description	
DECORATED_POT_PATTERN	饰纹陶罐图案类型	decorated_pot_pattern	
DENSITY_FUNCTION_TYPE	密度函数类型	worldgen/density_function_type	
DIALOG_ACTION_TYPE	对话框操作类型	dialog_action_type	
DIALOG_BODY_TYPE	对话框主体类型	dialog_body_type	
DIALOG_TYPE	对话框类型	dialog_type	
ENCHANTMENT_EFFECT_COMPONENT_TYPE	魔咒效果组件类型	enchantment_effect_component_type	
ENCHANTMENT_ENTITY_EFFECT_TYPE	魔咒实体效果类型	enchantment_entity_effect_type	
ENCHANTMENT_LEVEL_BASED_VALUE_TYPE	魔咒等级依赖函数类型	enchantment_level_based_value_type	
ENCHANTMENT_LOCATION_BASED_EFFECT_TYPE	魔咒位置依赖函数类型	enchantment_location_based_effect_type	
ENCHANTMENT_PROVIDER_TYPE	魔咒提供器类型	enchantment_provider_type	
ENCHANTMENT_VALUE_EFFECT_TYPE	魔咒值效果类型	enchantment_value_effect_type	
ENVIRONMENT_ATTRIBUTE	环境属性	environment_attribute	
ENTITY_SUB_PREDICATE_TYPE	实体子谓词类型	entity_sub_predicate_type	
ENTITY_TYPE	实体类型	entity_type	pig
FEATURE	地物类型	worldgen/feature	
FEATURE_SIZE_TYPE	树木生成的最小空间要求类型	worldgen/feature_size_type	
FLOAT_PROVIDER_TYPE	浮点数提供器类型	float_provider_type	
FLUID	流体类型	fluid	empty
FOLIAGE_PLACER_TYPE	树叶放置器类型	worldgen/foilage_placer_type	
GAME_EVENT	游戏事件	game_event	step
GAME_RULE	游戏规则	game_rule	step
HEIGHT_PROVIDER_TYPE	高度提供器类型	height_provider_type	
INCOMING_RPC_METHOD	服务端管理协议中的请求方法	incoming_rpc_methods	
INPUT_CONTROL_TYPE	对话框输入控件类型	input_control_types	

(续表)

注册表	资源类型	数据包路径	默认值
INT_PROVIDER_TYPE	整数提供器类型	int_provider_type	
ITEM	物品	item	air
SLOT_SOURCE_TYPE	槽位源类型	slot_source_type	
LOOT_CONDITION_TYPE	战利品表谓词类型	loot_condition_type	
LOOT_FUNCTION_TYPE	物品修饰器类型	loot_function_type	
LOOT_NBT_PROVIDER_TYPE	战利品表相关 NBT 源类型	loot_nbt_provider_type	
LOOT_NUMBER_PROVIDER_TYPE	值提供器类型	loot_number_provider_type	
LOOT_POOL_ENTRY_TYPE	抽取项类型	loot_pool_entry_type	
LOOT_SCORE_PROVIDER_TYPE	分数提供器类型	loot_score_provider_type	
MAP_DECORATION_TYPE	地图图标类型	map_decoration_type	
MATERIAL_CONDITION	地表规则条件	worldgen/material_condition	
MATERIAL_RULE	地表规则	worldgen/material_rule	
MEMORY_MODULE_TYPE	生物记忆	memory_module_type	dummy
MENU	屏幕类型	menu	
MOB_EFFECT	状态效果	mob_effect	
NUMBER_FORMAT_TYPE	记分板分数显示样式类型	number_format_type	
OUTGOING_RPC_METHOD	服务端管理协议中的通知方法	outgoing_rpc_methods	
PARTICLE_TYPE	粒子类型	particle_type	
PLACEMENT_MODIFIER_TYPE	放置修饰器类型	worldgen/placement_modifier_type	
PERMISSION_CHECK_TYPE	命令权限检查类型	permission_check_type	
PERMISSION_TYPE	命令权限类型	permission_type	
POINT_OF_INTEREST_TYPE	兴趣点类型	point_of_interest_type	
POOL_ALIAS_BINDING	模板池映射	worldgen/pool_alias_binding	
POSITION_SOURCE_TYPE	位置源类型	position_source_type	
POS_RULE_TEST	位置规则测试类型	pos_rule_test	
POTION	药水效果	potion	
RECIPE_BOOK_CATEGORY	配方书分类标签	recipe_book_category	
RECIPE_DISPLAY	预览配方类型	recipe_display	
RECIPE_SERIALIZER	配方序列化/反序列化器	recipe_serializer	

(续表)

注册表	资源类型	数据包路径	默认值
RECIPE_TYPE	配方类型	recipe_type	
REGISTRY	注册表	root	
ROOT_PLACER_TYPE	树根放置器类型	worldgen/root_placer_type	
RULE_TEST	规则测试	rule_test	
RULE_BLOCK_ENTITY_MODIFIER	方块实体数据修饰器	rule_block_entity_modifier	
SENSOR_TYPE	感受器类型	sensor_type	
SLOT_DISPLAY	预览槽位类型	slot_display	
SOUND_EVENT	声音事件	sound_event	
SPAWN_CONDITION_TYPE	通用变种选择器类型	spawn_condition_type	
STAT_TYPE	统计类型	stat_type	
STRUCTURE_PIECE	结构片段	worldgen/structure_piece	
STRUCTURE_PLACEMENT	结构放置方式	worldgen/structure_placement	
STRUCTURE_POOL_ELEMENT	结构模板池元素	worldgen/structure_pool_element	
STRUCTURE_PROCESSOR	结构处理器	worldgen/structure_processor	
STRUCTURE_TYPE	结构类型	worldgen/structure_type	
TEST_ENVIRONMENT_DEFINITION_TYPE	测试环境类型	test_environment_definition_type	
TEST_FUNCTION	硬编码测试函数	test_function	
TEST_INSTANCE_TYPE	测试实例类型	test_instance_type	
TICKET_TYPE	加载标签及计算标签类型	ticket_type	
TREE_DECORATOR_TYPE	树额外装饰器类型	worldgen/tree_decorator_type	
TRIGGER_TYPE	触发器类型	trigger_type	
TRUNK_PLACER_TYPE	树干放置器类型	worldgen/trunk_placer_type	
VILLAGER_PROFESSION	村民职业	villager_profession	none
VILLAGER_TYPE	村民类型	villager_type	plain

二、可写注册表

可写注册表允许数据包通过数据反序列化器向其中添加自定义（或称数据驱动）的游戏内容。

表 1.2 可写注册表

注册表	资源类型	数据包路径
ADVANCEMENT	进度	advancement
BANNER_PATTERN	旗帜图案	banner_pattern
BIOME	生物群系	worldgen/biome
CAT_VARIANT	猫的变种	cat_variant
CHAT_TYPE	聊天类型	chat_type
CHICKEN_VARIANT	鸡的变种	chicken_variant
CONFIGURED_CARVER	已配置的雕刻器	worldgen/configured_carver
CONFIGURED_FEATURE	已配置的地物	worldgen/configured_feature
COW_VARIANT	牛的变种	cow_variant
DAMAGE_TYPE	伤害类型	damage_type
DENSITY_FUNCTION	密度函数	worldgen/density_function
DIALOG	对话框	dialog
DIMENSION	维度	dimension
DIMENSION_TYPE	维度类型	dimension_type
ENCHANTMENT	魔咒	enchantment
ENCHANTMENT_PROVIDER	魔咒提供器	enchantment_provider
FLAT_LEVEL_GENERATOR_PRESET	超平坦世界生成预设	worldgen/flat_level_generator_preset
FROG_VARIANT	青蛙的变种	frog_variant
INSTRUMENT	山羊角乐器	instrument
ITEM_MODIFIER	物品修饰器	item_modifier
JUKEBOX_SONG	唱片机曲目	jukebox_song
LEVEL_STEM	维度	dimension
LOOT_TABLE	战利品表	loot_table
MULTI_NOISE_BIOME_SOURCE_PARAMETER_LIST	多噪声参数列表	worldgen/multi_noise_biome_source_parameter_list
NOISE	噪声	worldgen/noise
NOISE_SETTINGS	噪声设置	worldgen/noise_settings
PAINTING_VARIANT	画的变种	painting_variant
PIG_VARIANT	猪的变种	pig_variant
PLACED_FEATURE	已放置的地物	worldgen/placed_feature
PREDICATE	谓词	predicate
PROCESSOR_LIST	处理器列表	worldgen/processor_list
RECIPE	配方	recipe

(续表)

注册表	资源类型	数据包路径
STRUCTURE	已配置的结构地物	worldgen/structure
STRUCTURE_SET	结构集	worldgen/structure_set
TEMPLATE_POOL	结构池	worldgen/template_pool
TEST_ENVIRONMENT	测试环境	test_environment
TEST_INSTANCE	测试实例	test_instance
TIMELINE	时间线	timeline
TRADE_SET	交易集	trade_set
TRIAL_SPAWNER_CONFIG	试炼刷怪笼配置	trial_spawner
TRIM_MATERIAL	盔甲纹饰材料	trim_material
TRIM_PATTERN	盔甲纹饰图案	trim_pattern
VILLAGER_TRADE	村民交易	villager_trade
WOLF_VARIANT	狼的变种	wolf_variant
WORLD_CLOCK	世界时钟	world_clock
WORLD_PRESET	世界预设	worldgen/world_preset
ZOMBIE_NAUTILUS_VARIANT	僵尸鹦鹉螺变种	zombie_nautilus_variant

三、不属于任何注册表的游戏资源

有一些游戏资源不属于任何注册表,这些资源包括数据包内的函数、结构模板以及资源包内的所有内容。这些资源类型中部分都与可写注册表的性质类似,即可以自定义写入资源;部分则不能增添新的资源,但可以修改已有资源的配置文件。

表 1.3 其他游戏资源

类别	游戏资源	说明
数据包内容	函数	可写, <code>function</code> 路径下的内容
	结构模板	可写, <code>structure</code> 路径下的内容
资源包内容	纹理图集	位于资源包内路径 <code>atlases</code>
	方块状态	位于 <code>blockstates</code>
	纹饰图案	位于 <code>equipment</code>
	字体	可写, 位于 <code>font</code>
	物品模型映射	位于 <code>items</code>
	模型	包括方块模型和物品模型, 位于 <code>models</code>
	粒子	位于 <code>particles</code>
	着色器	可写, 位于 <code>shaders</code>

(续表)

类别	游戏资源	说明
	后处理管线	位于 <code>post_effect</code>
	声音	可写, 位于 <code>sounds</code>
	纹理	可写, 位于 <code>textures</code>
属性修饰符		可写, 存储于所属物品的堆叠组件内
Boss 栏		可写, 存储于存档文件夹中的 <code>level.dat</code>
命令存储		可写, 存储于存档文件夹中的 <code>data\command_storage_minecraft.dat</code>
随机序列		可写, 存储于存档文件夹中各自维度的 <code>data\random_sequences.dat</code> 文件内

1.1.2 扁平化 *

Minecraft 的历次版本更新都会对某一些特定的系统进行优化和更改, 比如: 战斗更新对 PVP 机制进行了颠覆性的更改, 使得 1.9 之前和之后的 PVP 是两个完全不同的系统。命令系统也经历过类似的大幅度更改, 这便是随着水域更新进行的**扁平化 (The flattening)**。

在 Minecraft 开发之初, 由于游戏资源的数量有限, 只需要使用 1 字节就可以设置所有游戏资源的 ID。在历次版本更新中, Minecraft 的方块、物品数量越来越多, 特别是自缤纷更新以来, 方块的数量呈爆炸式增长。在扁平化之前, 为了应对这些不断增多的游戏资源, 一种解决办法是将一大类全部收归到某一个特定的 ID 中, 用这个 ID 来表示这一类方块, 然后在后面附加一个 Damage 值来表示这一类方块中的某一种。比如花岗岩属于石头一类, 石头的数字 ID 为 1, 而花岗岩的 Damage 值为 1, 所以在旧版本中给予玩家一块花岗岩的命令为:

```
give @p 1 1 1
```

1.1

可以看到这条命令中有三个参数, 第一个 `1` 为石头的 ID, 第二个 `1` 为数量, 第三个 `1` 为 Damage 值。这种表示方式的底层逻辑是: 在访问花岗岩时, 必须先访问上一级的数字 ID, 再访问 Damage 值, 如此才能映射至花岗岩这个值。

缤纷更新做了一个小修改: 即启用了部分英文 ID, 于是在 1.8 中给予玩家一块花岗岩的命令变为:

```
give @p stone 1 1
```

1.2

但是缤纷更新做出的这种更改是不完全的, 虽然在命令的主体部分将数字 ID 替换成了英文 ID, 但是方块的 Damage 值仍然存在, 这种一级 ID——Damage 值的映射方式没有改变。

时间来到了 2017 年, 水域更新加入了大量的游戏内容, 原先的映射方式已不能适应新版本。于是, **Java 版 1.13 的更新基本上删除了所有的数字 ID, 使得每一个游戏资源都有其独立的英文 ID。同时也删除了用 Damage 值映射方块的办法, 去除了中间层, 使得资源的映射方式只需要一个键名即可, 这一过程便被称作“扁平化”^①**。比如, 在 1.13 中给予玩家一块花岗岩的命令为:

```
give @p granite 1
```

1.3

① 并非所有数字 ID 都被移除, 至今 Minecraft 仍保留了部分需要使用数字 ID 的地方。

其中参数 `granite` 为花岗岩的键名，`1` 为物品数量。扁平化对所有需要 ID 的对象都进行了修改，包括但不限于方块、物品、实体、生物群系、粒子、声音事件和画。其具体内容可分为以下几类：

1. 拆分

拆分是最能体现扁平化过程的一类。此举移除了用于指定大类下某一种方块的 Damage 值，使得一类方块下的每一种方块都有其独立的 ID。举例：`stone` 一类共有七种不同的方块：石头、花岗岩、磨制花岗岩、闪长岩、磨制闪长岩、安山岩和磨制安山岩，分别对应 0~6 的 Damage 值。拆分后这七种方块都被给予了独立的 ID：`stone`、`granite`、`polished_granite`、`diorite`、`polished_diorite`、`andesite` 和 `polished_andesite`。

2. 重命名

顾名思义，该类即对原有的英文 ID 进行重命名。有相当一部分重命名是为了迎合方块或物品的英文名称。举例：草方块在扁平化前的 ID 为 `grass`，扁平化后被重命名为 `grass_block`。

3. 重新分类

这种操作常见于台阶和由双台阶组成的完整方块。扁平化前的台阶和双台阶有两种不一样的 ID，扁平化后取消双台阶的 ID，并对台阶的下属分类进行拆分，同时取消相关的 Damage 值。举例：双木台阶被取消，统一更换为木制台阶，同时 ID 拆分为橡木、云杉、白桦、丛林木、金合欢和深色橡木。

4. 合并

合并常见于有不同方块状态的方块。举例：燃烧的熔炉和熔炉有不同的 ID，现合并为熔炉一种，同时将是否燃烧设定为方块状态。

1.1.3 命名空间 ID

游戏资源的指定有一个前提是这些对象的 ID 相互之间不能混淆。数字 ID 及使用 Damage 值作区分的指定方法能够避免对象之间的冲突，但扁平化后这种指定方法便无效了。为此在当前的版本中统一使用 **(赋) 命名空间 ID (Namespaced identifier)** 来映射注册表内的值。

命名空间 ID，又称 **(赋) 命名空间标识符**、**资源路径 (Resource location)**、**资源标识符 (Resource identifier)** 或 **命名空间字符串 (Namespaced string)**，是字符串化的映射方式。无论命名空间 ID 用于映射何种对象，它们都具有同一的表达方式：

```
<namespace>:<path>
```

1.4

其中的参数： `<namespace>` ——命名空间。

`<path>` ——路径。

在写法上，除用于分割命名空间和路径的冒号 `:` 外，其中所有的字符都只能为合法字符。合法字符包含以下几类：

1. 数字：`0123456789`；
2. 小写字母：`abcdefghijklmnopqrstuvwxyz`；
3. 下划线：`_`；
4. 连字符：`-`；
5. 点：`.`。

小提示

大写字母在命名空间 ID 中为非合法字符！

除此之外所有的字符均为非法字符，包括汉字、平假名、片假名、西里尔字母、希腊字母、制表符、几何图形符、`+` 等符号。斜杠 `/` 比较特殊，它在命名空间中为非法字符，但是在路径中可用于分割目录的不同层级，以下会有详细说明。

一、命名空间 ID 的实际意义

小提示

原版游戏中大部分对象都使用命名空间 `minecraft`，但是六种基本命令参数类型（见小节 1.2.1）却使用命名空间 `brigadier`。

命名空间（Namespace） 是游戏资源的区界，它位于资源类型的父层级，所有来自 Minecraft 原版游戏的资源均位于命名空间 `minecraft`。通过不同的自定义命名空间可以将新增的内容和原版内容区分开来，以防止新内容和原版内容、新内容和其他新内容之间产生冲突。例如，有两个命名空间 ID `minecraft:something` 和 `custom:something`，它们指定的是两个不同的对象，因为它们的命名空间不同，前者为 `minecraft`，后者为 `custom`，即使两者拥有相同的路径（名称）`something`。

部分游戏资源在命名空间下有一定的文件路径，尤其是资源包、数据包内容，这时就可以使用 `/` 以表明它们的文件路径。这里命名空间实际上是一个文件夹，这个文件夹的结构一般是 `<命名空间>\<资源类型>\<文件夹 1>\<文件夹 2>\...<文件名>.<后缀>`。若要映射到这个文件，则命名空间 ID 的写法为

```
<命名空间>:<文件夹 1>/<文件夹 2>/...<文件名>
```

1.5

注意以下几点：

1. 资源类型不作为命名空间 ID 的一部分；
2. 资源路径的子文件夹至文件之间的一系列路径必须完整；
3. 文件名后缀不写。

下面举两个例子以说明之：

例 1.1

有数据包函数文件路径为 `minecraft\function\load.mcfunction`，试用命名空间 ID 指定之。

解

这里 `function` 为资源类型，`.mcfunction` 为文件的后缀。故命名空间 ID 为

```
minecraft:load
```

1.6

例 1.2

有资源包纹理文件路径为 `minecraft\textures\block\command_block_front.png`，试用命名空间 ID 指定之。

解

这里 `textures` 为资源类型，`.png` 是文件后缀，依照其路径将命名空间 ID 写为

```
minecraft:block/command_block_front
```

1.7

二、命名空间 ID 字符串的识别与一般写法

命名空间 ID 是形如 `<字符串>:<字符串>` 的字符串，游戏在识别、调用相关对象时，如果冒号 `:` 存在，会将冒号前的内容视作命名空间，其中不能出现斜杠 `/`；将冒号后的内容视作路径，可以出现斜杠。为了保证识别的正确性，整个字符串中最多只能出现一个冒号，否则识别会出现错误。如果冒号不存在，字符串的形式为 `<字符串>`，则游戏会直接将整个字符串直接识别为路径，并默认命名空间为 `minecraft`，有些时候可以适当减少书写命名空间，但省略命名空间的行为仍是不被建议的：虽然命名空间在原版大部分需要 ID 的地方上不是必须的，但在一些情况下命名空间是必须的，包括但不限于 NBT 路径中指定 ID 的节点。鉴于读者在实践的过程中可能会忘记必须添加命名空间的地方，或是混淆原版内容和自定义的内容，那么最好还是完整地书写命名空间 ID。

例 1.3

下列命名空间 ID 的识别结果为何？

something	1.8
minecraft:something	1.9
custom:something	1.10
minecraft/custom:something	1.11
minecraft/something	1.12
minecraft:Something	1.13
minecraft:custom:something	1.14

解 以上各命名空间 ID 的识别结果列于下表：

表 1.4 命名空间 ID 的识别结果

命名空间 ID	识别的命名空间	识别的路径	识别结果
1.8	minecraft	something	minecraft:something
1.9	minecraft	something	minecraft:something
1.10	custom	something	custom:something
1.11	- (含有非法字符 <code>/</code>)	-	识别失败
1.12	minecraft	minecraft/something	minecraft:minecraft/something
1.13	minecraft	- (含有非法字符 <code>S</code>)	识别失败
1.14	- (冒号数量大于 1)	-	识别失败

一般而言，命名空间和路径推荐的写法是**蛇形命名法 (Snake case)**，即当名称中含有多个单字时，以下划线 `_` 取代每一个空格的写法。蛇形命名法的书写仍需遵守合法字符的规定，不能出现大写字母。例如，下面的命名空间 ID 在命名空间和路径上均使用了蛇形命名法：

```
ancient_city:get_out
```

1.15

1.1.4 数据包标签

一个单独的命名空间 ID 只能映射至单独的一个对象，如果要同时映射多个对象，一般的做法是将对象分类，通过映射同一种类别的对象从而映射多个对象。这种将游戏资源分类的手段被称为**数据包标签 (Tags in data packs)**，简称**标签 (Tag)**。由于命令系统存在多个名为“标签”的概念，笔者不建议使用这样的简称以防止与其他概念的混淆。原版游戏有一些既有数据包标签，数据包标签的名称大多拥有实际的意义：例如，数据包标签 `#fire` 映射至两种方块，即 `fire`（火焰）和 `soul_fire`（灵魂火焰）；`#mineable/axe` 映射至所有能被斧采集的方块。

数据包标签的表示方式类似于命名空间 ID，但需要在前面加上井号 `#`，写法为

```
#<namespace>:<id>
```

1.16

例如 `#minecraft:fire`。数据包标签映射的对象可以直接是一个游戏资源，如方块、实体等，也可以是另一个数据包标签。例如，数据包标签 `#minecraft:mineable/axe` 还包含了 `#minecraft:planks`（所有种类的木板）、`#minecraft:signs`（所有种类的告示牌）等数据包标签。指定某数据包标签时，其映射的其他数据包标签下的对象也会被选择。但是同一个数据包标签映射的资源类型必须相同，不能将不同类型的对象放入一个数据包标签中，例如，猪和石头不能被放在同一个数据包标签中。

数据包标签涵盖的对象类型非常广，包括方块、实体、物品、游戏事件、生物群系等。读者可以在既有数据包标签的基础上，使用数据包添加一些自定义的数据包标签。数据包标签的定义方式见小节 1.5.3。

1.2 命令

命令 (Command)，又称**控制台命令 (Console command)**、**斜杠命令 (Slash command)** 或 **MC-CMD**，是一种高级的、通过输入具有特定语法文本以实现控制游戏本身运行的功能。命令文本需要讲究严格的语法，不允许任何模糊的表达。目前 MC-CMD 已被正式确认为一种编程语言，名称为 `mcfuction`，与 C 语言、Java、Python 等并列——但这是一种只适用于游戏 Minecraft 内部的编程语言，无法与外部环境进行交互。

1.2.1 命令参数

参数是命令的组成部分，每一条命令都由一个命令头和若干参数组成，参数之间用空格分隔，由此得到命令的通用格式：

```
<命令名> [<参数 1>] [<参数 2>] ...
```

1.17

例如在下面的命令中，`say` 是该命令的命令名，`Hello World!` 是后续参数：

```
say Hello World!
```

1.18

在本教程中，为了方便解释命令中各参数的语法，会使用一系列括号或其他符号来表示这些参数的具体内容及可用性。为了使说明更加方便，本教程采用了和游戏本体、Minecraft Wiki 一样的命令语法格式，如下表所示。

表 1.5 语法指引格式

语法	含义	举例
<code>字面量</code>	按字面量原样输入	命令 <code>/locate biome</code> 中第 2 个参数 <code>biome</code> 即为字面量，编写命令时不要更改这个参数的写法。
<code><参数></code>	需要使用一合适的值来替换该参数	命令 <code>/say <message></code> 的第 2 个参数需要用自定的值，比如 <code>/say Hello World!</code> 。
<code>[<参数>]</code>	该参数是可选的，如果使用该参数，则需要使用一合适的值替换之	命令 <code>/summon <entity> [<pos>] [<nbt>]</code> 中第 3、4 个参数可选，填写这些参数时需要用自定的值。
<code>(参数 参数)</code>	(必须的) 在显示的值中选择一个填写。语法介绍中这些值由竖线分隔	命令 <code>/time query (daytime gametime day)</code> 的第 3 个参数是必填的，从 <code>daytime</code> 、 <code>gametime</code> 和 <code>day</code> 中选择一个填写。
<code>[参数 参数]</code>	(可选的) 在显示的值中选择一个填写。语法介绍中这些值由竖线分隔	命令 <code>/experience add <targets> <amount> [levels points]</code> 的第 5 个参数虽然不是必写的，但仍可以从参数 <code>points</code> 和 <code>levels</code> 中选择一个填写。
<code>-> 子命令</code>	必须接入一条子命令	命令 <code>/execute positioned <pos> -> execute</code> 的第 6 个参数是必填的，内容为命令 <code>/execute</code> 的一个子命令。
<code>-> [子命令]</code>	可以接入一条子命令	命令 <code>/execute (if unless) block <pos> <block> -> [execute]</code> 的第 7 个参数可以填写 <code>/execute</code> 的子命令，也可以不填写。

下面举一实例以说明之：

例 1.4

命令 `/data` 的一种语法如下所示：

```
data get (block <targetPos>|entity <target>|storage <target>) [<path>]
[<scale>]
```

1.19

解

语法中 `data` 为命令名，`get` 是字面量，这两者必须按语法指引中的字面量原样输入命令。第 3、4 个参数必须从 `block <targetPos>`、`entity <target>` 和 `storage <target>` 中选则

一种，且不得空缺。若使用 `block <targetPos>`，则 `block` 按原样输入，后续使用 `<targetPos>` 的自定义值，但不得在 `block` 后续使用 `<target>` 参数，因为 `<target>` 是 `entity` 或 `storage` 的后续参数。第 5、6 个参数 `[<path>]`、`[<scale>]` 可选并自定义值。使用该语法且实际可行的命令可以是：

```
data get entity @s SelectedItem
```

1.20

命令 1.20 使用了参数 `entity`，`@s` 是 `<target>` 使用的值，`SelectedItem` 是 `[<path>]` 使用的值，参数 `[<scale>]` 未使用。

为了使不同的命令具有不同的功能，它们使用的参数类型各不相同。有些命令作用的对象为实体，它们则会使用指定实体的参数，有些命令作用的对象为某一个坐标，则其使用坐标参数。游戏使用的命令参数有如布尔值、整型、函数、槽位值、坐标值、目标选择器、JSON、NBT 等。一些复杂参数会在本教程后文呈现，下面列举的是基本参数，即在 Brigadier 中使用的六种基本数据类型：

1. 布尔值 (Bool)

布尔值 (Bool) 只有两种可用参数，为 `true` 和 `false`，分别代表“是”与“否”。

2. 整数 (Integer)

使用 32 位整型数值，是介于 `-2147483648` 和 `2147483647` 之间的整数值，如 `1`、`0`、`-1` 等。不同命令中使用整型的参数规定的最大可用值和最小可用值不一致。

3. 长整数 (Long)

使用 64 位整型数值，是介于 `-9223372036854775808` 和 `9223372036854775807` 之间的整数值。

4. 单精度浮点数 (Float)

使用占据 4 字节的浮点数，范围大约介于 -3.4×10^{38} 和 3.4×10^{38} 之间，在不同命令中使用单精度浮点数的参数规定的最大可用值和最小可用值不一致。一些单精度浮点数的示例有：`0`、`1.1`、`-1`、`.5` 等，小数形式的整数部分可以省略。在命令参数中使用的浮点数暂时不支持科学计数法^①。

5. 双精度浮点数 (Double)

使用占据 8 字节的浮点数，范围大约介于 -1.8×10^{108} 和 1.8×10^{108} 之间。可以表示比单精度浮点数绝对值更大的有效数字。

6. 字符串 (String)

字符串是由多个字符组成的序列，可用于表示单词、句子或其他符号组合。

(1) 单个词 (Single word)

即不含空格的字符串，如 `word`，若单个词的内容由多个词语组成，则一般使用下划线 `_` 连接相邻词，如 `word_with_underscores`。

(2) 词组 (Quotable phrase)

可以由双引号括起，如 `"quoted phrase"`，也可以使用单引号来定义，如 `'quoted phrase'`，此时单词之间可以有空格。

^① 参见 MC-130925。

(3) 贪婪词组 (Greedy phrase)

这种形式的词组不带引号，任意使用空格。该形式的参数通常位于命令的末尾，将命令的剩余部分全部作为字符串参数。如：`words with spaces`。上文中命令 1.18 就使用了这种参数。

1.2.2 命令的输入

命令是一种文本输入，以下是可供命令输入的途径：

1. 使用聊天栏输入命令

为了和普通的聊天文本区分开来，在聊天栏中输入命令时会在命令前加一个前缀 `/`，此前缀必不可少。在不使用按键 `T` 召唤聊天栏时可以直接键入 `/` 输入命令，这是使玩家快速进入命令输入模式的一种办法。

呼出聊天栏后，可以使用 `↑` 或 `↓` 键调用**命令历史 (Command history)**，即先前键入的命令。如果之前输入的命令有语法错误的话，切换至该命令时依旧会有语法错误，不会自动更正，更不会因为含有语法错误就不显示该命令。这种快捷键在命令方块控制台中不适用。命令历史可以跨存档调用。

在聊天栏输入命令时，`Tab` 键可用于补全命令。未输入任何命令字符的时候，使用 `Tab` 键可以看到聊天栏上出现的一个命令列表（如图 1.1），鼠标滚轮有助于翻找需要的命令。

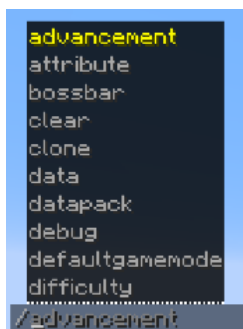


图 1.1 在聊天栏输入命令时使用 `Tab` 键

读者可以直接在命令列表中点击需要的命令，或者如图 1.2 (a) 所示，输入命令的前若干字符后使用 `Tab` 键补全。若这个命令后续还有其他参数，则也可以如图 1.2 (b) 所示用 `Tab` 键补全。



图 1.2 命令输入过程

聊天栏的字符数量被限制在 256 以内，命令开头的 `/` 也会被计入字符数。因此，聊天栏不能用于执行太长的命令。

2. 在命令方块或命令方块矿车内输入命令

命令方块控制台可输入的字符最多为 32500 个，较聊天栏的限制有很大提升。文本框长度有限，每次只能显示命令的其中一段，需要使用 `鼠标左键` 或 `←`、`→` 键移动光标调整命令显示的位置，且每次打开命令方块 GUI 时光标总显示在命令的末尾。

在命令方块或命令方块矿车内输入命令时，斜杠前缀 `/` 不是必须的。和在聊天栏中使用命令一样，当文本框中无任何内容时，下方会显示一个命令列表（如图 1.3），通过调整鼠标滚轮能够调整命令列表显示的位置，按 `Tab` 键能够在输入命令时自动补全或选择命令的部分。



图 1.3 命令方块 GUI

3. 在数据包函数文件中输入命令

这种编写方式需要使用一定的编译软件，常用的编译软件有 Windows 自带的记事本、Visual Studio Code 等。函数中的命令不能带有斜杠前缀。具体的内容可参阅《数据包》教程的描述。

4. 在服务器控制台中输入命令

5. 在带有 `run_command` 动作的点击事件的文本组件或对话框按钮中输入命令

1.2.3 权限等级与限制条件

命令功能强大、种类繁多，如果在任意情况下都能够随意使用，则很有可能会破坏玩家的游戏体验。因此，命令系统有一套专门的机制用于控制游戏内可用命令的情形，即权限等级。**权限等级 (Permission level)** 用于决定命令执行者可以使用什么样的命令。所有命令都有一个所需的权限等级，如果命令执行者没有达到该有的权限等级，则无法执行该命令。例如：`/advancement` 需要的权限等级为 2，命令方块的权限等级也为 2，因此命令方块可以执行该命令；而关闭命令的单人游戏玩家的权限为 0，所以该玩家不能执行该命令。

权限等级共分为 0、1、2、3、4 级，表罗列了 Java 版所有可用命令需要的权限等级与限制条件。除权限等级之外，一些命令还对当前的游戏世界有限制：一些命令只能在专用服务器（以下简称多人游戏）中使用，另有只能在非专用服务器（以下简称单人游戏，无论是否对局域网开放）中使用的命令，然而大部分命令都是无此限制条件的。

表 1.6 Java 版可用命令列表

命令	权限等级	限制条件	命令	权限等级	限制条件
<code>/advancement</code>	2		<code>/raid</code>	3	仅调试工具
<code>/attribute</code>	2		<code>/random</code>	0（不使用 <code>sequence</code> ）或 2（使用 <code>sequence</code> ）	
<code>/ban</code>	3	仅多人游戏	<code>/recipe</code>	2	
<code>/ban-ip</code>	3	仅多人游戏	<code>/reload</code>	2	

(续表)

命令	权限等级	限制条件	命令	权限等级	限制条件
<code>/banlist</code>	3	仅多人游戏	<code>/return</code>	2	
<code>/bossbar</code>	2		<code>/ride</code>	2	
<code>/chase</code>	0	仅调试工具	<code>/rotate</code>	2	
<code>/clear</code>	2		<code>/save-all</code>	4	仅多人游戏
<code>/clone</code>	2		<code>/save-off</code>	4	仅多人游戏
<code>/damage</code>	2		<code>/save-on</code>	4	仅多人游戏
<code>/data</code>	2		<code>/say</code>	2	
<code>/datapack</code>	2		<code>/serverpack</code>	2	仅调试工具
<code>/debug</code>	3		<code>/schedule</code>	2	
<code>/debugconfig</code>	3	仅调试工具	<code>/scoreboard</code>	2	
<code>/debugmobspawning</code>	2	仅调试工具	<code>/seed</code>	0(单人游戏) 或2(多人游戏)	
<code>/debugpath</code>	2	仅调试工具	<code>/setblock</code>	2	
<code>/defaultgamemode</code>	2		<code>/setidletimeout</code>	3	仅多人游戏
<code>/deop</code>	3	仅多人游戏	<code>/setworldspawn</code>	2	
<code>/dialog</code>	2		<code>/spawn_armor_trims</code>	2	仅调试工具
<code>/difficulty</code>	2		<code>/spawnpoint</code>	2	
<code>/effect</code>	2		<code>/spectate</code>	2	
<code>/enchant</code>	2		<code>/spreadplayers</code>	2	
<code>/execute</code>	2		<code>/stop</code>	4	仅多人游戏
<code>/experience</code>	2		<code>/stopsound</code>	2	
<code>/fill</code>	2		<code>/stopwatch</code>	2	
<code>/fillbiome</code>	2		<code>/summon</code>	2	
<code>/fetchprofile</code>	2		<code>/swing</code>	2	
<code>/forceload</code>	2		<code>/tag</code>	2	
<code>/function</code>	2		<code>/team</code>	2	
<code>/gamemode</code>	2		<code>/teammsg</code>	0	
<code>/gamerule</code>	2		<code>/teleport</code>	2	
<code>/give</code>	2		<code>/tell</code>	0	
<code>/help</code>	0		<code>/tellraw</code>	2	
<code>/item</code>	2		<code>/test</code>	2	
<code>/jfr</code>	4		<code>/tick</code>	3	
<code>/kick</code>	3		<code>/time</code>	2	

(续表)

命令	权限等级	限制条件	命令	权限等级	限制条件
<code>/kill</code>	2		<code>/title</code>	2	
<code>/list</code>	0		<code>/tm</code>	0	
<code>/locate</code>	2		<code>/tp</code>	2	
<code>/loot</code>	2		<code>/transfer</code>	3	仅多人游戏
<code>/me</code>	0		<code>/version</code>	0	
<code>/msg</code>	0		<code>/version</code>	0(单人游戏) 或 2(多人游戏)	
<code>/op</code>	3	仅多人游戏	<code>/w</code>	0	
<code>/pardon</code>	3	仅多人游戏	<code>/waypoint</code>	2	
<code>/pardon-ip</code>	3	仅多人游戏	<code>/warden_spawn_tracker</code>	2	仅调试工具
<code>/particle</code>	2		<code>/weather</code>	2	
<code>/perf</code>	4		<code>/whitelist</code>	3	仅多人游戏
<code>/place</code>	2		<code>/worldborder</code>	2	
<code>/playsound</code>	2		<code>/xp</code>	2	
<code>/publish</code>	4	仅单人游戏			

1.2.4 命令的解析 *

游戏处理命令的过程可分为**解析**和**执行**两个阶段。Minecraft使用 **Brigadier** 作为命令的解析器、派发器。

命令的实质是一个根命令节点的直接量分支,这就意味着所有的命令都是一个树状结构,命令中每一个参数都作为一个节点,而命令名作为根节点使用。显然,一个节点可能会有两种类型:字面量和变量,反映到命令文本语法中分别为字面量参数和需要自定义值的参数。

游戏在读取命令后,会首先解析根节点是否是已注册的命令,其次解析下一个参数即子节点是否可用,然后依次解析余下的节点。**Brigadier** 读取到某一个节点时,会枚举其子节点的所有可行节点,并在聊天栏或命令方块控制台内显示为可读性较强的可视化参数列表。

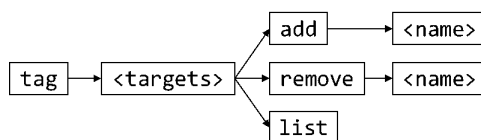


图 1.4 命令 `tag` 的所有结点和分支

以命令 `/tag` 为例,其命令树如图 1.4 所示。`tag` 是根命令,其子节点 `<target>` 是一个需要特定参数类型(这里是 `entity`)的节点,解析此节点的时候,会判断输入的参数是否为 `entity` 类型,若为否则解析异常,命令无法执行。2 级子节点是已注册的字面量 `add`、`remove` 和 `list`,解析该级节点的工作比较简单:只需读取该节点的文本是否与注册的字面量吻合。若 2 级子节点的

参数指定为 `add`、`remove`，则读取 3 级子节点 `<name>`，这个节点又是一个需要自定义的量；若 2 级子节点的参数指定为 `list`，则不能再添加后续参数。

1.2.5 命令上下文

当一条命令被执行时，该命令一定有一个调用者以及调用环境，这一系列调用者及调用环境构成的集合被称为**命令上下文 (Command context)**，或称**执行上下文 (Execution context)**、**命令源 (Command origin)**、**命令来源堆叠 (Command source stack)**。

命令上下文由以下参数构成：

1. 执行权限等级

2. **执行者 (Executor)**

由“执行者名称”和“执行者实体”两个参数构成，但执行者实体不一定存在，例如执行者为命令方块、命令方块矿车或服务端的时候。

3. **执行位置 (Execution position)**

这个参数是命令执行时所在的坐标，包含 x 、 y 、 z 三个坐标参数。

4. **执行朝向 (Execution rotation)**

这个参数是命令执行时面向的方向，包含偏航角和俯仰角两个参数。

5. **执行锚点 (Execution anchor)**

这个参数是局部坐标的原点，当执行者为实体时，这个参数可以指定执行的锚点基于实体的脚部还是眼部，因此有脚部和眼部两个可用参数。其中脚部即为原本的执行位置，眼部为原本的执行位置在 y 轴方向加上实体眼睛的高度。

6. **执行维度 (Execution dimension)**

这个参数是命令执行所在的维度，执行位置位于这个维度内。

7. 执行输出反馈

尝试执行命令会产生一定的执行效果，并在执行失败或执行成功时返回**成功次数 (Success)**和**结果 (Result)**两个返回值。其中成功次数总是为 0 或 1，结果一定为整数，遇到小数时则向下取整。下面讨论所有种类的命令执行效果：

(1) 无法解析

这种效果会在命令中存在无法解析的参数、输入的命令不完整或执行上下文不符合命令的限制条件，如执行者拥有的权限等级不够、超出游戏世界的限制时出现。此时命令没有返回值。在聊天栏或命令方块内输入的命令若无法解析，则会返回语法错误信息。在函数内的命令若无法解析，则该函数无法加载。

(2) 执行错误

出现这种效果说明命令中存在严重的漏洞。此时命令没有返回值。

(3) Void

当且仅当执行命令 `/function` 时会出现这种效果，说明 `/function` 调用了一个 void 类型的函数，没有返回值。

(4) 执行中断

当且仅当执行命令 `/execute` 时会出现这种效果，此时 `/execute` 的分支数量为 0，在 `run` 子命令执行前执行就已经中止。此时命令没有返回值。

(5) 执行失败和执行成功

当命令执行效果不是上述4种中任意一种时,才能出现执行失败或执行成功。且只有当执行效果为执行失败或执行成功时,才能返回成功次数和结果。执行失败并不意味着命令没有起作用,执行成功也不意味着命令会对游戏做出更改。

不同情况下的命令上下文列举于下表:

表 1.7 不同情况的命令上下文

情况	执行权限等级	执行者	执行位置	执行朝向	执行锚点	执行维度
玩家	视情况而定	玩家	玩家的位置	玩家的朝向	脚部	玩家所在的维度
服务器控制台	4	Server	世界出生点 方块的西北 下角顶点	水平向南	脚部	主世界
服务端	2	Server	世界出生点 方块的西北 下角顶点	水平向南	脚部	主世界
命令方块	2	命令方块	命令方块的 正中心	命令方块的 朝向	脚部	命令方块所 在维度
命令方块矿 车	2	命令方块矿 车	命令方块矿 车的位置	命令方块矿 车的朝向	脚部	命令方块矿 车所在维度
函数	可修改	该函数的调 用者	函数调用者 的位置	函数调用者 的朝向	函数调用者 的锚点	函数调用者 所在维度
告示牌	2	点击告示牌 的玩家	告示牌所在 方块正中心	水平向南	脚部	告示牌所在 维度
<code>/execute</code>	-	修饰后的执 行者	修饰后的执 行位置	修饰后的执 行朝向	修饰后的执 行锚点	修饰后的执 行维度
自定义进度	2	获得进度的 玩家	玩家的位置	玩家的朝向	脚部	玩家所在的 维度
自定义魔咒	2	魔咒作用的 实体	魔咒作用的 位置	魔咒作用实 体的朝向	脚部	魔咒作用的 维度

小提示

玩家的权限等级与其游戏模式无关,需要分情况讨论:

1. 若该玩家是服务器管理员,则他的权限等级由`ops.json`中的值决定,默认为4级;
2. 若该玩家处于启用命令的单人世界中或为启用命令的局域网世界所有者,则他的权限等级为4级;
3. 若该玩家处于启用命令的局域网世界中,则他的权限等级为4级;
4. 非上述情况者权限等级一律为0级。

函数的权限等级默认为2级,可在`server.properties`中修改。

1.3 JSON 格式

在讲述数据包和资源包这两种开发实例之前，有必要先介绍 Minecraft 使用的用于数据交换的格式，即 JSON 格式。事实上，这种数据格式的应用极其广泛，是当今互联网最通用的数据交换格式之一。在 Minecraft 中，为方便网络传输和数据存储，一些游戏内程序会被**序列化**为 JSON 格式。而对于数据包、资源包内容中的 JSON 格式而言，游戏会尝试将这些 JSON 数据**反序列化**为游戏内相应的程序。

游戏的一些数据以 `.json` 文件的格式存储在各文件夹中。这些文件有如：资源包的模型文件，数据包的进度、谓词文件，游戏版本信息文件，等等。下面展示的是金合欢木按钮的模型文件：

```
assets > minecraft > models > block > acacia_button.json
1 {
2   "parent": "minecraft:block/button",
3   "textures": {
4     "texture": "minecraft:block/acacia_planks"
5   }
6 }
```

1.3.1 JSON 数据类型

JSON (JavaScript Object Notation, JavaScript 对象表示法)是一种轻量级数据交换格式，独立于编程语言，是 JavaScript 的一个子集。其内容主要由键和值构成，即**键值对 (Name-value pair)**，这些键值对可认为是一个个**字段 (Field)**。这种格式主要有两个优点：第一，便于编写者阅读和修改；第二，由于其轻量级的特点，其对环境的依赖程度较小，因此能用于存储大量不同种类的信息。Minecraft 使用的 JSON 标准为 ECMA-404。

JSON 格式键值对的基本语法为：

```
"<键>":<值>
```

1.21

对于一个键，可以给它定义一个值。在书写时，JSON 的所有键的键名必须用**双引号**引起。若有多个键值对，则需使用逗号将这些键值对分隔开来，最后一个键值对的后面不加逗号，如：

小提示

键名的两侧必须是**英文引号**，且不接受单引号！

```
"<键>":<值>,"<键>":<值>
```

1.22

在一个 `.json` 文件中，须使用花括号 `{ }` 将所有的键值对封装包裹在一起，如：

```
{"<键>":<值>,"<键>":<值>}
```

1.23

对于值而言，每一个不同的键都需的值的类型不尽相同，比如键 `color` 可能需要的是颜色值，`bold` 可能需要的是布尔值，`text` 可能需要的是字符串，等等。JSON 一共使用六种不同的数据类型：

1. 字符串 (String)

常见的数据类型，可以包含任意字符（如空格），字符串由一对（英文）双引号定义，不接受单引号，用法举例：

```
"description": "The default data for Minecraft"
```

1.24

也可以使用中文：

```
"description": "我的世界默认数据包"
```

1.25

JSON 同时也支持 Unicode，表示方式为 `\uxxxx`，其中每一个 `x` 都为十六进制数字。例如，符号★的 Unicode 为 `U2605`，则在字符串中输入★的方式可以为：

```
"text": "\u2605"
```

1.26

这样便可以在字符串中输入一些生僻字或是在键盘上无法直接打出来的字符。但是 Minecraft 的字库是有限的，并非所有的字符都可以在 Minecraft 中显示。

2. 布尔值 (Bool)

由 `true`（真）或 `false`（假）定义，这两者是 JSON 中的字面量符号，不需要使用双引号引起，举例：

```
"bold": true
```

1.27

```
"italic": false
```

1.28

3. 数值 (Number)

由数字定义，允许使用整数、浮点数或是科学计数法表示的数，举例：

```
"min": 1.0
```

1.29

在 JSON 中使用的数值不需要注明它们的数据类型。

4. 数组 (Array, 或称为列表)

由一对方括号定义，数组中元素与元素之间使用逗号隔开，最后一个元素后不能有逗号。这些元素可以是其他的数据类型，如字符串、布尔值、数值和对象，数组中甚至能嵌套数组。在定义其他的数据类型时，需注意这些数据类型的定义方法。以下为包含了数值的数组：

```
"frames": [1, 2, 3, 4, 5]
```

1.30

下面为包含了字符串的数组，字符串均由一对双引号定义：

```
"text": ["A", "B", "C"]
```

1.31

对于数组内的元素，其数据类型不必完全一致，例如：

```
"extra": [1, {"text": "2"}, "3"]
```

1.32

5. {} 对象 (Object)

由一对花括号定义，对象内字段与字段之间使用逗号隔开，**最后一个字段后不能有逗号**。对象中可以包含其他数据类型，也可以在对象中嵌套对象。整个 `.json` 文件就可以看作是一个大的对象。在编写 JSON 的时候，通常需要用到对象嵌套对象，因此花括号一定要检查是否匹配。用法举例：

```
{
  "rolls": {
    "type": "minecraft:binomial",
    "n": 3,
    "p": 0.2
  }
}
```

1.33

6. Null

空值，作为字面量符号使用。Minecraft 基本不使用这种数据类型。

由于 JSON 为多层级结构，为了方便说明，本系列教程会使用 and Minecraft Wiki 一样的树状图来表示。例如：



对应的 JSON 为：

```
{
  "string": "这是一个字符串",
  "boolean": true,
  "number": 5,
  "array": [
    "这是数组的第一个元素，是一个字符串",
    {
      "string": "这是对象内的一个字符串"
    }
  ],
  {
    "string": "这是对象内的一个字符串"
  }
}
```

1.34

小提示

如何看懂树状图？

1. 父节点和子节点的关系会以这样的方式表示：

这是父节点
└ 这是子节点

相同层级的节点会表示为相同的缩进。

2. 对于一个字段：

field: 这是一个字段

- (1) 字段开头的 **field** 表示这个字段使用的数据类型。如果出现了多种数据类型，则表示这些数据类型均可使用。
- (2) 加粗红色的字表示这个字段的键名。
- (3) 冒号后面如果只有 **代码块**，表示此 **代码块** 是该字段使用的真实值。如果冒号后面是一段文字，则这是对于该字段的解释。如：
field: 这是对于这个字段的解释。
- (4) 如果键名有下划线，则表示这个字段是必填项：
string: 此项为必选项。

1.3.2 JSON 的转义序列

使用 JSON 字符串时，如果字符串本身的内容中含有英文引号 **"**，如一个 JSON 字段 **text** 的值需要为 **"Hello World!"**，那该如何编写 JSON 呢？若使用如下的 JSON：

```
"text": "Hello World!"
```

1.35

这样通常会产生报错，这是由于用于定义字符串的引号和值中的英文引号发生了配对从而导致了错误，因此需要使用**转义字符 (Escape character)** **** 对文本引号进行转义。转义的作用为：将被转义的字符转换成字符，被转换的引号便不再与用于定义字符串的引号发生配对。除用于转义英文引号外，反斜杠还可以用于转义反斜杠以及创造一些特定的转义序列。JSON 中可用的转义序列如下：

1. **"**，是半角双引号（英文引号）**"** 的转义方式（中文引号不需要转义）；
2. ****，是反斜杠 **** 的转义方式，已被转义的反斜杠被视为普通字符，不再具有转义作用；
3. **\b**，退格；
4. **\f**，换页；
5. **\n**，换行；
6. **\r**，回车；
7. **\t**，制表符；
8. **\u<unicode>**，用四位十六进制表示 Unicode 字符。

小提示

\b、**\f**、**\n**、**\r**、**\t**

这些特殊的转义序列能在 JSON 中使用，但这不代表这些转义序列能在相应的游戏实例中真正起作用。例如，在物品修饰器中定义物品名称时，虽然 JSON 支持输入换行符 **\n**，但物品名称本身不支持换行。

由此可以得出字段 1.35 的正确写法：

```
"text": "\"Hello World!\""
```

1.36

同样地，如果值为 `\Hello World!\`，需要对反斜杠进行转义，正确的写法为：

```
"text": "\\Hello World!\\"
```

1.37

例 1.5

判断下列 JSON 的转义是否有效，若有效，则写出其值。

```
"text": "\\Hello World\\"
```

1.38

```
"text": "\\Hello World!\\\\"
```

1.39

```
"text": "\\Hello World!\\\\"
```

1.40

```
"text": "\\Hello World!\\\\"
```

1.41

解

字段 1.38 的值出现了连续三个反斜杠 `\\` 的情况，第一个反斜杠用于转义第二个反斜杠，而第二个反斜杠不再具有转义作用；而第三个反斜杠后面没有其他转义序列，故值无效。

对于 1.39，依次检验所有反斜杠：第一个反斜杠用于转义引号，第二个反斜杠用于转义第三个反斜杠，第四个反斜杠用于转义第五个反斜杠，第六个反斜杠用于转义引号。故值为 `"\Hello World!\\"`。

字符串两端的反斜杠数量不一定需要相等，因此字段 1.40 是有效的，输出结果 `"Hello World\"`。

注意，1.41 的所有转义序列都书写正确，但第三个反斜杠后面存在一个引号，而第三个反斜杠已被第二个反斜杠转义而失去转义作用。因此用于定义字符串的引号配对混乱，该值无效。判断一个值是否有效，不仅需要有效的转义序列，还要注意字符串本身的双引号是否正确配对。

特殊情况下，JSON 字段可能会以字符串类型包含另一个需要被解析的 JSON 数据而非直接嵌套为相应类型的值，例如：

```
"content": {"text": "Hello World!"}
```

字段 `"content"` 的值类型为字符串，两端一定需要一对引号。为了防止决定字符串的引号与值中的引号发生匹配混乱，会将值中的引号进行转义：

```
"content": "{ \"text\": \"Hello World!\" }"
```

1.42

如果 `"text"` 字段的值也带有引号，如 `"Hello World!"`，则需要相应地添加反斜杠：

```
"content": "{ \"text\": \"\\\"Hello World!\\\"\" }"
```

这时如果把字段 `"content"` 写成如下的形式：

```
"content": "{ \"text\": \"\\\"Hello World!\\\"\" }"
```

1.43

现在来手动分析这个字段。把 `"content"` 的值拆出来，对所有的转义序列都去掉反斜杠。首先，键 `"text"` 两端的引号被转义，因此能够正常匹配。其次，冒号 `:` 之后、字符 `H` 之前有两个已被转义的引号；而字符感叹号 `!` 后又有两个已被转义的引号，一共有四个被转义的引号：

```
{"text": "Hello World!"}
```

1.44

所以不可避免地又发生了引号匹配混乱的情况。现在要解决的问题就是如何让这些引号不发生匹配混乱。最有效的写法就是**从里层向外层书写，每嵌套一层，就在上一层所有需要被转义的字符（如 `"` 和 `\`）前添加反斜杠**。因此，对于里层的 `{"text": "\"Hello World!\""}`，需要在所有的 `"` 和 `\` 之前都添加一个反斜杠：

```
"content": "{ \"text\": \"\\\"\\\"\\\"Hello World!\\\"\\\"\\\"\" }
```

1.45

1.45 才是正确的写法。如果更进一步, 将 1.45 封装在对象中, 让它作为另一个 `content` 的值, 这样就又增加了一层嵌套, 于是应在 1.45 的每一个 `"` 和 `\` 之前都添加一个反斜杠:

[illegible]

1.46

1.4 游戏文件

游戏的各项数据被零零散散地存放在各个游戏文件里，部分数据对于做技术性开发而言非常重要，因此有必要适当掌握游戏文件的结构。

1.4.1 常用文件格式

存储 Minecraft 数据的文件格式有很多种，下面介绍一些常见的文件格式。

1. **.txt** 文件

`.txt` 文件是非常常见的文本文件，用 Windows 自带的记事本即可打开。这种文件通常被用于存储一些简易的文本，如游戏标题画面上的闪烁标语，有时也被用于存储游戏中的设置，在这些 `.txt` 文件中更改的内容会在游戏本体上有相应的改动。有时候 `.txt` 文件也可用于记录一些自定义的、不作为游戏数据的文本。有效的 `.txt` 文件必须为无 BOM 的 UTF-8 格式。

2. .mcfunfunction 文件

`.mcfunction` 文件，即函数文件，同样必须为无 BOM 的 UTF-8 格式。函数文件可以用 Windows10 自带的记事本打开并编辑，默认的 Windows10 记事本已经为无 BOM 的 UTF-8 格式，这点从记事本页面下方的状态栏就可以看到。记事本无法指出函数中的语法错误，必须得手动检查，笔者更推荐在编译软件中打开函数文件。本教程推荐的辅助工具是  Data-pack Helper Plus (DHP)，这是编译软件  Visual Studio Code (VS Code) 的一个扩展，可在  VS Code 的应用商店中找到。 DHP 是专门用于制作 Minecraft 数据包或资源包部分文件的辅助工具，在编写数据包或资源包的过程中， 提供了高亮显示，并为部分错误的语法提供解决方案。

《数据包》教程提供了该文件格式的具体编写规范。

3. .json 和 文件

 和  文件都是使用 JSON 格式的文件。这些文件中的 JSON 格式是允许换行的，且为了美观、可读性，编写者在习惯上会在所有的  和  文件中使用换行，并使得同一层级的字段在行前缩进上保持一致。 和  文件没有专门用于注释的语法，若需要注释，则使用游戏不需要、不会被游戏识别的键，如 、。

4. .mca、 .dat、 .dat_old 和 文件

、、 和  文件均是使用 NBT 格式的文件，通常用于存储世界的全局信息和结构信息。同样地，这两类文件不能用  DHP 在编译软件内进行编辑，但可以在 NBT 编辑器内编辑，本教程推荐的编辑器为  NbtStudio。一些无法由命令进行编辑的信息可以通过  NbtStudio 修改。

5. .png 文件

 文件是图片文件，被用于存储游戏中的绝大部分图像，包括但不限于图标、游戏截图、资源包纹理。可以使用 Windows 自带的  画图、 PS 或  GIMP 处理，但需要注意  画图不支持透明背景。

6. .ogg 文件

游戏中所有的声音文件都为  格式，从外部导入声音时应注意格式转换。直接修改文件名后缀是无效的，可以使用

7. .zip 文件

压缩文件，即  文件，也是常用的文件格式，通常被用于数据包和资源包的压缩。读者可自行选择合适的压缩软件对数据包或资源包进行压缩。

8. 其他的文件格式

Minecraft 还使用其他一些文件格式，如  文件、 文件等。具体见下文的说明。

1.4.2 .minecraft 文件夹 *

 文件夹，macOS 上为   minecraft，是存储 Java 版所有游戏数据的文件夹。

对于 Windows 系统，这个文件夹默认位于   C: Users\Admin\AppData\Roaming\.minecraft，其中  AppData 文件夹一般是隐藏的，可以在文件资源管理器  工具栏，在  显示/隐藏 一项勾选  隐藏的项目 以显示这个文件夹。

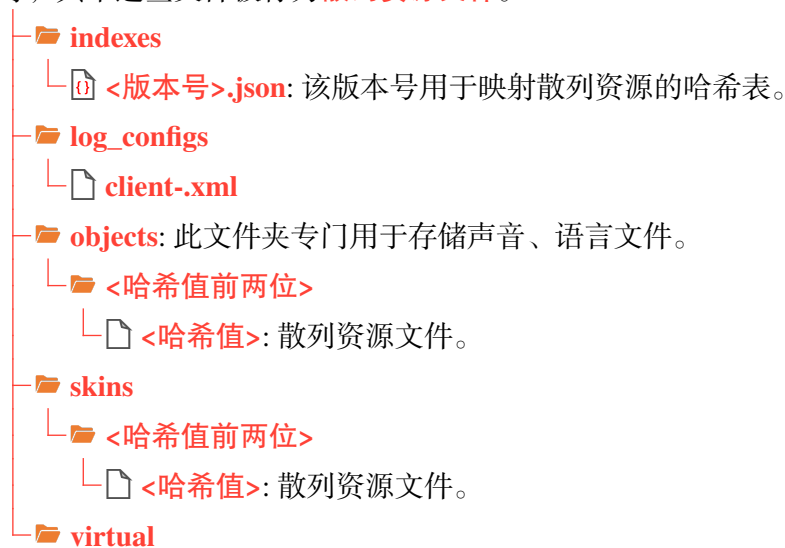
对于 Mac 系统，这个文件夹默认位于   home\用户名\Library\Application Support\.minecraft。对于 Linux 系统，这个文件夹默认位于   home\用户名\.minecraft，其中以  开头的文件夹默认是隐藏的，需要使用  Ctrl + H 切换是否可见。

第三方启动器会有其特殊的文件夹路径，具体见各启动器的设置。由官方启动器运行的游戏可以在启动器内手动修改存储路径，或者在默认存储路径处使用快捷方式重定向至自定义路径下。

随着游戏内容的增多、各种其他资源（如光影、模组）不断被下载到游戏中，  .minecraft 中的子文件（夹）可能会持续增多。鉴于无法讲到所有可能出现的文件（夹），本节仅列举原版游戏使用文件（夹）。文件结构如下所示：

.minecraft

-  **assets**: 存放原版资源包部分游戏资源的文件夹，如简体中文的语言文件、声音  文件等，其中这些文件被称为**散列资源文件**。



小提示

 **assets** 文件夹内的资源文件都是用**哈希值 (Hash value, 散列值)**加密的，以哈希表的方式映射资源位置。要查询  **assets** 内的任意一个资源文件，需按照以下步骤：

- 打开  **indexes** 文件夹，找到需要提取资源的  **<版本号>.json** 文件。其中的内容大致如下所示：

```
.minecraft > assets > indexes > <版本号>.json
1  {
2    "objects": {
3      "icons/icon_128x128.png": {
4        "hash": "b62ca8ec10d07e6bf5ac8dae0c8c1d2e6a1e3356",
5        "size": 9101
6      },
7      "icons/icon_16x16.png": {
8        "hash": "5ff04807c356f1beed0b86ccf659b44b9983e3fa",
9        "size": 781
10     },
11     "icons/icon_256x256.png": {
12       "hash": "8030dd9dc315c0381d52c4782ea36c6baf6e8135",
13       "size": 19642
14     },
15     "_comment": "后续还有很多其他资源"
16   }
17 }
```

- 用编译软件的查询功能在  **<版本号>.json** 文件中查找所需资源，记录对应  **hash** 字段的值，此即为映射该资源的哈希值。
- 打开  **objects** 文件夹，找到匹配的  **<哈希值前两位>** 文件夹，在此文件夹内找寻对应哈希值命名的文件，此即为需要找寻的资源。

例 1.6

在  **assets** 文件夹内找到 1.21.4 版本(哈希表版本号显示为  19)简体中文语言的资源文件。



在 `<19>.json` 文件中查询 `zh_cn`，可以找到一个键名为 `minecraft/lang/zh_cn.json` 的键值对：

```
"minecraft/lang/zh_cn.json": {  
  "hash": "4674523c91196e0898c24a06531f94154111f2a3",  
  "size": 459788  
}
```

1.47

这时获取到哈希值 `4674523c91196e0898c24a06531f94154111f2a3`，其前两位是 `46`。然后打开文件夹 `objects\46`，在其中找到名为 `4674523c91196e0898c24a06531f94154111f2a3` 的文件，此即为简体中文的语言文件。打开后会发现文件中的汉字均是用 Unicode 码表示的。

.minecraft

backups: 存放备份存档的文件夹。

└─ `<日期>_<时间>_<存档名称>.zip`: 一个备份存档。

bin

└─ `<随机 ID>`

└─ `.dll` 或 `.so` 文件

crash-reports: 存储游戏崩溃报告的文件夹。

└─ `crash-<日期>_<时间>-<逻辑端类型>.txt`: 一份崩溃报告（Crash Report）文件。

小提示

游戏可能会以各种原因而发生崩溃（Crash），读者可以从崩溃报告中查询崩溃原因。例如，以下是一份崩溃报告的开头部分内容：

```
.minecraft > crash-reports > crash-2024-02-08_21.25.56-server.txt
```

```
1  --- Minecraft Crash Report ---  
2  // I bet Cylons wouldn't have this problem.  
3  
4  Time: 2024-02-08 21:25:56  
5  Description: Ticking entity
```

其中第二行是“诙谐的评论”，对崩溃报告的分析没有作用。`Description` 行是崩溃原因，此处的崩溃原因是 `Ticking entity`，这种崩溃通常意味着有实体发生了错误。后文通常是崩溃的具体原因。

鉴于崩溃原因多种多样，本教程无法介绍每一种崩溃原因及其解决办法，读者可以从社区获取各种崩溃原因的解决办法或者使用 AI 分析。

.minecraft

└─ **debug**: 存储函数调试结果的文件夹。

└─ `debug-trace-<日期>_<时间>.txt`: 一份调试结果。

小提示

命令 `/debug` 可用于函数的调试，并将调试的结果以 `.txt` 的文件格式存入 `debug` 中。文件中的内容极为详细，可以以此观察函数的整个运行过程，并从中找到错误的地方。调试结果的具体内容如：

```
.minecraft > debug > debug-trace-2022-10-10_19.16.40.txt
```

```

1 [C] scoreboard players reset @s heart_of_life
2 [E] 未知的记分项'heart_of_life' [C] advancement revoke @s only plus:items/heart_of_life
3 [E] 无法撤销Mu_xian的进度plus:items/heart_of_life, 因为该玩家并未达成此进度 [C] effect clear @s absorption
4 [M] 已移除Mu_xian的伤害吸收效果
5 [R = 1] effect clear @s absorption
6 [C] scoreboard players add @s max_health 2
7 [M] 将Mu_xian的[max_health]增加了2 (现在是44)
8 [R = 44] scoreboard players add @s max_health 2
9 [C] function plus:game/attributes/max_health
10 [M] 已执行函数plus:game/attributes/max_health中的0条命令
11 [R = 0] function plus:game/attributes/max_health
12 [F] plus:game/attributes/max_health size=60
13 [C] execute as @a if score @s max_health matches 1 run attribute @s max_health base set 1 -> 0
14 [C] execute as @a if score @s max_health matches 2 run attribute @s max_health base set 2 -> 0

```

其中首行是函数的命名空间 ID。以 [C] 开头的内容为函数中的命令行；以 [E] 开头的内容指明了上一条命令行出现错误的地方；以 [M] 开头的内容说明上一条命令执行成功，并有 [R=<值>] 输出执行的结果。[F] 说明上一条命令引用了其他函数，并指出被引用的函数的大小。

📁 .minecraft

📁 **libraries**: 按 Maven 仓库的标准目录结构组织和存储的第三方库。

└─ 📁 一个第三方库。

📁 **logs**: 存储日志文件的文件夹。

└─ 🇺🇸 <日期>-<日志编号>.log.gz: 压缩文件，可使用解压软件打开。

└─ 📄 <日期>-<日志编号>.log: 日志文件。

└─ 📄 **latest.log**: 最新一次游戏或当前正在进行的游戏所生成的日志文件。

小提示

日志文件会存储游戏运行全过程的各种反馈，包括加载错误时的反馈，这些文件对游戏调试很重要。文件内容大致如下所示：

```
.minecraft > log > 2025-03-07-1.log.gz > 2025-03-07-1.log
```

```

1 [19:06:06] [Render thread/INFO]: Using default channel type
2 [19:06:06] [Render thread/INFO]: Started serving on 10000
3 [19:06:06] [Render thread/INFO]: [System] [CHAT] 本地游戏已在端口[10000]上开启
4 [19:06:41] [User Authenticator #1/INFO]: UUID of player XVExodus is 0caaf85c-27b0-4cf6-8f63-7278c55aa0e1
5 [19:06:42] [Server thread/INFO]: XVExodus[/172.20.10.5:53055] logged in with entity id 252 at (-8.221424908762929, 0.0, 8.503485026934579)
6 [19:06:42] [Server thread/INFO]: XVExodus加入了游戏
7 [19:06:42] [Render thread/INFO]: [System] [CHAT] XVExodus加入了游戏
8 [19:09:58] [Server thread/WARN]: XVExodus moved too quickly! -7.169010753171733, -1.6414613841373864, 7.738717431310306
9 [19:12:34] [Server thread/INFO]: [Mu_xian: 已将Mu_xian传送至XVExodus]
10 [19:13:54] [Server thread/INFO]: [Mu_xian: 已将Mu_xian传送至XVExodus]

```

```

11 [19:14:43] [Server thread/INFO]: [Mu_xian: 已将Mu_xian传送至XVExodus]
12 [19:16:11] [Server thread/INFO]: [XVExodus: 已将XVExodus传送至Mu_xian]
13 [19:16:37] [Render thread/INFO]: Loaded 610 advancements
14 [19:18:25] [Render thread/WARN]: Unable to play empty soundEvent:
    minecraft:entity.salmon.ambient
15 [19:18:32] [Render thread/INFO]: [System] [CHAT] 已设置重生点
16 [19:20:23] [Render thread/INFO]: Loaded 612 advancements
17 [19:22:03] [Server thread/WARN]: XVExodus moved too quickly!
    -4.311401208300595, -1.8757037979856364, 17.86772802981045
18 [19:23:27] [Server thread/INFO]: [Mu_xian: 已将Mu_xian传送至XVExodus]
19 [19:23:27] [Render thread/INFO]: Loaded 612 advancements
20 [19:24:38] [Server thread/INFO]: [XVExodus: 已将XVExodus传送至Mu_xian]
21 [19:25:06] [Server thread/INFO]: [XVExodus: 已将XVExodus传送至Mu_xian]

```

📁 .minecraft

- 📁 **resourcepacks**: 存储所有资源包的文件夹，其基本结构见 1.6，具体的制作方式将在《资源包》教程中给出。
- 📁 **saves**: 存储游戏中所有存档的文件夹，具体结构见 4.1 。
 - └ 📁 **<存档名称>**: 一个存档。
- 📁 **screenshots**: 存储 **[F2]** 截屏图片的文件夹。
 - └ 🖼️ **<日期>_<时间>.png**: 一张截屏，名称可手动修改。
- 📁 **versions**: 存储游戏不同版本游戏资源的文件夹。
 - └ 📁 **<版本号>**: 一个游戏版本，可以是正式版，也可以是快照。
 - 📄 **<版本号>.jar**: 物理客户端文件，是存放该版本号游戏源代码的地方。可以用压缩软件打开这个文件。
 - 📁 **assets**: 存放该版本号原版资源包内容的文件夹，它决定了客户端游戏内容的外观。在制作资源包时可以参考这个文件夹的结构。不含在 📁 **.minecraft\assets** 中存放的语言和声音文件。
 - 📁 **com**
 - 📁 **data**: 存放该版本号原版数据包内容的文件夹，它决定了可写注册表的内容，如进度、战利品表、配方、结构等。在制作数据包时可以参考这个文件夹的结构。
 - 📄 **flightrecorder-config.jfc**: Java Flight Recorder 配置文件，可用于 JFR 分析。
 - 📁 **META-INF**: **.jar** 文件的元数据。
 - └ 📄 **LICENSE**: 游戏许可协议。
 - └ 📄 **MANIFEST.MF**: 清单文件。
 - └ 📄 **MOJANGCS.RSA**: 用于验证 JAR 的文件。
 - └ 📄 **MOJANGCS.SF**: JAR 签名。
 - 📁 **net**: 自 25w45a 起，Mojang 发布的未经混淆的客户端其源代码均存储于该文件夹内。其中的类文件均未被混淆，可查看，是制作 Mods 的重要依据。
 - └ 📁 **minecraft**
 - └ 📄 **<名称>.class**: 一个未混淆的 Java 类文件。

 **pack.png**: 原版资源包的图标。



图 1.5 原版资源包的图标 (pack.png)

 **versions.json**: 版本信息文件，存储该版本的信息。

 **<版本号>.json**: 客户端清单文件。

 **webcache2**

 **allowed_symlinks.txt**: 信任符号链接列表文件。

 **command_history.txt**: 命令历史文件，最多只能保留 50 条记录。

 **debug.stitched_items.png**

 **debug.stitched_terrain.png**

 **hotbar.nbt**: 存储在创造模式中保存的快捷栏信息的文件，在创造模式中的快捷栏以 **C** + **<数字>** 存储，然后以 **X** + **<数字>** 调用。可以用 NBT 编辑器打开这个文件。

 **launcher_cef_log.txt**

 **launcher_entitlements.json**

 **launcher_gamer_pics.json**

 **launcher_msa_credentials.json**

 **launcher_profiles.json**: 启动器档案文件。

 **launcher_quick_play.json**: 启动器快速进入游戏存档信息文件。

 **launcher_settings.json**: 启动器配置文件。

 **launcher_skins.json**

 **launcher_ui_state.json**

 **options.txt**: 该文件存储了游戏中设定的选项，可以通过更改该文件中的内容以更改在游戏中的设置。此外一些在选项界面中不存在的设置也可以通过该文件更改。文件中内容如下所示：

.minecraft > options.txt

```
1 version:4189
2 ao:true
3 biomeBlendRadius:2
4 enableVsync:false
5 entityDistanceScaling:1.0
6 entityShadows:true
```

 **output-client.log**

 **output-server.log**

 **realms_persistence.json**: 存储 Realms 数据的文件。

 **servers.dat**: 存储玩家添加到服务器列表的多人游戏服务器的数据。

 **textures_0.png**

 **textures_1.png**

- textures_2.png
- textures_3.png
- textures_4.png
- usercache.json: 游戏为减少重复获取玩家档案信息所使用的缓存文件。

1.5 数据包

Minecraft 的命令系统虽然完善，但其功能十分有限。例如，命令没有办法直接指导游戏世界的生成；直接用命令模拟一些游戏机制也不够灵活。数据包可以看作是命令系统功能的延伸：它不仅为命令提供了程序化执行的环境，更开放了部分 API 以允许数据驱动内容。

数据包（Data pack） 允许玩家在不修改游戏代码的前提下覆盖既有的或添加自定义的游戏内容。因此，**原版技术性开发从不添加任何不在可写注册表内的游戏内容，只会用各种手段模拟这些游戏内容**。数据包本质上是一个文件夹或压缩文件。一个数据包仅对特定的游戏世界有效，它被储存在 `.minecraft\saves\<存档名称>\datapacks` 中。数据包可以是文件夹，也可以是 `.zip` 类型的压缩文件。同一个 `datapacks` 文件夹内能存放多个数据包。

数据包有两种添加方式——

1. 手动添加

直接将数据包添加至 `.minecraft\saves\<存档名称>\datapacks`。

2. 创建世界时添加数据包

在创建新的世界界面，选择 **更多**，点击 **数据包** 选项，此时会进入选择数据包窗口，类似于资源包选项的窗口，可在“可用”一栏内选用数据包，只有“已选”一栏的数据包有效，且数据包的加载顺序可以在该栏中调换。点击 **打开包文件夹** 选项后游戏会弹出一个临时的文件夹，此时可以将数据包拖入其中。



图 1.6 选择数据包窗口

当一个存档中存在多个有效的已启用数据包时,游戏会根据数据包的顺序加载其内容,这里的“有效”是指数据包有合法的元数据且数据包内无任何语法错误。已启用数据包的加载顺序存储于 `level.dat` 中。在选择数据包窗口“已选”一栏的加载顺序表现为从下到上。

若这些数据包对同种资源进行定义,则**后加载的数据包会对先加载的数据包进行覆盖**,表明**越靠后加载的数据包其优先级越高**。可使用命令 `/datapack` 查询、修改、控制这些数据包的启用或禁用, `/datapack` 所需的权限等级为 2, 以下是所有用法:

1. 启用指定数据包

```
datapack enable <name>
```

1.48

其中的参数: `<name>` (字符串 `brigadier:string`)^①——指定数据包的名称。必须使用单个词,可用引号括起整个字符串。可用字符有:

- 数字 `0123456789`;
- 大写字母 `ABCDEFGHIJKLMNOPQRSTUVWXYZ`;
- 小写字母 `abcdefghijklmnopqrstuvwxyz`;
- 下划线 `_`;
- 加号 `+`、减号 `-`;
- 点 `.`;
- 引号 `'`、`"`;
- 反斜杠 `\`。

如果用引号括起整个字符串,字符串内的同种引号与反斜杠前需要加上反斜杠 `\` 转义。

2. 禁用指定数据包

```
datapack disable <name>
```

1.49

3. 列举所有数据包

```
datapack list [available|enabled]
```

1.50

其中的参数: `[available|enabled]`——可选,若设为 `available` 则列举所有可用数据包,无论是否启用;若设为 `enabled`,则仅列举已启用数据包。默认为 `available`。

4. 启用指定的数据包,并设置其优先级为最低或最高

```
datapack enable <name> (first|last)
```

1.51

其中的参数: `(first|last)`——设置 `first` 以将该数据包的加载位次设为**首位**,因此优先级设为**最低**;设置 `last` 以将该数据包的加载位次设为**末位**,因此优先级设为**最高**。

① 括号中内容为该参数的类型及该参数类型在注册表内的命名空间 ID, 后续教程均如此。

5. 启用指定的数据包，并调整其加载优先级居于另一个数据包

```
datapack enable <name> (before|after) <existing>
```

1.52

其中的参数：(before|after)——设置 before 以将该数据包的加载放于数据包 <existing> 之前 1 位，因此优先级比数据包 <existing> 低 1 级；设置 after 以将该数据包的加载放于数据包 <existing> 之后 1 位，因此优先级比数据包 <existing> 高 1 级。

<existing>（字符串 brigadier:string）——必须为一个存在并已启用的数据包的名称。可用字符与 <name> 一致。

6. 新建一个空数据包，并设置此数据包的描述，注意，被创建的数据包默认为禁用状态

```
datapack create <id> <description>
```

1.53

其中的参数：<id>（字符串 brigadier:string）——新建数据包的名称，可用字符与上述 <name> 参数一致。

<description>（文本组件 minecraft:component）——该数据包的描述，是为元数据 pack.mcmeta 内 ” description 的值。需要是文本组件，具体写法可参照第三章。

编写数据包是一个“修改——调试——再修改——再调试”的重复过程，在既有内容的基础上对数据包做出修改并保存后，游戏不会立即识别这些修改的内容，而是依旧在修改前数据包的基础上运行原先的内容。此时需要重新加载数据包。

每次玩家进入存档（或称之为启动服务端）后，游戏都会按照加载顺序加载数据包，无论是否为首次进入存档。如果新加载的数据包内含有无效数据，则使用先前版本的数据包。在游戏过程中可以用命令 /reload 重新加载数据包而不必退出游戏并重新进入存档，这种方式被称为**热重载**。/reload 需要的权限等级为 2，其语法中不需附带任何参数：

```
reload
```

1.54

但是，使用 /reload 和重启服务端加载数据包的加载行为不同。/reload 只能用于重新加载数据包标签、函数、进度、战利品表、物品修饰器、战利品表谓词和配方这些注册项，剩余的注册项无法使用 /reload 加载，必须通过重启服务端加载。

其中维度、世界生成这些控制世界生成方式的注册项，对于已经生成的区块或维度，也无法通过世界加载重新生成这些区块或维度。因此自定义世界生成模块需要在世界首次加载时就被应用，即需要在创建世界时添加数据包而非在世界运行过程中手动添加数据包。如果确需在在世界运行过程中修改世界生成，可在确保世界仅作调试使用的前提下删除存档中需要重新生成区块的 Anvil 文件或自定义维度文件夹使其重新加载。

小提示

删除 Anvil 文件或自定义维度文件夹是比较危险的行为，可能会删掉存档中一些重要数据，删除之前需慎重考虑！

对于非数据包标签、函数、进度、战利品表、物品修饰器、战利品表谓词或配方的注册项，进入存档会出现**实验性设置（Experimental settings）**的警告，此时可点击创建备份并加载或我知道我在做什么！。但若这些注册项出现各种各样的错误（不一定是语法错误），则进入存档会出现**安全模式（Safe mode）**错误，可在官方启动器设置中打开“当《Minecraft：Java 版》启动时

输出日志”一项以随时获得错误日志，或在 `.minecraft\debug` 文件夹中获取 `.txt` 输出日志以检查存在的错误。

数据包的编写是一个极为繁琐的过程，需要不断地调试、纠错，有时甚至要对其底层逻辑进行重构。在编写数据包之前，读者应提前做好规划，对其可行性进行初步的研究，还要考虑数据包运行过程中的流畅性、玩家游玩过程中的平衡性。编写过程合理使用文件层级，对文件适当分类，以免内容混乱，降低文件可读性。

原版数据包位于 `.minecraft\versions\<版本号>\<版本号>.jar\data`，是编写自定义数据包的重要依据，读者可参考之。

1.5.1 数据包的基本结构

一个数据包拥有以下的基本结构：

`<数据包名称>` 或 `<数据包名称>.zip`

├ `<子数据包>`

└ 递归此文件夹结构

├ `data`: 数据包的主体内容。

├ `pack.mcmeta`: 数据包的元数据。

└ `pack.png`: 可选，作为数据包的图标使用。

如果该数据包以压缩文件的形式存在，则 `<数据包名称>.zip` 和 `<子数据包>`、`assets`、`pack.mcmeta`、`pack.png` 这些文件（夹）之间不要插入其他层级的文件夹。

一、元数据

`pack.mcmeta` 是数据包的**元数据（Metadata）**。所谓元数据，就是用于决定 `<数据包名称>` 或 `<数据包名称>.zip` 这个文件（夹）是否为一个数据包的基本数据。只有当元数据存在时，游戏才能识别数据包。

`pack.mcmeta` 使用 JSON 格式，其包含的内容如下所示：

文件封装

├ `pack`: 此数据包的基本信息。

└ `description`: 任意文本，使用文本组件格式，可用于对数据包的简单介绍。此段文本会出现在选项数据包中。使用 `/datapack list` 列举数据包时，将鼠标悬停于数据包名称上也会显示此文本。

└ `max_format`: 数据包最高兼容的版本号。若使用 `[I]` 形式，则内部包含两个整数，第一个为主要版本号，第二个为次要版本号。若使用 `[N]` 形式或在 `[I]` 形式内只填写一个数值，则视为只写主要版本号，次要版本号默认为次要版本号 `0x7fffffff`。

└ `min_format`: 数据包最低兼容的版本号。若使用 `[I]` 形式，则内部包含两个整数，第一个为主要版本号，第二个为次要版本号。若使用 `[N]` 形式或在 `[I]` 形式内只填写一个数值，则视为只写主要版本号，次要版本号默认为次要版本号 `0`。

N pack_format: 25w31a 以前用于指定数据包版本号的字段，现已弃用，可用于兼容旧版数据包。

   **supported_formats**: 25w31a 以前用于指定数据包版本号兼容范围的字段，现已弃用，可用于兼容旧版数据包。

若使用 `N` 形式，则精确匹配，效果与 `N pack_format` 一致

若使用 `[ ]` 形式，则内部包含两个整数，第一个为最低兼容的版本号，第二个为最高兼容的版本号

若使用 `{ }` 形式，则有以下字段：

N max inclusive: 最高兼容的版本号。

N min inclusive: 最低兼容的版本号。

features: 可选，用于启用实验性内容，若指定该键，则数据包必须在创建世界时添加。

- **enabled**: 启用实验性内容数据包的列表。

☞ 一个实验性内容数据包的命名空间 ID，当前版本可用值有 `minecraft:trade_rebalance`（村民交易平衡性调整）、`minecraft:redstone_experiments`（红石实验性内容）和 `minecraft:minecart_improvements`（矿车改进）。

- filter**: 可选，用于指定在数据包加载列表中优先级低于该包的数据包内要禁用的内容。

- **[] block**: 禁用内容列表。

{ } 一项被禁用的内容。

namespace: 要禁用的命名空间, 若省略则禁用所有命名空间, 可使用正则表达式。

path: 要禁用的资源路径，若省略则禁用所有路径，可使用正则表达式。

- **{}** **overlays**: 可选，用于子数据包的识别。

- entries**: 可用子数据包的列表。

{} 一个子数据包。

” **directory**: 该子数据包相对于主数据包根目录的路径。允许使用的字符有：小写字母、**0123456789**、**_** 和 **-**。

[N][I] max_format: 该子数据包最高兼容的版本号。若使用 **[I]** 形式，则内部包含两个整数，第一个为主要版本号，第二个为次要版本号。若使用 **[N]** 形式或在 **[I]** 形式内只填写一个数值，则视为只写主要版本号，次要版本号默认为次要版本号 **0x7fffffff**。

[N][n] min_format: 该子数据包最低兼容的版本号。若使用 **[n]** 形式，则内部包含两个整数，第一个为主要版本号，第二个为次要版本号。若使用 **[N]** 形式或在 **[n]** 形式内只填写一个数值，则视为只写主要版本号，次要版本号默认为次要版本号 **0**。

formats: 25w31a 以前用于指定子数据包版本号兼容范围的字段, 现已弃用, 可用于兼容旧版数据包。

若使用 **N** 形式，则精确匹配

若使用 `[1]` 形式，则内部包含两个整数，第一个为最低兼容的版本号，第二个为最高兼容的版本号

若使用 `{ }` 形式，则有以下字段：

N max inclusive: 最高兼容的版本号。

 **min_inclusive**: 最低兼容的版本号。

例如，下面是 1.21.11 版本的一个标准  `pack.mcmeta` 文件：

```
pack.mcmeta
1 {
2   "pack": {
3     "description": "The default data for Minecraft",
4     "max_format": 94.1,
5     "min_format": 94.1,
6   }
7 }
```

二、数据包版本号

元数据中有一个很重要的参数：**数据包版本号 (Data pack format)**，这是一个用于区分不同版本数据包的参数。每当 Mojang 对数据包做出修改时，版本号都会发生变动。在 1.19.4 以前，数据包版本号一般一个大版本变更一次；自 1.19.4 起，由于 Mojang 对技术性开发的更新变得频繁，版本号一般每个快照变更一次。数据包应当使用其所在游戏版本的版本号，由于 Mojang 对数据包的改动可能是颠覆性的，版本号不对应可能会出现错误。

1.21.8 以前的版本号均为整数，例如，1.21.8 的数据包版本号为 81，25w31a 是 1.21.9 的快照，其引入了**次要版本号 (Minor versions)**的概念，原先的整数形式的版本号为**主要版本号 (Major versions)**，25w31a 是第一个使用此版本号格式的版本，是为 82.0。同一个主版本号内的数据包可以向下兼容，例如 83.1 的数据包可以兼容 83.0 的数据包。

下表罗列了所有主版本使用的数据包版本号，不包括快照版本。包含快照版本的数据包版本号参考附录 I.1 中的表 I.1。

表 1.8 数据包版本号

游戏版本	数据包版本号	游戏版本	数据包版本号
1.13 ~ 1.14.4	4	1.20.3 ~ 1.20.4	26
1.15 ~ 1.16.1	5	1.20.5 ~ 1.20.6	41
1.16.2 ~ 1.16.5	6	1.21 ~ 1.21.1	48
1.17 ~ 1.17.1	7	1.21.2 ~ 1.21.3	57
1.18 ~ 1.18.1	8	1.21.4	61
1.18.2	9	1.21.5	71
1.19 ~ 1.19.3	10	1.21.6	80
1.19.4	12	1.21.7 ~ 1.21.8	81
1.20 ~ 1.20.1	15	1.21.9 ~ 1.21.10	88.0
1.20.2	18	1.21.11	94.1

游戏允许编写者在元数据内指定数据包版本号的区间以使数据包兼容多个版本。但由于在不同版本中  `pack.mcmeta` 本身的格式也会发生变化，数据包版本号需要进行校验。不过，这个校验仅仅作作为“门槛”，数据包能否运行取决于其实际内容，而非元数据声明。在 26.1 以前，校

验失败会现实“已损坏或不兼容”；而在 26.1 以后，校验失败会直接认为元数据无效，从而不识别此数据包。

校验规则以 25w31a（1.21.9）为分水岭实行“新旧双轨制”，如下表所示：

表 1.9 数据包版本号校验规则

配置要求	元数据中必须使用的字段	元数据中可以使用的字段	元数据中不能使用的字段
仅适用于 25w31a之前	N pack_format	N 0 0 supported_formats 若使用，则此区间必须包含 N pack_format 的值，且最大值不能低于 16，因为此字段是在 23w31a 引入的	N max_format 和 N min_format
仅适用于 25w31a及之后	N max_format 和 N min_format	-	N pack_format 和 N 0 0 supported_formats
同时适用于 25w31a之前及之后	同时指定 N pack_format 、 N 0 0 supported_formats 、 N max_format 和 N min_format ，且必须满足以下要求： 区间验证： N pack_format 必须落在兼容区间内。 对最低版本号的验证： N 0 0 supported_formats 的下限必须与 N min_format 相等。 对最高版本号的验证，以下两种方案二选一： N 0 0 supported_formats 的上限与 N max_format 相等。 N 0 0 supported_formats 的上限固定为 81，此时最高版本号由 N max_format 决定。		

例 1.7

现需要编写一个适用于 1.20.5 至 1.21.11 的数据包，尝试编写其元数据。

解

查表 1.8，1.20.5 的版本号为 41，1.21.11 的版本号为 94.1。因为此数据包同时适用于 25w31a 之前及之后的版本，根据表 1.9，需要同时指定

N

pack_format

、

N

0

0

supported_formats

、

N

max_format

 和

N

min_format

。

首先，

N

pack_format

 的值需要在 41 和 94.1 之间，此处直接写 41。其次，可将

N

0

0

supported_formats

 的下限调整为与

N

min_format

 一致，上限设为 81，

N

max_format

 设为 94.1。故元数据可写为：

pack.mcmeta

```

1  {
2    "pack": {
3      "description": "元数据"
4      "max_format": [ 94, 1 ],
5      "min_format": [ 41, 0 ],
6      "pack_format": 41,
7      "supported_formats": [ 41, 81 ]
8    }
9  }

```

三、子数据包

子数据包会在当前主数据包的基础上添加内容, 同时也会覆盖主数据包相同路径的文件。不过, 仅在主数据包文件夹的子层级添加一个数据包并不会让主数据包识别到这个子数据包, 应当在元数据的 `overlays` 中配置。配置方式见上文的数据格式。注意, 由于 `directory` 字段允许包含的字符仅有小写字母、`0123456789`、`_` 和 `-`, 那么子数据包的名称及相对路径也只能包含这些字符。

例 1.8

一个版本号为 88.0 的数据包需要使用 `jigsaw_marker_v1.0` 这个数据包作为其子包, 尝试配置子数据包。

解

首先, 将数据包 `jigsaw_marker_v1.0` 移入主数据包, 文件夹结构如下:

主数据包

```

├── jigsaw_marker_v1.0
├── data
└── pack.mcmeta

```

其次, 在 `pack.mcmeta` 中做如下配置:

pack.mcmeta

```

1  {
2    "overlays": {
3      "entries": [
4        {
5          "directory": "jigsaw_marker_v1.0",
6          "max_format": [ 88, 0 ],
7          "min_format": [ 88, 0 ]
8        }
9      ]
10   },
11   "pack": {
12     "description": "数据包",

```



```
13     "max_format": [ 88, 0 ],
14     "min_format": [ 88, 0 ]
15 }
16 }
```

子数据包的版本号也需要进行校验，校验规则与主数据包的校验规则类似，如下表所示：

表 1.10 子数据包版本号校验规则

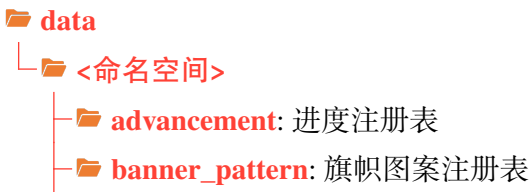
子数据包的配置要求	overlays 必须使用的字段	overlays 不能使用的字段
仅适用于 25w31a 之前	<code>formats</code>	<code>max_format</code> 和 <code>min_format</code>
仅适用于 25w31a 及之后	<code>max_format</code> 和 <code>min_format</code> 注意：如果主数据包适用于 25w31a 之前，则必须保留 <code>formats</code>	-
同时适用于 25w31a 之前及之后	同时指定 <code>formats</code> 、 <code>max_format</code> 和 <code>min_format</code> ，且必须满足以下要求： 对最低版本号的验证： <code>formats</code> 的下限必须与 <code>min_format</code> 相等。 对最高版本号的验证，以下两种方案二选一： 1. <code>formats</code> 的上限与 <code>max_format</code> 相等。 2. <code>formats</code> 的上限固定为 81，此时最高版本号由 <code>max_format</code> 决定。	-

四、data 文件夹

`data` 文件夹是存储数据包主要内容的文件夹，下面展示了 `data` 文件夹的基本结构，这些文件（夹）就是表 1.2 所展示的可写注册表以及其他一些配置项的路径，它们不一定必须全部存在，游戏会根据指定的资源路径读取可写注册表中的内容，若相应的可写注册表需要存在，则必须有正确的资源路径和文件（夹）名称。

一个 `data` 文件夹中可以存在多个不同的命名空间，而命名空间 `minecraft` 下的内容会覆盖原版游戏内容。

在命名空间下的这些文件夹中，`function` 内的文件使用 `.mcfunction` 格式，`structure` 内的文件使用 `.nbt` 格式，除 `datapacks` 外其余文件夹内的文件一律使用 `.json` 格式，编写时务必使用正确的编译软件打开它们。此外，除了 `datapacks` 的文件夹内部都是可以自由指定资源路径的，那么在各游戏资源的命名空间 ID 中就可以使用这些资源路径。可参考例 1.1。



- **cat_variant**: 猫的变种注册表
- **chat_type**: 聊天类型注册表
- **chicken_variant**: 鸡的变种注册表
- **cow_variant**: 牛的变种注册表
- **damage_type**: 伤害类型注册表
- **datapacks**: 内置数据包，均为功能数据包
- **dialog**: 对话框注册表
- **dimension**: 维度注册表
- **dimension_type**: 维度类型注册表
- **enchantment**: 魔咒注册表
- **enchantment_provider**: 魔咒提供器注册表
- **frog_variant**: 青蛙的变种注册表
- **function**: 函数
- **instrument**: 山羊角乐器注册表
- **item_modifier**: 物品修饰器注册表
- **jukebox_song**: 唱片机曲目注册表
- **loot_table**: 战利品表注册表
- **painting_variant**: 画的变种注册表
- **pig_variant**: 猪的变种注册表
- **predicate**: 谓词注册表
- **recipe**: 配方注册表
- **structure**: 结构
- **tags**: 数据包标签
- **test_environment**: 测试环境注册表
- **test_instance**: 测试实例注册表
- **timeline**: 时间线注册表
- **trade_set**: 交易集注册表
- **trial_spawner**: 试炼刷怪笼配置注册表
- **trim_material**: 盔甲纹饰材料注册表
- **trim_pattern**: 盔甲纹饰图案注册表
- **villager_trade**: 村民交易注册表
- **wolf_variant**: 狼的变种注册表
- **world_clock**: 世界时钟注册表
- **worldgen**: 世界生成模块
 - **biome**: 生物群系注册表
 - **configured_carver**: 已配置的雕刻器注册表
 - **configured_feature**: 已配置的地物注册表

- **density_function**: 密度函数注册表
- **flat_level_generator_preset**: 超平坦世界生成预设注册表
- **multi_noise_biome_source_parameter_list**: 多噪声参数列表注册表
- **noise**: 噪声注册表
- **noise_settings**: 噪声设置注册表
- **placed_feature**: 已放置的地物注册表
- **processor_list**: 处理器列表注册表
- **structure**: 已配置的结构地物注册表
- **structure_set**: 结构集注册表
- **template_pool**: 结构池注册表
- **world_preset**: 世界预设注册表
- **zombie_nautilus_variant**: 僵尸鹦鹉螺变种注册表

1.5.2 实验性内容 *

自 22w42a 起, Minecraft 部分更新内容会以内置数据包的形式加入游戏, 使玩家可以提前体验这些内容。这些内容被称为**实验性内容 (Experiments)**。在当前版本 (26.1), 可用的实验性内容有三项: 村民交易平衡性调整、红石实验性内容和矿车改进。

所有实验性内容都是**功能数据包 (Feature datapack)**的形式, 启用这些实验性内容的方式有两种: 一是在选择数据包窗口选择功能数据包; 二是创建新世界时点击实验性内容从而操控这些实验性内容的开关。

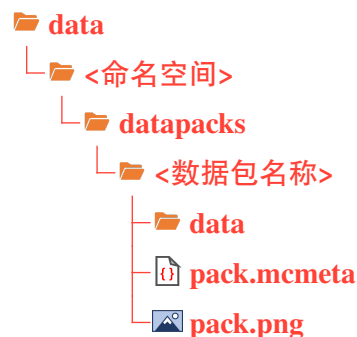


图 1.7 试验性内容窗口

数据包无法直接修改游戏代码, 但功能数据包似乎“注册”了新的游戏内容, 功能数据包是否有其独特的行为? 其实, 实验性内容分为硬编码内容和数据驱动内容, 其中的硬编码内容被称为特定组别的**功能元素 (Feature Element)**。**功能开关 (Feature Flag)**则用于启用或禁用这些功能元素。当一个功能数据包被启用时, 元数据中 `enabled` 字段启用, 相应的功能开关被打开, 其中的功能元素就能在游戏中正常运行。若一个功能数据包被关闭, 则相应的功能元素被过滤。

实验性内容除了可在新创建存档时启用或禁用外，也可以通过修改  `level.dat` 中的 `enabled_features` 字段以在已创建的存档中启用或禁用。相应格式见 4.1 节的描述。

实验性内容中的数据驱动部分则交由数据包完成，这些功能数据包作为子数据包存在，存储于 `datapacks` 文件夹，相应文件结构如下：



其中的 `<数据包名称>` 即为一个功能数据包，其结构与正常数据包无异，也需要有元数据。但这些数据包无法由自定义的数据包添加，仅由游戏内部提供，仅作参考。

1.5.3 数据包标签定义格式

小节 1.1.4 已提出了 **数据包标签 (Tags in data packs)** 的概念，它是将游戏资源分类的一种办法。玩家不仅可以使使用原版数据包既有的数据包标签，也可以新增或删改原有的标签。数据包标签模块在数据包内的文件结构如下：



每个注册表下的数据包标签只允许引用该注册表内的游戏资源。在数据包标签中所有可用的注册表如下表所示：

表 1.11 数据包标签可用注册表

资源类型	注册表 / 配置的路径	资源类型	注册表 / 配置的路径
旗帜图案	<code>banner_pattern</code>	画的变种	<code>painting_variant</code>
方块	<code>block</code>	兴趣点类型	<code>point_of_interest_type</code>
伤害类型	<code>damage_type</code>	药水效果	<code>potion</code>
魔咒	<code>enchantment</code>	时间线	<code>timeline</code>
实体类型	<code>entity_type</code>	村民交易	<code>villager_trade</code>
流体	<code>fluid</code>	自定义世界生成（生物群系）	<code>worldgen\biome</code>
函数	<code>function</code>	自定义世界生成（超平坦预设）	<code>worldgen\flat_level_generator_preset</code>
游戏世界	<code>game_event</code>	自定义世界生成（结构）	<code>worldgen\structure</code>

(续表)

资源类型	注册表 / 配置的路径	资源类型	注册表 / 配置的路径
山羊角乐器	<code>instrument</code>	自定义世界生成（世界预设）	<code>worldgen\world_preset</code>
物品	<code>item</code>		

对于一个特定的数据包标签 `data\<命名空间>\tags\<注册名>\<标签>.json`，引用它的方式是带 `#` 号的命名空间 ID，其中 `<注册表>` 层级不书写：

#<命名空间>:<标签>

1.55

若命名空间不写，则默认使用 `minecraft` 内的数据包标签。`<注册表>` 层级下可以添加一定的路径。例如，`data\<命名空间>\tags\<注册名>\<路径>\<标签>.json` 的引用格式为

#<命名空间>:<路径>/<标签>

1.56

所有的数据包标签 `.json` 文件，无论其所属的注册表，一律有如下的格式：

文件封装

replace: 指定此标签的引用是否覆盖较低优先级数据包中同命名空间内的同名标签，若设为 `true`，则忽略较低优先级数据包内的引用；若设为 `false`，则此标签内的引用作为对同名标签内引用内容的补充。默认为 `false`。

values: 此标签引用的游戏资源，必须引用同类型的游戏资源。可以引用游戏资源本身，也可以引用其他的同类型数据包标签。

“ ” 一个被引用游戏资源的命名空间 ID。

“ ” 一个被引用的同类型数据包标签，需要带 `#` 号。

“ ” 引用游戏资源的完整格式。

“ ” **id**: 一个被引用游戏资源的命名空间 ID 或同类型数据包标签。

required: 用 `false` 表示该条目是可选的，若该条目 `id` 所述内容不存在，则不会使标签加载失败。默认为 `true`。

例 1.9

原版存在一个名为 `#air` 的方块标签，有三种方块属于这个标签：空气、洞穴空气和虚空空气，试编写这个标签。

解

这个标签没有使用命名空间，默认命名空间为 `minecraft`。首先确定这个标签的文件路径：

```

data
├── minecraft
│   ├── tags
│   │   └── block
│   │       └── air.json

```

标签内容如下所示：

minecraft > tags > block > air.json

```
1 {
2   "values":[
3     "minecraft:air",
4     "minecraft:void_air",
5     "minecraft:cave_air"
6   ]
7 }
```

例 1.10

有一个生物群系标签如下所示：



- (1) 写出该标签的引用方式。
- (2) 同个数据包内已有如下的生物群系，尝试在该标签中引用这些生物群系。



解

- (1) `the_backrooms` 是命名空间，`tags` 是标签的路径，`worldgen` 和 `biome` 是标签内注册表的路径，因此该标签的引用方式为 `#the_backrooms:level_37`。
- (2) `the_backrooms` 是命名空间，`worldgen` 和 `biome` 是注册表的路径，因此这些生物群系的命名空间 ID 分别为 `the_backrooms:level_37/normal`、`the_backrooms:level_37/deep_water` 和 `the_backrooms:level_37/dark_zone`，现在在标签内引用它们：

data > the_backrooms > tags > worldgen > biome > level_37.json

```
1 {
2   "values":[
```



```

3     "the_backrooms:level_37/normal",
4     "the_backrooms:level_37/deep_water",
5     "the_backrooms:level_37/dark_zone"
6 ]
7 }

```

原版的一些数据包标签具有特殊的行为。例如，实体标签 `#arthropod` 引用的实体均被视为节肢生物，会受到节肢杀手魔咒的作用，如果往标签中添加新的实体，则新使用的实体也会被视为节肢生物。所有的原版数据包标签列举于附录 I.3。

1.6 资源包

为了搭配所制作的小游戏、冒险地图或原版模组，使得游戏的观感和体验感提高，作者通常会系统性地改变游戏的外观，例如方块的纹理、外形等。于是就需要使用资源包。

资源包 (Resource pack) 允许玩家在不修改源代码的情况下自定义纹理、模型、声音、语言等外观性资源，对客户端有效。资源包本质上是一个文件夹或压缩文件，被储存在 `.minecraft/resourcepacks` 中，同一个 `resourcepacks` 文件夹内能存放多个资源包。选项资源包窗口“可用”一栏仅罗列 `resourcepacks` 文件夹内的所有的有效资源包，可在这一栏选用资源包，只有位于“已选”一栏的资源包有效。点击打开包文件夹后可以手动添加资源包。



图 1.8 选择资源包窗口

在游戏中可以同时使用多个资源包，这些资源包按照“已选”一栏中从下到上的顺序依次加载，资源包的加载顺序可以在该栏中调换。和数据包类似，若这些资源包对同种资源的外观进行定义，则**后加载的资源包会对先加载的资源包进行覆盖，表明越靠后加载的资源包其优先级越高。**

资源包也可以以压缩包的形式存放在存档文件夹中，这时资源包作为**世界指定资源包 (World specific resources)** 使用，仅在当前存档起作用，且会使该资源包的优先级设为最高，并

将已定义的资源外观覆盖选项资源包中已启用的资源包。有效的世界指定资源包必须以  `resources.zip` 为压缩文件名。

在服务器中，管理员可在 `server.properties` 中的 `resource-pack` 一项指定一个 `.zip` 文件的下载地址，从而将此 `.zip` 文件设为服务器的指定资源包。若启用，则游戏会强制将该资源包设为最顶层资源包且无法更改位置。

原版资源包位于 `.minecraft\versions\<版本号>\<版本号>.jar\assets`，是制作自定义资源包的重要依据，读者可参考之。

1.6.1 资源包的基本结构

一个资源包拥有以下的基本结构：

 **<资源包名称>** 或  **<资源包名称>.zip**

-  **<子资源包>**
 - └ 递归此文件夹结构
-  **assets**: 资源包的主体内容。
-  **pack.mcmeta**: 资源包的元数据。
-  **pack.png**: 可选，作为资源包的图标使用。

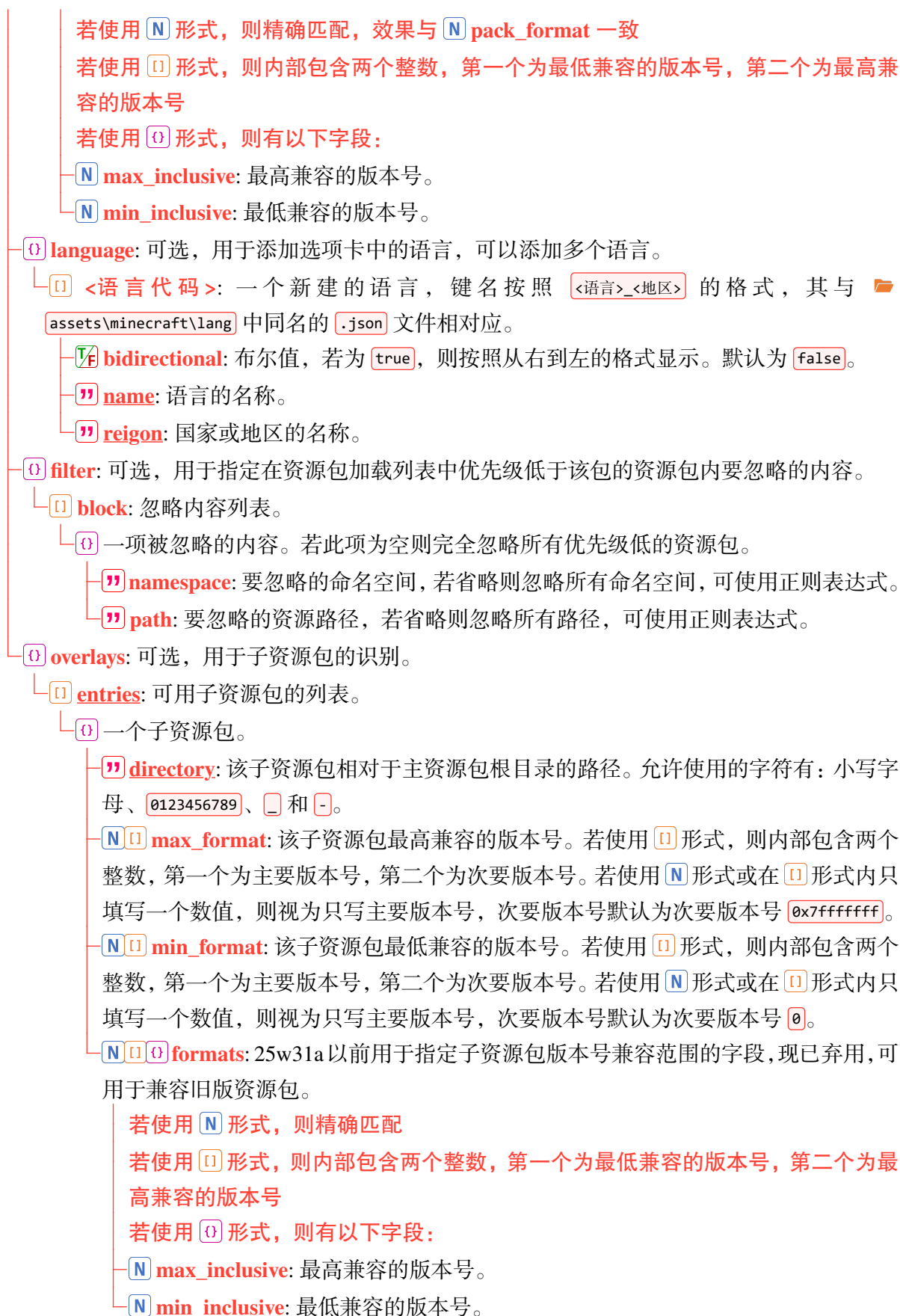
如果该资源包以压缩文件的形式存在，则  `<资源包名称>.zip` 和  `<子数据包>`、 `assets`、 `pack.mcmeta`、 `pack.png` 这些文件之间不要插入其他层级的文件夹。若该资源包为世界指定资源包，则名称一定为  `resources.zip`。

资源包中  `assets` 用于存放各种资源文件， `pack.mcmeta` 作为资源包的**元数据 (Metadata)** 使用。和数据包一样，所谓元数据，就是用于决定  `<资源包名称>` 或  `<资源包名称>.zip` 这个文件(夹)是否为一个资源包，只有当元数据存在时，游戏才能识别资源包。

 `pack.mcmeta` 包含的内容如下所示：

 文件封装

-  **pack**: 此资源包的基本信息。
 -    **description**: 任意文本，使用文本组件格式，可用于对资源包的简单介绍。此段文本会出现在选项资源包中。
 -   **max_format**: 资源包最高兼容的版本号。若使用  形式，则内部包含两个整数，第一个为主要版本号，第二个为次要版本号。若使用  形式或在  形式内只填写一个数值，则视为只写主要版本号，次要版本号默认为次要版本号 `0x7fffffff`。
 -   **min_format**: 资源包最低兼容的版本号。若使用  形式，则内部包含两个整数，第一个为主要版本号，第二个为次要版本号。若使用  形式或在  形式内只填写一个数值，则视为只写主要版本号，次要版本号默认为次要版本号 `0`。
 -  **pack_format**: 25w31a 以前用于指定资源包版本号的字段，现已弃用，可用于兼容旧版资源包。
 -    **supported_formats**: 25w31a 以前用于指定资源包版本号兼容范围的字段，现已弃用，可用于兼容旧版资源包。



例如，下面是 1.21.11 版本的一个标准 **O** `pack.mcmeta` 文件：

```

pack.mcmeta
1 {
2   "pack": {
3     "description": "The default look and feel of Minecraft",
4     "max_format": 75.0,
5     "min_format": 75.0,
6   }
7 }

```

和数据包一样，**资源包版本号（Resource pack format）**是一个用于区分不同版本数据包参数。每当 Mojang 对资源包做出修改时，版本号都会发生变动。同样，1.21.8 以前的版本号均为整数，例如，1.21.8 的数据包版本号为 64，25w31a 引入了次要版本号，是为 65.0。同一个主版本号内的资源包可以向下兼容，例如 65.2 的资源包可以兼容 65.0 的资源包。

下表罗列了所有主版本使用的资源包版本号，不包括快照版本。包含快照版本的资源包版本号参考附录 I.1 中的表 I.1。

表 1.12 资源包版本号

游戏版本	资源包版本号	游戏版本	资源包版本号
1.6.1 ~ 1.8.9	1	1.20.2	18
1.9 ~ 1.10.2	2	1.20.3 ~ 1.20.4	22
1.11 ~ 1.12.2	3	1.20.5 ~ 1.20.6	32
1.13 ~ 1.14.4	4	1.21 ~ 1.21.1	34
1.15 ~ 1.16.1	5	1.21.2 ~ 1.21.3	42
1.16.2 ~ 1.16.5	6	1.21.4	46
1.17 ~ 1.17.1	7	1.21.5	55
1.18 ~ 1.18.2	8	1.21.6	63
1.19 ~ 1.19.2	9	1.21.7 ~ 1.21.8	64
1.19.3	11	1.21.9 ~ 1.21.10	69.0
1.19.4	12	1.21.11	75.0
1.20 ~ 1.20.1	15		

资源包的版本号同样具有校验规则，也以 25w31a (1.21.9) 为分水岭实行“新旧双轨制”，如下表所示：

表 1.13 资源包版本号校验规则

配置要求	元数据中必须使用的字段	元数据中可以使用的字段	元数据中不能使用的字段
仅适用于 25w31a 之前	 <code>pack_format</code>	   <code>supported_formats</code> 若使用，则此区间必须包含  <code>pack_format</code> 的值，且最大值不能低于 16，因为此字段是在 25w31a 引入的	 <code>max_format</code> 和  <code>min_format</code>
仅适用于 25w31a 及之后	 <code>max_format</code> 和  <code>min_format</code>	-	 <code>pack_format</code> 和  <code>supported_formats</code>

(续表)

配置要求	元数据中必须使用的字段	元数据中可以使用的字段	元数据中不能使用的字段
同时适用于 25w31a 之前及之后	同时指定 <code>N</code> <code>pack_format</code> 、 <code>N</code> <code>supported_formats</code> 、 <code>N</code> <code>max_format</code> 和 <code>N</code> <code>min_format</code> ，且必须满足以下要求： 区间验证： <code>N</code> <code>pack_format</code> 必须落在兼容区间内。 对最低版本号的验证： <code>N</code> <code>supported_formats</code> 的下限必须与 <code>N</code> <code>min_format</code> 相等。 对最高版本号的验证，以下两种方案二选一： 1. <code>N</code> <code>supported_formats</code> 的上限与 <code>N</code> <code>max_format</code> 相等。 2. <code>N</code> <code>supported_formats</code> 的上限固定为 64，此时最高版本号由 <code>N</code> <code>max_format</code> 决定。	-	-

和子数据包一样，子资源包的版本号也需要进行校验，校验规则与主资源包的校验规则类似，如下表所示：

表 1.14 子资源包版本号校验规则

子资源包的配置要求	<code>O</code> overlays 必须使用的字段	<code>O</code> overlays 不能使用的字段
仅适用于 25w31a 之前	<code>N</code> <code>formats</code>	<code>N</code> <code>max_format</code> 和 <code>N</code> <code>min_format</code>
仅适用于 25w31a 及之后	<code>N</code> <code>max_format</code> 和 <code>N</code> <code>min_format</code> 注意：如果主数据包适用于 25w31a 之前，则必须保留 <code>N</code> <code>formats</code>	-
同时适用于 25w31a 之前及之后	同时指定 <code>N</code> <code>formats</code> 、 <code>N</code> <code>max_format</code> 和 <code>N</code> <code>min_format</code> ，且必须满足以下要求： 对最低版本号的验证： <code>N</code> <code>formats</code> 的下限必须与 <code>N</code> <code>min_format</code> 相等。 对最高版本号的验证，以下两种方案二选一： 1. <code>N</code> <code>formats</code> 的上限与 <code>N</code> <code>max_format</code> 相等。 2. <code>N</code> <code>formats</code> 的上限固定为 64，此时最高版本号由 <code>N</code> <code>max_format</code> 决定。	-

例 1.11

判断以下的资源包元数据是否符合版本号的校验要求。

pack.mcmeta

```

1  {
2    "pack": {
3      "pack_format": 55,
4      "supported_formats": {
5        "min_inclusive": 55,
6        "max_inclusive": 64
7      },
8      "description": {
9        "translate": "森罗物语: 装饰"
10     },
11     "min_format": [ 55, 0 ],
12     "max_format": [ 1000, 0 ]
13   }
14 }

```

解

`N` `pack_format`、`N` `supported_formats`、`N` `max_format` 和 `N` `min_format` 四个字段同时存在，说明此资源包同时适用于 25w31a 之前及之后。

首先进行区间验证：`N` `pack_format` 的值在兼容区间内。

其次对最低版本号进行验证：`N` `supported_formats` 的下限与 `N` `min_format` 相等。

最后对最高版本号进行验证，`N` `supported_formats` 的上限与 `N` `max_format` 不相等。再检查，`N` `supported_formats` 的上限为 64，`N` `max_format` 是一个大于 64 的值。

故此资源包的版本号编写正确。

下面展示了 `assets` 文件夹的基本结构，这些文件（夹）不一定必须全部存在，游戏会根据指定的资源路径读取资源包中的内容，因此若相应的资源文件（夹）需要存在，则必须有正确的资源路径和文件（夹）名称。

assets

<命名空间>

- `atlases`: 纹理图集
- `blockstates`: 方块状态映射
- `equipment`: 装备模型
- `font`: 字体
- `items`: 物品模型映射
- `lang`: 语言
- `models`: 烘焙模型
- `particles`: 粒子纹理定义
- `post_effect`: 后处理管线
- `sounds`: 声音
- `shaders`: 着色器
- `texts`: 文本

- 📁 **texture**: 纹理
- 📁 **waypoint_style**: 路径点样式
- 📄 **gpu_warnlist.json**: GPU 警告列表
- 📄 **regional_compliances.json**: 地区合规性警告
- 📄 **sounds.json**: 声音事件定义文件

1.6.2 GPU 警告列表 *

资源包负责游戏的画面渲染。部分计算机显卡太旧、驱动版本不匹配，或者 GPU 属于某些已知会造成游戏崩溃的型号，因此资源包内存在 📄 **gpu_warnlist.json** 这个用于自检硬件兼容性的配置文件。其格式如下所示：

📄 文件封装

- 📄 **renderer**: 需要显示渲染器警告的渲染器名称（显卡型号）。
 - ” 一个渲染器名称，使用正则表达式。
- 📄 **version**: 需要显示渲染器版本警告的渲染器版本（通常为显卡驱动的版本号）。
 - ” 一个渲染器版本，使用正则表达式。
- 📄 **vendor**: 需要显示渲染器厂商警告的渲染器生产厂商。
 - ” 一个渲染器厂商，使用正则表达式。

例如，原版资源包的 📄 **gpu_warnlist.json** 文件内容如下：

```
assets > minecraft > gpu_warnlist.json
1 {
2   "renderer" : [],
3   "version" : [
4     "\\bMetal\\b"
5   ],
6   "vendor" : []
7 }
```

这份配置专门针对 macOS 用户。当渲染器版本信息中包含独立的 Metal 单词时，会触发警告。因为 macOS 使用苹果系统独特的图形接口 Metal，而不使用 OpenGL。

警告页面会在玩家开启游戏极佳画质时出现。不过，这个页面以及 GPU 警告列表的配置都只是警告机制，并不是禁止硬件被匹配到的计算机运行 Minecraft。游戏具体的运行情况取决于硬件本身。例如，以下的配置文件可以使得持有 RTX 4060 显卡的计算机显示警告页面，显示的页面如图 1.9 所示。

```
assets > minecraft > gpu_warnlist.json
1 {
2   "renderer": [
3     ".*RTX 4060.*"
4   ],
5   "version": [],
6   "vendor": []
7 }
```



图 1.9 GPU 警告页面

1.6.3 地区合规性警告 *

部分国家或地区针对游戏颁布了一定的法律法规，资源包内 `regional_compliances.json` 可以相应地设置游戏在运行一段时间后出现的弹窗警告，其格式如下所示：

文件封装

- ❏ **<地区代码>**: 键名为 ISO 3166-1 三位字母地区代码，游戏会针对该系统地区进行弹窗。
- ❏ 一项弹窗。
 - ❏ **delay**: 第一次弹窗时游戏的运行时间，单位为分钟，默认值为 `0`。
 - ❏ **period**: 弹窗周期，单位为分钟。
 - ❏ **title**: 弹窗标题，需要是一个翻译标识符，详见小节 3.1.1。
 - ❏ **message**: 弹窗的具体信息，需要是一个翻译标识符，详见小节 3.1.1。

原版资源包内的 `regional_compliances.json` 内容如下：

```
assets > minecraft > gpu_warnlist.json
1  {
2    "KOR" : [
3      {
4        "delay": 1440,
5        "period": 60,
6        "title": "compliance.playtime.greaterThan24Hours",
7        "message": "compliance.playtime.message"
8      },
9      {
10       "period": 60,
11       "title": "compliance.playtime.hours",
12       "message": "compliance.playtime.message"
13     }
14   ]
15 }
```

它只设置了针对韩国的弹窗警告：每 1 小时提醒一次，连续游戏时间超过 24 小时也会提醒。

1.7 游戏机制

虽然技术性开发是能够调控游戏运行方式的手段，但开发成果还是不免受到游戏机制的制约。在时间上受到游戏循环驱动的影响，以游戏刻为单位计算内容；在空间上受到区块加载的影响，绝大多数操作都只能在允许运算的区块中进行。本节旨在介绍游戏加载、运行、更新的一些基本游戏机制。

1.7.1 端

Minecraft 的架构是**客户端-服务端模型**，顾名思义，Minecraft 使用**客户端（Client）**和**服务器端（Server）**来运作自身。这两个端（Sides）之间的通信是由**封包（Packet）**实现的。在网络工程中，这个概念一般译为“数据包”，而 Minecraft 中另有一个叫 Datapack（数据包）的概念，故 Packet 在 Minecraft 技术性开发领域会特地译为“封包”。

然而，仅通过客户端和服务端理解 Minecraft 的运作是远远不够的，因为 Minecraft 的架构还包括**物理端（Physical sides）**和**逻辑端（Logical sides）**，并且物理端和逻辑端分别具有各自的客户端和服务端。

一、物理客户端

物理客户端（Physical client）是指下载游戏版本得到的 `<version>.jar` 文件，它的默认文件路径为 `.minecraft\<版本号>\<version>.jar`。物理客户端包含了游戏的全部内容，也包含了内置的客户端和服务端，即**逻辑客户端（Logical client）**和**逻辑服务端（Logical server）**，其中逻辑服务端又称**内置服务器（Integrated server，或译为集成服务端）**。内置服务器会受到客户端的影响。

逻辑客户端负责接收来自玩家的输入、处理资源包、渲染游戏画面，并将数据输送给逻辑服务端处理；逻辑服务端负责处理由客户端发送的数据，运行游戏逻辑。例如，当玩家在游戏中移动时，客户端会根据玩家输入的移动方向渲染玩家此时的游戏画面，同时又将玩家移动的信息通过封包发送给逻辑服务端，逻辑服务端计算玩家的坐标、玩家周围是否存在任何的碰撞箱阻止玩家移动，将计算结果通过封包返还给逻辑客户端，渲染玩家移动的游戏画面。客户端的渲染会与服务端产生不一致的情况，例如标记是一种仅存在于服务端的实体，在客户端上并不会渲染标记，参见 4.3。

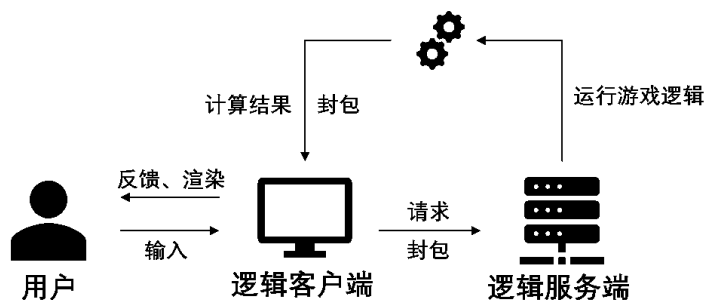


图 1.10 逻辑客户端和逻辑服务端的运行流程

即使是进行单人游戏，Minecraft 依旧会在玩家进入本地世界时创建一个内置服务器，在本地世界关闭时内置服务器即被关闭。这个内置服务器可以开放至局域网，从而将单人游戏开放为局域网联机的多人游戏。此时内置服务器拥有一个地址，其格式为

```
<IPv4 地址>:<端口>
```

1.57

局域网联机的 IPv4 地址可由 CMD 的 `ipconfig` 命令查询。端口是一个数值，可以自由指定，范围为 0 至 65535（含两端）。除了通过暂停游戏的对局域网开放选项外，玩家还可以通过命令 `/publish` 开放内置服务器，该命令所需权限等级为 4，且仅能在单人游戏中使用，其语法为：

```
publish [<allowCommands>] [<gamemode>] [<port>]
```

1.58

其中的参数：`<allowCommands>`（布尔值 `brigadier:bool`）——可选，指定是否启用命令，默认为否。

`<gamemode>`（游戏模式 `minecraft:gamemode`）——指定新玩家进入游戏的游戏模式，可用值有 `survival`（生存模式）、`creative`（创造模式）、`adventure`（冒险模式）和 `spectator`（旁观模式）。若不指定，则使用该游戏世界的默认游戏模式。

`<port>`（整数 `brigadier:integer`）——指定端口，必须为介于 0 和 65535 之间（含）的值，若不指定，则随机选择大于 1024 的端口。

例 1.12

使用命令 `/publish` 开放当前本地世界，要求关闭命令、设置玩家游戏模式为生存模式、端口指定为 12345。

解

命令为

```
publish false survival 12345
```

1.59

二、物理服务端

除了使用局域网联机进行多人游戏，Minecraft 提供了另一种进行多人游戏的方法，即**物理服务端 (Physical server)**。物理服务端只包含一个逻辑服务端，并不包含逻辑客户端。这意味着物理服务端只能负责服务端的任务，而不能使用户参与游戏；但同时也意味着若服主不在游戏中，服务器也不会关闭；此外，物理服务端在运行过程中只能加载一个游戏世界，切换其他游戏世界需要重启服务器。

物理服务端内的逻辑服务端又可被称为**专用服务器 (Dedicated server, 或译为独立服务端)**，该逻辑服务端包含配置文件 `server.properties`，用于存储服务器的所有设置。专用服务器不会受到连接的逻辑客户端的影响。同局域网联机一样，专用服务器也拥有一个地址，其格式与语法 1.57 所述一致。

表 1.15 各种情况使用的客户端和服务端

	单人游戏	局域网多人游戏	专用服务器多人游戏
客户端	逻辑客户端	玩家各自的逻辑客户端	玩家各自的逻辑客户端
服务端	内置服务器	内置服务器	专用服务器

1.7.2 游戏刻

由于游戏不可能时时刻刻都进行计算，正常情况下，游戏以一定的频率循环驱动，即每隔一定的时间进行一次计算，计算完毕后游戏会进行休眠，此时游戏不作任何计算直到下一游戏刻。一个循环周期被称为一个**刻（Tick）**，或**游戏刻（Game tick，简称 gt）**。

一、刻率和帧率

每秒游戏刻的数量数由**每秒刻数（Ticks per second，简称 TPS）**这个指标显示；此外还有一个指标与每秒游戏刻数相关，即**每刻毫秒数（Milliseconds per tick，简称 MSPT）**，它反映的是游戏刻计算的平均时间。TPS 是一个可变量，它可以由命令 `/tick` 修改，不做修改的默认值为 20。也就是说，正常情况下每秒有 20gt，或者称最大 TPS 频率为 20。MSPT 可以由 `F3`（调试屏幕）查看，这个统计量名称为 `ms ticks`。正常情况下 MSPT 不会大于 50，且只有当 MSPT 值不大于 50 时才能保证 TPS 维持在 20。

MSPT 与 TPS 的数量关系可表示为

$$\text{MSPT} \times \text{TPS} \leq 1000 \quad (1.1)$$

受限于游戏中的计算量及计算机的性能，若计算量过大，MSPT 增大，则 TPS 会相应地减小，造成**掉刻**。TPS 无法维持在最大频率时，可由下式计算出实际的 TPS：

$$\text{TPS} = \frac{1000}{\text{MSPT}} \quad (1.2)$$

如果按照默认的每秒 20gt 的频率渲染画面，难免会产生肉眼可见的不连续画面。因此客户端渲染游戏画面时，并不是完全按照刻率渲染，而是在刻之间**补帧**以形成平滑画面。用于描述渲染频率的指标为**帧率（Frame per second，简称 FPS）**，它反应的是客户端的每秒渲染的帧数。帧率受到客户端渲染计算量、计算机性能的影响，可以通过**最大帧率**选项控制最高 FPS。

当客户端渲染计算量较大时，FPS 会下降，造成**掉帧**。因此分析客户端画面卡顿时，可以考虑的若干可能性有：渲染计算量较大，或是游戏刻计算量较大造成渲染补帧无法形成平滑画面。若遇到画面较为流畅、但游戏内容卡顿——如实体不移动、放置破坏方块相应时间较长——则说明客户端渲染计算正常而游戏刻计算量大。

游戏的流畅程度是影响玩家游戏体验的关键因素，**无论是搭建红石电路还是制作数据包，均需要综合考虑成品对 TPS 和 FPS 的影响。**

二、命令/tick 的用法

命令 `/tick` 可用于控制游戏刻运行，该命令所需权限等级为 3。以下是所有用法。

1. 查询当前游戏刻频率，并返回性能数据。语法为：

```
tick query
```

1.60

2. 定义游戏刻频率，语法为：

```
tick rate <rate>
```

1.61

其中的参数：`<rate>`（浮点数 `brigadier:float`）——需要设置的游戏刻频率。设置后，最大 TPS 频率即为这个参数设置的值。

例 1.13

将游戏刻频率设为 40。

解

所需命令为

```
tick rate 40
```

1.62

负载正常时，MPST 应不大于 $1000 \div 40 = 25$ 。

3. 冻结游戏刻，语法为：

```
tick freeze
```

1.63

4. 步进特定数量的游戏刻，语法为：

```
tick step [<time>]
```

1.64

该语法仅能在游戏刻已冻结的情况下使用。步进结束后，游戏刻会继续冻结。

其中的参数：`[<time>]`（时间 `minecraft:time`）——步进时间长度。格式为：

`<单精度浮点数>[<单位>]`

单位可以为：`t`（游戏刻），`s`（秒）或 `d`（游戏日，1 游戏日固定为 24000 游戏刻），若不写单位，则默认单位为游戏刻。时间参数会在单位转化成游戏刻后取最近的整数。例如，`.2d` 会换算为 $24000 \times 2 = 48000$ gt。若游戏刻频率为 40，则 `.5s` 会换算为 $40 \times 0.5 = 20$ gt。

例 1.14

使游戏刻步进 10 秒。

解

所需命令为

```
tick step 10s
```

1.65

5. 停止正在进行的步进，并冻结游戏刻。语法为：

```
tick step stop
```

1.66

6. 取消冻结游戏刻，语法为：

```
tick unfreeze
```

1.67

7. 忽略游戏刻频率的限制持续进行更新，语法为：

```
tick sprint [<time>]
```

1.68

该语法用于在指定的时间 [<time>] 内尽可能快地运行游戏，结束后恢复先前的游戏刻速率并返回性能信息。参数 [<time>] 的用法与语法 1.64 中所述的一致。

8. 停止正在进行的忽略游戏刻频率进行的更新，并恢复先前的游戏刻频率。语法为：

```
tick sprint stop
```

1.69

三、游戏刻计算流程 *

Minecraft 的游戏计算内容繁多，在同一个线程中的计算不可能同步完成，各模块的计算在同一游戏刻内一定有先后顺序，这种先后顺序被称为**微时序（Microtiming，或称微观延迟）**。在一个游戏刻内，服务端的计算流程顺序如下所示^①：

1. 检查游戏是否被暂停，若已暂停，则自动保存游戏。若此时内置服务器正在主持局域网联机，则统计世界打开时间。
2. 已暂停游戏若恢复运行，则对所有玩家强制时间同步。
3. 运行服务端逻辑。内容顺序如下：
 - (1) 更新游戏刻计数器。
 - (2) 若此时正在使用 `/tick step` 步进游戏刻，则计算剩余需步进游戏刻。
 - (3) 关闭与客户端的网络自动发送队列的刷新。
 - (4) 如果游戏世界被重新加载（如使用 `/reload`），则调用 `#minecraft:load` 中的函数，调用顺序与列表 `l value` 中的函数顺序一致。一个函数被调用时按 `.mcfunction` 文件内的命令顺序依次执行命令。
 - (5) *调用一次 `#minecraft:tick` 中的函数，顺序与 (4) 中所述一致。
 - (6) 遍历所有维度，遍历顺序为：主世界、下界、末地、有先后顺序的自定义维度。遍历到某个维度时，按以下流程计算：
 - ① 每隔 20 gt 对玩家同步一次该维度的时间。
 - ② 运行维度游戏刻逻辑，若计算出现异常，则游戏崩溃。游戏刻逻辑按以下流程计算：
 - A. 更新世界边界。
 - B. *计算天气循环、更新降雨和雷暴计时器。
 - C. 计算日夜更替，若玩家入睡情况满足跳过当前时间至下一次日出，则将时间调整至下一次日出。若此时正在降雨，则重置天气循环。
 - D. 更新内部光照等级乘数。
 - E. *更新时间。
 - F. *在非调试维度执行方块计划刻。
 - G. *在非调试维度执行流体计划刻。
 - H. *更新袭击事件。

① 带*的项目表示在游戏刻冻结时忽略执行。

- I. 计算区块数据，按以下流程计算：
 - a. *更新计算标签。
 - b. 计算需要执行区块刻的区块，并打乱区块刻执行顺序。
 - c. *计算生物生成。
 - d. *按区块顺序执行区块刻。
 - e. *执行计划周期生成。
 - f. 更新各个玩家追踪的区块。
 - g. 更新兴趣点数据。
 - h. 卸载不需要的区块。
- J. *计算方块事件。
- K. 如果维度内有玩家或强加载区块，则重置限制超时。
- L. 如果闲置超时小于 300 gt，则运行实体和方块计算逻辑，按以下流程计算：
 - a. *计算末影龙战斗数据。
 - b. 遍历所有实体，进行实体计算。
 - c. *计算方块实体。
- M. 加载区块内未加载的实体，卸载不需要的实体。
- (7) 检查客户端连接。
- (8) 每 600gt 向所有玩家发送更新延迟网络包。
- (9) 若存在服务器 GUI，则更新服务器 GUI。
- (10) 对所有玩家发送正在等待发送的区块网络包，并打开与玩家客户端的网络自动发送队列刷新。
- (11) 若与上一次自动保存已有 100 gt，则尝试自动保存。
- (12) 运行处理队列中的事件。
- 4. 在客户端更新渲染距离和模拟距离。

1.7.3 区块加载

一个 Minecraft 世界在水平方向上的长宽均超过了六千万格，如果一次性将所有内容全部加载出来，则会消耗非常多的内存。为使游戏运行过程中不占用过多的内存，游戏仅会加载部分区域供玩家游玩，这些加载和数据存储的单位被称为**区块 (Chunk)**，每一个区块均是 $16 \times 384 \times 16$ 的立方体，一个区块又可以沿高度分割成 24 个 $16 \times 16 \times 16$ 大小的**区段 (Chunk section)**，或称**子区块 (Sub chunk)**。在必要的情况下，游戏会**卸载**一些区块，并在需要的时候**加载**区块。已卸载的区块不会处理游戏的任何事件，这其中也包括了实体的生成、活动，红石电路、命令的运行等等。若长距离执行命令，区块的加载是需要着重考虑的一方面。

一、加载等级和计算等级

游戏使用**加载等级 (Load level)**和**计算等级**^①来确定一个区块的加载情况。

1. 加载等级

区块的加载等级范围可表示为 0 ~ 45 (含) 的整数，且等级越高，区块内加载的内容就越多。加载等级可划分为如下表所示的不同加载等级类型：

^① 无英文原文。

表 1.16 加载等级类型

类型	加载等级	描述
实体计算	≤ 31	可以计算实体、方块实体和计划刻，方块修改可以被追踪。
方块计算	32	不可计算但可以追踪实体，可以计算方块实体和计划刻，可以追踪方块修改。
完全加载	33	不可计算但可以追踪实体，不可以计算方块实体和计划刻，不可以追踪方块修改。
不可访问	34	大于该加载等级时，实体和方块修改都不可以追踪，方块实体和计划刻不可以计算，但可以进行初始化光照计算。
	35	可进行地形雕刻。
	36	可填充生物群系。
	37 ~ 44	结构允许生成。
卸载	45	区块已被卸载

2. 计算等级

类似于加载等级，区块的加载等级范围可表示为 0 ~ 33（含）的整数。且等级越高，区块内加载的内容就越多。区块内具体可以加载何内容由加载等级和计算综合决定。下表列举了不同加载程度的区块^①：

表 1.17 区块加载类型

区块类型	加载等级	计算等级	每游戏刻的具体行为
强加载区块	≤ 31	≤ 31	可以计算实体、方块实体和计划刻，任何命令都能正常执行。
弱加载区块	32	32	可以计算方块实体和计划刻，实体可以被命令追踪和记录，但无法计算实体。
加载边界区块	33	32	实体可以被命令追踪和记录，但不计算实体、方块实体和计划刻。
不可访问	≥ 34	33	不计算实体、方块实体和计划刻，任何命令都不可执行。

二、加载标签

一个区块加载等级和计算等级的具体数值可以由**加载标签 (Load ticket)**决定。一个加载标签有**基础等级**、**标签类型**、**存活时间**和**持久化**四个属性，基础等级指一个区块被赋予某种加载标签时该区块的加载等级和计算等级（两者不一定相等），存活时间指该类加载标签能够持续的时长。以下是注册表内所有类型的加载标签：

1. 玩家标签

玩家标签在注册表内分为**玩家加载标签 (Player loading ticket)**和**玩家计算标签 (Player simulation ticket)**。玩家所在的区块会被赋予这玩家标签，其中加载等级由玩家加载标签赋予，计算等级由玩家计算标签赋予，此时该区块的加载等级主要取决于**渲染距离 (Render distance)**，计算等级主要取决于**模拟距离 (Simulation distance)**，渲染距离和模拟距离均可以由选项设置。渲染距离主要决定游戏世界的渲染距离，即渲染区块的数量，对 FPS 的影响程度较高。通常来说渲染区域以玩家所在区块为中心，在平面上为正方形，其边长为

^① 表中一种类型的区块必须同时满足该行加载等级和计算等级的要求。

$$a = 2d_r + 1 \quad (1.3)$$

式中： a ——渲染区域边长。

d_r ——在单人游戏中为渲染距离，原版的渲染距离必须为介于 2 和 32 之间（含）的整数。

在多人游戏中为 `server.properties` 中 `view-distance` 的值。

对于上述正方形区域内的每一个区块，其加载等级均为 31。

模拟距离主要决定实体更新、计算方块计划刻、流体计划刻，这些计算主要由服务端负责，因此相对于渲染距离而言，它对 TPS 的影响更大。玩家所在区块的计算等级为

$$L_s = \max\{0, 31 - d_s\} \quad (1.4)$$

式中： L_s ——计算等级。

d_s ——模拟距离，原版的模拟距离必须为介于 5 和 32 之间（含）的整数。

2. 传送门标签

当有实体在下界传送门或末地传送门中且正在被传送至另一个维度时，游戏将会为目的维度的目标区块赋予传送门标签，加载等级和计算等级均为 30，存活时间为 300 gt。

3. 末影龙标签

玩家与末影龙发生战斗时会在末地的 [0,0] 区块产生末影龙标签，加载等级和计算等级均为 24，存活时间不定，直到末影龙被杀死或与末影龙战斗的玩家数量清零时才撤销该标签。

4. 强制加载标签

命令 `/forceload` 所指定区块的加载等级和计算等级为 31，并被添加该标签，这是持久性标签，除非使用命令 `/forceunload` 移除标签。参见节 2.2.1。

5. 末影珍珠标签

当玩家掷出末影珍珠后，该末影珍珠所在区块每 39 gt 会被赋予该标签，加载等级和计算等级均为 31，存活时间 40 gt。

6. 临时标签

由游戏代码决定而创建，通常用于计算生物的 AI 和生成，加载等级一般大于等于 32，不设置计算等级。存活时间仅为 1 gt。

整理上述信息可得下表：

表 1.18 加载标签

标签类型	注册名称	基础等级		存活时间	持久化
		加载等级	计算等级		
玩家加载标签	<code>player_loading</code>	31	无	永久	否
玩家计算标签	<code>player_simulation</code>	无	$\max\{0, 31 - d_s\}$		
传送门标签	<code>portal</code>	30		300 gt	是
末影龙标签	<code>dragon</code>	24		永久	否
强制加载标签	<code>forced</code>	31		永久	是
末影珍珠标签	<code>ender_pearl</code>	31		40 gt	否
临时标签	<code>unknown</code>	≥ 32	无	40 gt	否

三、等级传播

拥有一定加载等级和计算等级的区块可以向周围的 8 个区块传播加载等级和计算等级，每次传播时等级增加 1。加载等级的最大值为 45，等级为 45 的区块会直接从内存中卸载。计算等级的最大值为 33，此时若继续向外传播则等级会维持在 33。

34	34	34	34	34	34	34
34	33	33	33	33	33	34
34	33	32	32	32	33	34
34	33	32	31	32	33	34
34	33	32	32	32	33	34
34	33	33	33	33	33	34
34	34	34	34	34	34	34

图 1.11 等级的传播

根据这种传播规则，假设有一个区块 A 被赋予了加载标签，已知其加载等级或计算等级，在周围没有其他区块具有加载标签的情况下，区块等级的传播使用**切比雪夫距离 (Chebyshev distance)** 来计算。则区块 B 的等级为

$$L_{z,B} = \begin{cases} L_{z,A} + d_{\infty}(A, B), & L_{z,A} + d_{\infty}(A, B) \leq 44 \\ \text{不存在} & , \text{otherwise} \end{cases} \quad (1.5)$$

$$L_{s,B} = \min\{33, L_{s,A} + d_{\infty}(A, B)\} \quad (1.6)$$

式中： $L_{z,A}$ 、 $L_{z,B}$ 、 $L_{s,A}$ 、 $L_{s,B}$ ——下标“z”表示加载等级，“s”表示计算等级，逗号后的字母表示该等级所属的区块。

x_A 、 x_B 、 z_A 、 z_B ——区块 A 、 B 分别在 x 、 z 方向上的区块坐标。

$d_{\infty}(A, B)$ ——区块 A 、 B 之间的切比雪夫距离， $d_{\infty}(A, B) = \max\{|x_B - x_A|, |z_B - z_A|\}$

若一个区块接受到多个传播等级，则选取等级最低者作为自己的加载（计算）等级。假设一定区域内存在被赋予加载标签的区块 1、2、3、……则该区域内任意区块的加载（计算）等级可表示为

$$L_z = \begin{cases} \min_{i \geq 1} \{L_{z,i} + d_{\infty}\}, & \min_{i \geq 1} \{L_{z,i} + d_{\infty}\} \leq 44 \\ \text{不存在} & , \text{otherwise} \end{cases} \quad (1.7)$$

$$L_s = \begin{cases} \min_{i \geq 1} \{L_{s,i} + d_{\infty}\}, & \min_{i \geq 1} \{L_{s,i} + d_{\infty}\} \leq 44 \\ \text{不存在} & , \text{otherwise} \end{cases} \quad (1.8)$$

式中： x 、 z ——被计算区块的区块坐标。

x_i 、 z_i ——区块 i 的区块坐标， $i = 1, 2, 3 \dots$ 。

d_{∞} ——被计算区块到区块 i 的切比雪夫距离， $d_{\infty} = \max\{|x - x_i|, |z - z_i|\}$ 。

例 1.15

图 1.12 展示了一个区域的区块，玩家所在的区块为 P ，此时模拟距离为 5，渲染距离为 4，判断在区块 A 内能否正常执行命令。

							A
P							

图 1.12

解

解析 显然区块 P 被赋予了玩家标签, 已知模拟距离为 5, 由式 (1.4) 可得区块 P 的计算等级为 26。已知渲染距离为 4, 由式 (1.3) 得以区块 P 为中心 9×9 区块的加载等级均为 31。由等级的传播, 该区域的等级如图 1.13 所示。

31	31	31	31	31	31	31	32
30	30	30	30	30	30	31	32
29	29	29	29	29	30	31	32
28	28	28	28	29	30	31	32
27	27	27	28	29	30	31	32
27	26	27	28	29	30	31	32
27	27	27	28	29	30	31	32

(a) 计算等级

[illegible]

(b) 加载等级

图 1.13

则区块 A 的计算等级为 32, 加载等级为 33, 属于加载边界区块, 可以使用命令追踪该区块内的实体。

例 1.16

一玩家所在区块的区块坐标为 $[5, 12]$ ，试通过调整渲染距离和模拟距离使区块 $[17, -5]$ 为强加载区块。

解

遇到这种两个区块之间相隔较远不能通过画图直接得到结果的, 可先计算两个区块之间的切比雪夫距离 $d_{\infty} = \max\{|17 - 5|, |-5 - 12|\} = 17$, 要使区块 $[17, -5]$ 为强加载区块, 则该区块的加载和计算等级必须均小于等于 31, 所以渲染距离必须至少为 17。设模拟距离为 d_s , 则玩家当前区块的计算等级为 $\max\{0, 31 - d_s\}$, 切比雪夫距离为 17 的区块的计算等级 $\max\{0, 31 - d_s\} + 17 \leq 31$, 得 $d_s \geq 17$ 。所以渲染距离和模拟距离都应大于 17。

四、闲置超时

若玩家离开某一维度的时间超过 300 gt，除非该维度中有其他玩家或有使用 `/forceload` 强制加载的区块，否则该维度会无视区块加载等级而停止方块实体、实体等有关的运算，这种限制被称为**闲置超时（Idle timeout）**。玩家进入或离开维度时均会重置闲置超时。

1.7.4 区块刻 随机刻 计划刻 红石刻

游戏刻的计算过程中还有一些特殊的计算模式。

一、区块刻

游戏计算区块数据时，以区块为单位进行一次计算，该计算被称为**区块刻（Chunk tick）**。区块刻按以下流程计算：

1. 按随机顺序遍历所有满足区块刻条件、加载标签类型为实体计算、水平方向上区块中心距玩家 128 格以内的区块。
 - (1) 增加区块时间。
 - (2) 如果当前为雷暴，执行雷暴相关逻辑。
 - (3) 在世界边界内执行周期生成。
2. 遍历所有满足区块刻条件的区块。
 - (1) 如果正在降雨，执行雨雪相关逻辑。
 - (2) 执行随机刻。

二、随机刻

接受到区块刻的区块会随机挑选区块内的一些方块并赋予**随机刻（Random tick）**，接收到随机刻的方块会进行方块更新，如农作物的生长、藤蔓的蔓延、草方块的蔓延等。随机刻的赋予规则是：每一游戏刻在所有区段内随机挑选若干方块，游戏规则 `randomTickSpeed` 可用于指定随机挑选方块的数量（默认为 3），每次挑选的方块可以为同一个。

假设 `randomTickSpeed` 的值为 m ，则在一个游戏刻内区段内某方块被选中的概率为

$$\begin{aligned} p_0 &= 1 - C_0^m \left(\frac{1}{16^3} \right)^0 \left(1 - \frac{1}{16^3} \right)^m \\ &= 1 - \left(1 - \frac{1}{16^3} \right)^m \end{aligned} \quad (1.9)$$

显然该方块在 t gt 内获取随机刻是独立的重复伯努利试验，服从参数为 p_0 的几何分布，则第 t gt 得到随机刻的概率为

$$P(X = t) = (1 - p_0)^{t-1} p_0 \quad (1.10)$$

则方块接受到随机刻的平均间隔为

$$E(X) = \sum_{t=1}^{+\infty} X_t P_t = \frac{1}{p_0} = \frac{1}{1 - \left(1 - \frac{1}{16^3} \right)^m} \quad (1.11)$$

上式结果的单位为游戏刻。若将 `randomTickSpeed` 的默认值 3 代入，且此时游戏刻率为 20，可以得到默认的随机刻的平均间隔时间 1365.67 gt，合 68.28 秒。若 `randomTickSpeed` 的值为零，有 $\lim_{x \rightarrow 0} E(X) = +\infty$ ，即永远不会接受到随机刻。在设计方块更新的速率时可参考上述公式。不过，上述公式是在概率论及数理统计的范围内讨论的，并不意味着随机刻平均间隔时间一定是该值，极端情况下可能会出现 1 gt 或超过 1000000 gt 的间隔，只能说接受到随机刻这一事件大致服从几何分布。总体上来说，`randomTickSpeed` 的值越大，方块更新的频率就越高。

三、计划刻 *

一些方块除了会被动接受随机刻，有时还会主动请求在未来某一游戏刻更新方块，这种更新方式被称为**计划刻 (Schedule tick)**，如水的流动、红石中继器的信号变更等。计划刻分为**方块计划刻**和**流体计划刻**，分别控制普通方块和液体的计划更新。方块计划刻会根据优先级按顺序依次执行，优先级一般为非正数，且优先级越小，执行时间越早。流体计划刻则没有优先级。一个游戏刻内最多执行 65536 个计划刻。超出限制数量的计划刻将延后至下一游戏刻处理。

四、红石刻

一般而言，大部分红石元件的工作时间以 2 gt 为基本单位，可令 2 gt 为 1 个**红石刻 (Redstone tick, 简称 rt)** 以方便计算电路的延迟。在 TPS 等于 20 的情况下，1 rt 等效于 0.1 秒。不过，红石刻不是真实存在的游戏机制，是仅在红石电路或使用命令方块的命令系统中用于描述延迟的基本单位。

例 1.17

创建一个 10 秒的延迟至少需要多少红石中继器？

解

10 秒钟的延迟即 100 rt，一个红石中继器最多可提供 4 rt 的延迟，至少需要 $100 \div 4 = 25$ 个红石中继器。

1.7.5 方块

方块 (Block) 是构成 Minecraft 的基本单位。一般而言方块是 $1 \times 1 \times 1$ 大小的实物，部分方块可能有特殊的大小和形状。

空气和**液体**是两类特殊的方块。其中空气包括普通空气（命名空间 ID `minecraft:air`）、洞穴空气（命名空间 ID `minecraft:cave_air`）和虚空空气（命名空间 ID `minecraft:void_air`）三种。洞穴空气是生成世界时创建洞穴生成的空气，虚空空气是生成于建造区域以外的空气。这三类空气共同填满了任何未被其他方块占用的空间，因此使用命令移除其他方块是通过在这些方块的位置上放置空气从而实现。此外，物品形式的空气也广泛存在于物品栏中未被其他物品填充的槽位，使用命令 `/item` 移除物品栏中的物品也是通过在该槽位放置空气从而实现。

液体是可以自由流动的方块，目前游戏中只有水和熔岩两种液体。液体会扩散，具有深度，该性质用于控制液体最大可扩散的距离。

一、方块状态

特别地、当指定部分方块时，这些方块很可能拥有变种，比如门的开关状态、小麦的成熟度等，这些变种便是**方块属性 (Block property)**。不同方块属性的集合被称为**方块状态 (Block states)**，相应地，液体的属性集合是**流体状态 (Fluid states)**。方块状态在命名空间 ID 的基础上进一步定义了一个方块的模型、行为，方块状态是方块本身拥有的性质，是硬编码的。附录 I.2 列举了所有方块及其可用的方块状态。

以 Minecraft 中的门为例，如果给门的开关状态分别配置一个 ID，那门的一个 ID 就会被拆分成两个。不仅如此，门还有不同的朝向、上半扇和下半扇、门轴的位置、是否被激活这些变种，不同的变种组合在一起的所有结果一共有 64 种。

可见，如果将这些方块的每一个变种都用单独的命名空间 ID 来表示则可能会占用更多的内存，也会让原本清晰的命名空间 ID 变得难以书写、阅读，而使用数字 ID 附带 Damage 值的做法在扁平化后已被淘汰了。为此，在指定方块状态的时候，不直接使用命名空间 ID，而是在命名空间 ID 后添加类似键值对的表示方式，格式为：

```
<命名空间>:<ID>[<属性>=<值>,<属性>=<值>,...]
```

1.70

注意：

1. 括号 `{ }` 要紧贴命名空间 ID，不能出现空格。
2. 不同的键值对之间用英文逗号 `,` 隔开，最后一个键值对后面不要加逗号。
3. 属性必须是这个方块所拥有的，门没有年龄这个属性，因此无法对门设置年龄这种方块状态。属性可以使用 `Tab` 键补全。
4. 属性的值必须按照 Minecraft 要求的参数格式填写，比如布尔值、方向或是整型。每个属性都有其需要的参数类型，这些参数类型不尽相同，有些需要布尔值、有些需要方向值。比如，门的 `open` 属性需要布尔值 `true` 或 `false`，像 `east` 这样的方向值是无效的。

例如，一个开启的、朝向为东的铁门可写成如下的形式：

```
minecraft:iron_door[open=true,face=east]
```

1.71

不定义任何方块状态时，系统会选择这个方块的默认方块状态，铁门的默认方块状态为朝向北、关闭。

调试棒 (Debug stick) 可用于快速更改一个方块的方块状态，对方块点击（默认为 `鼠标左键`）可以切换更改的方块状态种类，对方块使用（默认为 `鼠标右键`）可以切换此方块状态的值。

二、方块实体

方块实体 (Block entity) 是一个很有趣的概念，它将一般被认为相对静态的方块和相对动态的实体结合起来。这样做的意义在于——在方块状态规定的有穷集合的基础上，使方块能够容纳更多数据，并使得方块数据便于编辑、修改。方块实体可以每游戏刻都进行计算，从而提供更好的渲染动画，但同时也可能使得计算量超过游戏刻计算的负载。

例如，告示牌是典型的既有方块实体又有多个方块状态的方块，其方块状态 `Rotation` 决定告示牌为何种朝向（告示牌的朝向只有 16 种），从而调用相应的模型使告示牌显示出需要的朝向。而告示牌又是一种可以显示文本的方块，显然几个模型无法承载大量各种式样的文本，因此使用方块实体让告示牌能够容纳更多的数据。

和方块状态一样，只有部分方块拥有方块实体。方块实体的数据使用NBT格式，详见节4.2。

三、方块更新 *

受限于计算机性能，游戏无法每游戏刻都对方块进行计算。只有当方块被放置、破坏、修改，或方块状态产生变化时，该方块会通知其毗邻方块（即上、下、东、南、西、北六个面的邻接方块）进行相应，这种游戏机制被称为**方块更新（Block update）**。使用非 `strict` 模式的命令放置、移除方块时也会产生方块更新。Minecraft 有三种类型的方块更新，即 PP 更新、NC 更新和比较器更新。

方块更新一般依照以下的顺序依次计算：调用被替代方块状态的破坏行为 → 调用替代方块状态的放置行为 → 进行 NC 更新 → 进行比较器更新 → 进行 PP 更新。

方块更新会向外传播，在执行更新的过程中可能在毗邻方块产生新的更新，一直到所有可用的更新都执行完毕，但是在更新无法完全清除的情况下可能会造成游戏崩溃。例如在只有一层沙子的超平坦世界中破坏任意沙子，则方块更新传播会持续进行，并且计算更新的方块数量越来越多，最终会不可避免地造成游戏崩溃。服务器配置文件 `server.properties` 的 `max-chained-neighbor-updates` 一项可用于设置最大的连锁更新数量，超过此值的新增更新将会被忽略。

当一个方块发生变化时，即产生 PP 更新。对于一个方块上的六个毗邻方块，依次沿 x 、 z 、 y 轴的方向，各方向上先检查负轴方向上的方块，再检查正轴方向上的方块，即按照西、东、北、南、下、上的顺序传播 PP 更新。PP 更新是广泛存在的一种更新类型，包括但不限于附着性方块的掉落、连接性方块的连接判断、重力方块的掉落检测等。

NC 更新是一种在红石元件中更常见的更新类型，除了方块放置、修改或移除外，方块实体也可能产生 NC 更新。与 PP 更新不同，除红石线外，NC 更新的传播次序是西、东、下、上、北、南；对于红石线则为更新传入方向的后、前、左、右、下、上方。

比较器更新只适用于在放置后可发出模拟信号的方块。

1.7.6 实体

实体（Entity）是一个动态的对象，包括**玩家**、**生物**、交通工具（所有种类的船和所有种类的矿车）、物品、物品展示框、画、盔甲架、经验球、所有的弹射物、激活的 TNT、下落的方块、漂浮的鱼饵、闪电、拴绳结、末影水晶、尖牙和标记等。**实体每游戏刻都会被计算，是技术性开发的主要研究对象。又由于其计算量较大，当已加载区域的实体数量过多，则容易引起掉刻，因此在开发过程中涉及实体计算的部分需要仔细斟酌。**

判定箱（Hitbox）是规定的方块和实体的边界，用于计算碰撞和选取。所有实体的判定箱都是长方形，无论该实体的外形如何。末影龙比较特殊，它的判定箱是由多个判定箱组合而成的。同方块判定箱一样，实体判定箱也是硬编码的，无法通过命令或数据包修改，但可以通过修改实体属性对其进行放缩。实体判定箱有以下几种类型：**边界箱（Boundary box）**是计算实体碰撞、交互事件的区界，可使用快捷键 `F3 + B` 查看；**视平线（Eye level）**显示为红色，用于判定窒息和溺水伤害。

Java 版原版所有可用的实体可分为若干类别，这些实体的命名空间均为 `minecraft`。下面列举了所有可用的实体：

1. 玩家
2. 生物

表 1.19 所有可用生物及其 ID

ID	名称	ID	名称	ID	名称
allay	悦灵	goat	山羊	skeleton	骷髅
armadillo	犛狳	guardian	守卫者	skeleton_horse	骷髅马
armor_stand	盔甲架	happy_ghast	快乐恶魂	slime	史莱姆
axolotl	美西螈	hoglin	疣猪兽	sniffer	嗅探兽
bat	蝙蝠	horse	马	snow_golem	雪傀儡
bee	蜜蜂	husk	尸壳	spider	蜘蛛
blaze	烈焰人	illusioner	幻术师	squid	鱿鱼
breeze	旋风人	iron_golem	铁傀儡	stray	流浪者
camel	骆驼	llama	羊驼	strider	炽足兽
cat	猫	magma_cube	岩浆怪	tadpole	蝌蚪
cave_spider	洞穴蜘蛛	mannequin	玩家模型	trade_llama	行商羊驼
chicken	鸡	mooshroom	哞菇	tropical_fish	热带鱼
cod	鲑鱼	mule	骡	turtle	海龟
copper_golem	铜傀儡	nautilus	鹦鹉螺	vex	恼鬼
cow	牛	ocelot	豹猫	villager	村民
creaking	嘎吱	panda	熊猫	vindicator	卫道士
creeper	苦力怕	parrot	鹦鹉	wandering_trader	流浪商人
dolphin	海豚	phantom	幻翼	warden	监守者
donkey	驴	pig	猪	witch	女巫
drowned	溺尸	piglin	猪灵	wither	凋灵
elder_guardian	远古守卫者	piglin_brute	猪灵蛮兵	wither_skeleton	凋灵骷髅
ender_dragon	末影龙	pillager	掠夺者	wolf	狼
enderman	末影人	polar_bear	北极熊	zombie	僵尸
endermite	末影螨	pufferfish	河豚	zombie_horse	僵尸马
evoker	唤魔者	rabbit	兔子	zombie_nautilus	僵尸鹦鹉螺
fox	狐狸	ravager	劫掠兽	zombified_piglin	僵尸猪灵
frog	青蛙	salmon	鲑鱼	zombie_villager	僵尸村民
ghast	恶魂	sheep	绵羊	zoglin	僵尸疣猪兽
giant	巨人	shulker	潜影贝		
glow_squid	发光鱿鱼	silverfish	蠢虫		

3. 弹射物

表 1.20 所有可用弹射物及其 ID

ID	名称	ID	名称	ID	名称
arrow	箭	fireball	火球	small_fireball	小火球
breeze_wind_charge	旋风人风弹	firework_rocket	烟花火箭	snowball	雪球
dragon_fireball	末影龙火球	fishing_bobber	浮漂	spectral_arrow	光灵箭
egg	掷出的鸡蛋	llama_spit	羊驼唾沫	trident	三叉戟
ender_pearl	掷出的末影珍珠	potion	药水	wind_charge	风弹
experience_bottle	掷出的附魔之瓶	shulker_bullet	潜影弹	wither_skull	凋灵之首

4. 交通工具

表 1.21 所有可用交通工具及其 ID

ID	名称	ID	名称	ID	名称
boat	船	chest_minecart	运输矿车	hopper_minecart	漏斗矿车
chest_boat	运输船	command_block_minecart	命令方块矿车	spawner_minecart	刷怪笼矿车
minecart	矿车	furnace_minecart	动力矿车	tnt_minecart	TNT 矿车

5. 可悬挂实体

这一类实体有物品展示框 `item_display`、荧光物品展示框 `glow_item_frame`、画 `painting`。

6. 技术类实体

这一类实体有标记 `marker`、展示实体和交互实体 `interaction`。其中展示实体分为方块展示实体 `block_display`、物品展示实体 `item_display` 和文本展示实体 `text_display`。

7. 物品 `item`，即掉落物形式的物品，它是一种实体。8. 下落的方块 `falling_block`9. 被激活的 TNT `tnt`10. 末地水晶 `end_crystal`11. 区域效果云 `area_effect_cloud`12. 唤魔者尖牙 `evoker_fangs`13. 经验球 `experience_orb`14. 末影之眼 `eye_of_ender`15. 烟花火箭 `firework_rocket`16. 拴神结 `leash_knot`17. 闪电束 `lightning_bolt`

1.7.7 游戏规则

游戏规则（Game rule）是控制游戏玩法的一种手段

所有游戏规则及其可用值如表 `hspace{2pt}ref{tab:gamerule}hspace{2pt}`所示，不是所有的游戏规则都适用布尔值。

1.8 服务器管理

服务器是在 Minecraft 中实现多人游戏的一种手段。玩家们可以连接服务器游玩各种小游戏，体验 SMP、PVP 或各种自定义多人游戏地图，极大地提高了 Minecraft 的可玩性。篇幅有限，本教程并不提供服务器的架设方法，仅提供服务器配置以及能够在服务器上使用的命令的解释，供服务器管理人员参考。

1.8.1 server.properties *

`server.properties` 是存储服务器所有配置的文件，文件中一个属性占据一行，每一行的格式为：

<属性>=<值>

1.72

例如：

gamemode=survival

1.73

enable-command-block=false

1.74

第二章 坐标

Minecraft 的游戏世界是三维的。在编写数据包的时候，有时需要确定实例所需的位置参数。这样的参数被称为**坐标 (Coordinate)**。本章将详细介绍各种坐标参数以及这些参数在命令上的应用。

2.1 坐标系与坐标

Minecraft 使用的空间直角坐标系是右手坐标系。在这种空间直角坐标系中， x 轴和 z 轴所反映的是水平方向上的位置， y 轴所反映的是垂直方向上的位置。其中， x 轴的正方向指向正东，而 z 轴的正方向指向正南。

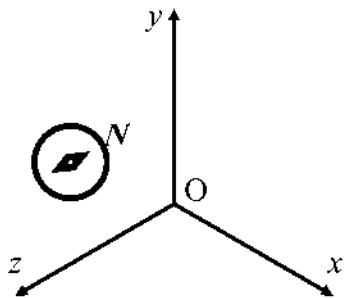


图 2.1 适用于 Minecraft 的空间直角坐标系

在命令参数中，可以用三个分量来表示某一点的位置，这是一个有序的实数三元组：

<x> <y> <z>

2.1

比如，用数学方法表示的点 $(0, 4, 4)$ 在一些命令参数中直接表示为 `0 4 4`。

例 2.1

空间直角坐标系中有两个点 $A(124.5, 76, -64.29)$ 、 $B(-10.003, 80, -33.33)$ ，则 B 点在水平位置上位于 A 点的什么方向？

解

B 点 x 坐标位于 A 点 x 坐标的负方向，因此 B 点在东西方向上位于 A 点的西方； B 点 z 坐标位于 A 点 z 坐标的正方向，因此 B 点在东西方向上位于 A 点的南方。综上所述， B 点位于 A 点的西南方向。

虽然不同的坐标表示方式基本一致，但不同的命令作用的对象不同，其坐标参数类型也不完全一致，下文将梳理命令系统使用的所有种类的坐标参数。

2.1.1 坐标的命令参数类型

命令使用 4 种与坐标相关的参数类型：方块坐标 `minecraft:block_pos`、三维坐标 `minecraft:vec3`、平面方块坐标 `minecraft:column_pos` 和二维坐标 `minecraft:vec2`。它们的关系和应用场景可以很清晰地列于下表：

表 2.1 Minecraft 中的坐标参数

	一般应用于方块	一般应用于非方块的游戏内容
三维	<code>minecraft:block_pos</code>	<code>minecraft:vec3</code>

(续表)

	一般应用于方块	一般应用于非方块的游戏内容
二维	<code>minecraft:column_pos</code>	<code>minecraft:vec2</code>

一、方块坐标

坐标表示的是一个没有体积的点，而方块是有体积的，因此需要给方块规定一个基准点，用这个基准点的坐标来表示该对象的坐标。

Minecraft 规定：大部分方块的长、宽和高均为 1 米，体积为 1 立方米。用坐标系表示这些位置时，默认了方块的边长和坐标系的单位长度在数值上相等，这意味着坐标系的基本单位为米，或称为“格”。

一个方块使用其**西北下角**的点作为它的**方块坐标 (Block position)**。若一个方块的西北下角顶点坐标为 (x, y, z) ，则该方块的方块坐标记为 (x, y, z) ，而这个方块位于 (x, y, z) 和 $(x + 1, y + 1, z + 1)$ 这两个坐标围成的立体几何图形之间。

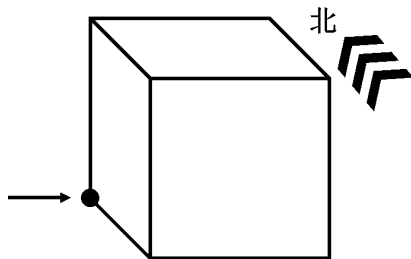


图 2.2 用方块这个方向的顶点来表示方块坐标

由于方块的角总是位于整数坐标点，作为命令参数 `minecraft:block_pos` 的方块坐标一定是由三个整数构成的有序三元组。

例 2.2

如图所示，方块坐标为 `0 0 0` 的方块为哪一个？

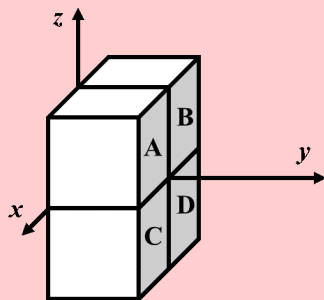


图 2.3

解

方块坐标严格按照西北下角顶点来计算，而正西、正北分别是 x 轴和 z 轴的负方向，因此用方块坐标指示的方块位置，永远位于实际坐标的东南方向，且在垂直方向上位于上方，在空间直角坐标系中的反映即为 x 、 y 、 z 三个坐标轴的正方向。因此方块坐标为 `0 0 0` 的方块位于第一卦限，即方块 A。

二、三维坐标

三维坐标 (Three-dimensional coordinates) 是精确表示一个位置的坐标参数, 命令参数类型为 `minecraft:vec3`, 用于表示坐标位置的三个元素均为双精度浮点数。三维坐标一般应用于实体, 它也可能在粒子生成和声音播放的时候被使用。例如, 这是一个合法的三维坐标:

```
5.0 56.0 17.0
```

2.2

这个坐标带有小数点, 因为三维坐标的三个参数均是双精度浮点数。但是, 这并不意味着三维坐标只能使用浮点数。也可以在三维坐标中使用整数形式, 如:

```
5 56 17
```

2.3

注意, 上述这两个坐标描述的位置并不是一致的。在实际操作中, 却发现这个玩家位于三维坐标 $(5.5, 56.0, 17.5)$ 。如图, 可以观察到玩家的坐标发生了“偏移”, 与实际坐标有所出入。其中 x 坐标和 z 坐标都发生了“偏移”, 而 y 坐标不受影响。

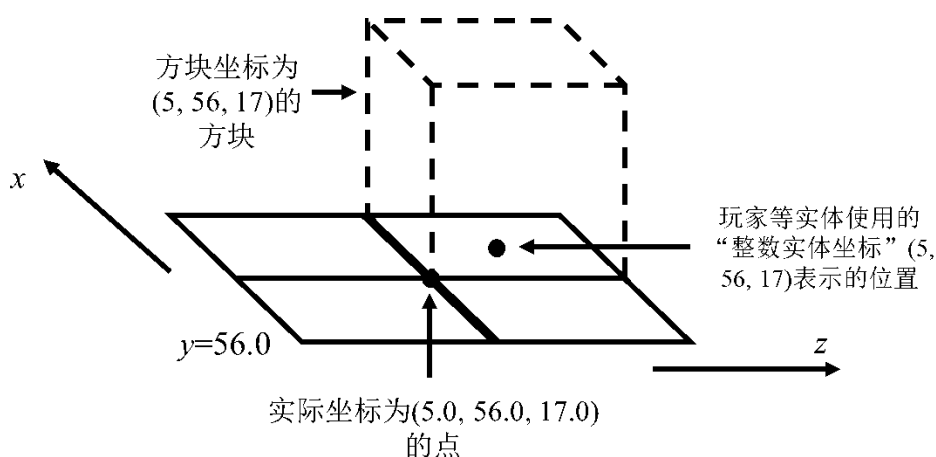


图 2.4 整数坐标发生的“偏移”

这些位置的偏移都位于相对方块两条对边的中心线上, 这是因为三维坐标使用了**中心校准 (Center correct)**, 即使用整数形式的三维坐标, 当其某一个坐标参数为 n ($n \in \mathbb{Z}$) 时, 其实际坐标为 $n - 0.5$, 这样可以使得实体位置与方块位置相适应。注意**中心校准仅适用于 x 坐标和 z 坐标**。 **y 坐标严格使用实际坐标**。

注意这里不使用“三维坐标根据方块坐标位于方块中心”的说法, 是因为三维坐标的三个参数中整数和浮点数形式可以混用, 并且使用小数形式的参数严格遵循实际坐标, 整数形式的参数则使用中心校准。比如, 位于 `5 56 17.0` 的玩家实际位于 $(5.5, 56, 17.0)$ 。

三、平面方块坐标

故名思义, 平面方块坐标 `minecraft:column_pos` 就是二维的方块坐标, 以西北角的二维坐标作为一个方块纵列的平面坐标, 两个元素均为整数。

四、二维坐标

即只由 x 坐标和 z 坐标构成的**二维坐标 (Two-dimensional coordinates)**。二维坐标的命令参数类型为 `minecraft:vec2`, 两个元素均为双精度浮点数。二维坐标若为整数, 则也使用中心校准。

2.2 区块

2.2.1 命令/forceload

第三章 文本组件

3.1 文本组件内容

3.1.1 翻译文本

第四章 存档格式

4.1 存档文件夹的结构

4.2 方块实体

4.3 技术性实体

附录 I 数据库

I.1 数据包和资源包版本号

表 I.1 数据包和资源包版本号总表

游戏版本名称	数 据 包 版本号	资 源 包 版本号	游戏版本名称	数 据 包 版本号	资 源 包 版本号
13w41a	-	1	1.15.2 Pre-Release 1	5	5
13w41b	-	1	1.15.2 Pre-release 1	5	5
13w42a	-	1	1.15.2	5	5
13w42b	-	1	Combat Test 4	5	5
13w43a	-	1	Combat Test 5	5	5
1.7	-	1	Snapshot 20w06a	5	5
1.7.1	-	1	20w07a	5	5
1.7.2	-	1	20w08a	5	5
13w47a	-	1	20w09a	5	5
13w47b	-	1	20w10a	5	5
13w47c	-	1	20w11a	5	5
13w47d	-	1	20w12a	5	5
13w47e	-	1	20w13a	5	5
13w48a	-	1	20w13b	5	5
13w48b	-	1	20w15a	5	5
13w49a	-	1	20w16a	5	5

(续表)

游戏版本名称	数 据 包 版本号	资 源 包 版本号	游戏版本名称	数 据 包 版本号	资 源 包 版本号
1.7.3	-	1	20w17a	5	5
1.7.4	-	1	20w18a	5	5
1.7.5	-	1	20w19a	5	5
1.7.6-pre1	-	1	20w20a	5	5
1.7.6-pre2	-	1	20w20b	5	5
1.7.6	-	1	20w21a	5	5
1.7.7	-	1	20w22a	5	5
1.7.8	-	1	1.16 Pre-release 1	5	5
1.7.9	-	1	1.16 Pre-release 2	5	5
1.7.10-pre1	-	1	1.16 Pre-release 3	5	5
1.7.10-pre2	-	1	1.16 Pre-release 4	5	5
1.7.10-pre3	-	1	1.16 Pre-release 5	5	5
1.7.10-pre4	-	1	1.16 Pre-release 6	5	5
1.7.10	-	1	1.16 Pre-release 7	5	5
14w02a	-	1	1.16 Pre-release 8	5	5
14w02b	-	1	1.16 Release Candidate 1	5	5
14w02c	-	1	1.16	5	5
14w03a	-	1	1.16.1	5	5
14w03b	-	1	20w27a	5	5
14w04a	-	1	20w28a	5	5
14w04b	-	1	20w29a	5	5
14w05a	-	1	20w30a	5	5
14w05b	-	1	1.16.2 Pre-release 1	5	5
14w06a	-	1	1.16.2 Pre-release 2	5	5
14w06b	-	1	1.16.2 Pre-release 3	5	5
14w07a	-	1	Combat Test 6	5	5
14w08a	-	1	1.16.2 Release Candidate 1	6	6
14w10a	-	1	1.16.2 Release Candidate 2	6	6
14w10b	-	1	1.16.2	6	6
14w10c	-	1	Combat Test 7	6	6
14w11a	-	1	Combat Test 7b	6	6
14w11b	-	1	Combat Test 7c	6	6
14w17a	-	1	Combat Test 8	6	6
14w18a	-	1	Combat Test 8b	6	6
14w18b	-	1	Combat Test 8c	6	6

(续表)

游戏版本名称	数 据 包 版本号	资 源 包 版本号	游戏版本名称	数 据 包 版本号	资 源 包 版本号
14w19a	-	1	1.16.3 Release Candidate 1	6	6
14w20a	-	1	1.16.3	6	6
14w20b	-	1	1.16.4 Pre-release 1	6	6
14w21a	-	1	1.16.4 Pre-release 2	6	6
14w21b	-	1	1.16.4 Release Candidate 1	6	6
14w25a	-	1	1.16.4	6	6
14w25b	-	1	1.16.5 Release Candidate 1	6	6
14w26a	-	1	1.16.5	6	6
14w26b	-	1	20w45a	6	7
14w26c	-	1	20w46a	7	7
14w27a	-	1	20w48a	7	7
14w27b	-	1	20w49a	7	7
14w28a	-	1	20w51a	7	7
14w28b	-	1	21w03a	7	7
14w29a	-	1	21w05a	7	7
14w29b	-	1	21w05b	7	7
14w30a	-	1	21w06a	7	7
14w30b	-	1	21w07a	7	7
14w30c	-	1	21w08a	7	7
14w31a	-	1	21w08b	7	7
14w32a	-	1	21w10a	7	7
14w32b	-	1	21w11a	7	7
14w32c	-	1	21w13a	7	7
14w32d	-	1	21w14a	7	7
14w33a	-	1	21w15a	7	7
14w33b	-	1	21w16a	7	7
14w33c	-	1	21w17a	7	7
14w34a	-	1	21w18a	7	7
14w34b	-	1	21w19a	7	7
14w34c	-	1	21w20a	7	7
14w34d	-	1	1.17 Pre-release 1	7	7
1.8-pre1	-	1	1.17 Pre-release 2	7	7
1.8-pre2	-	1	1.17 Pre-release 3	7	7
1.8-pre3	-	1	1.17 Pre-release 4	7	7
1.8	-	1	1.17 Pre-release 5	7	7

(续表)

游戏版本名称	数 据 包 版本号	资 源 包 版本号	游戏版本名称	数 据 包 版本号	资 源 包 版本号
1.8.1-pre1	-	1	1.17 Release Candidate 1	7	7
1.8.1-pre2	-	1	1.17 Release Candidate 2	7	7
1.8.1-pre3	-	1	1.17	7	7
1.8.1-pre4	-	1	1.17.1 Pre-release 1	7	7
1.8.1-pre5	-	1	1.17.1 Pre-release 2	7	7
1.8.1	-	1	1.17.1 Pre-release 3	7	7
1.8.2-pre1	-	1	1.17.1 Release Candidate 1	7	7
1.8.2-pre2	-	1	1.17.1 Release Candidate 2	7	7
1.8.2-pre3	-	1	1.17.1	7	7
1.8.2-pre4	-	1	1.18 Experimental Snapshot 1	7	7
1.8.2-pre5	-	1	1.18 experimental snapshot 2	7	7
1.8.2-pre6	-	1	1.18 experimental snapshot 3	7	7
1.8.2-pre7	-	1	1.18 experimental snapshot 4	7	7
1.8.2	-	1	1.18 experimental snapshot 5	7	7
1.8.3	-	1	1.18 experimental snapshot 6	7	7
1.8.4	-	1	1.18 experimental snapshot 7	7	7
1.8.5	-	1	21w37a	8	7
1.8.6	-	1	21w38a	8	7
1.8.7	-	1	21w39a	8	8
1.8.8	-	1	21w40a	8	8
1.8.9	-	1	21w41a	8	8
15w31a	-	2	21w42a	8	8
15w31b	-	2	21w43a	8	8
15w31c	-	2	21w44a	8	8
15w32a	-	2	1.18 Pre-release 1	8	8
15w32b	-	2	1.18 Pre-release 2	8	8
15w32c	-	2	1.18 Pre-release 3	8	8
15w33a	-	2	1.18 Pre-release 4	8	8
15w33b	-	2	1.18 Pre-release 5	8	8
15w33c	-	2	1.18 Pre-release 6	8	8
15w34a	-	2	1.18 Pre-release 7	8	8
15w34b	-	2	1.18 Pre-release 8	8	8
15w34c	-	2	1.18 Release Candidate 1	8	8
15w34d	-	2	1.18 Release Candidate 2	8	8
15w35a	-	2	1.18 Release Candidate 3	8	8

(续表)

游戏版本名称	数据包版本号	资源包版本号	游戏版本名称	数据包版本号	资源包版本号
15w35b	-	2	1.18 Release Candidate 4	8	8
15w35c	-	2	1.18	8	8
15w35d	-	2	1.18.1 Pre-release 1	8	8
15w35e	-	2	1.18.1 Release Candidate 1	8	8
15w36a	-	2	1.18.1 Release Candidate 2	8	8
15w36b	-	2	1.18.1 Release Candidate 3	8	8
15w36c	-	2	1.18.1	8	8
15w36d	-	2	22w03a	8	8
15w37a	-	2	22w05a	8	8
15w38a	-	2	22w06a	8	8
15w38b	-	2	22w07a	8	8
15w39a	-	2	Deep Dark Experimental Snapshot 1	8	8
15w39b	-	2	1.18.2 Pre-release 1	9	8
15w39c	-	2	1.18.2 Pre-release 2	9	8
15w40a	-	2	1.18.2 Pre-release 3	9	8
15w40b	-	2	1.18.2 Release Candidate 1	9	8
15w41a	-	2	1.18.2	9	8
15w41b	-	2	22w11a	10	9
15w42a	-	2	22w12a	10	9
15w43a	-	2	22w13a	10	9
15w43b	-	2	22w14a	10	9
15w43c	-	2	22w15a	10	9
15w44a	-	2	22w16a	10	9
15w44b	-	2	22w16b	10	9
15w45a	-	2	22w17a	10	9
15w46a	-	2	22w18a	10	9
15w47a	-	2	22w19a	10	9
15w47b	-	2	1.19 Pre-release 1	10	9
15w47c	-	2	1.19 Pre-release 2	10	9
15w49a	-	2	1.19 Pre-release 3	10	9
15w49b	-	2	1.19 Pre-release 4	10	9
15w50a	-	2	1.19 Pre-release 5	10	9
15w51a	-	2	1.19 Release Candidate 1	10	9
15w51b	-	2	1.19 Release Candidate 2	10	9

(续表)

游戏版本名称	数 据 包 版本号	资 源 包 版本号	游戏版本名称	数 据 包 版本号	资 源 包 版本号
16w02a	-	2	1.19	10	9
16w03a	-	2	22w24a	10	9
16w04a	-	2	1.19.1 Pre-release 1	10	9
16w05a	-	2	1.19.1 Release Candidate 1	10	9
16w05b	-	2	1.19.1 Pre-release 2	10	9
16w06a	-	2	1.19.1 Pre-release 3	10	9
16w07a	-	2	1.19.1 Pre-release 4	10	9
16w07b	-	2	1.19.1 Pre-release 5	10	9
1.9-pre1	-	2	1.19.1 Pre-release 6	10	9
1.9-pre2	-	2	1.19.1 Release Candidate 2	10	9
1.9-pre3	-	2	1.19.1 Release Candidate 3	10	9
1.9-pre4	-	2	1.19.1	10	9
1.9	-	2	1.19.2 Release Candidate 2	10	9
1.9.1-pre1	-	2	1.19.2 Release Candidate 3	10	9
1.9.1-pre2	-	2	1.19.2	10	9
1.9.1-pre3	-	2	22w42a	10	11
1.9.1	-	2	22w43a	10	11
1.9.2	-	2	22w44a	10	11
16w14a	-	2	22w45a	10	12
16w15a	-	2	22w46a	10	12
16w15b	-	2	1.19.3 Pre-release 1	10	12
1.9.3-pre1	-	2	1.19.3 Pre-release 2	10	12
1.9.3-pre2	-	2	1.19.3 Pre-release 3	10	12
1.9.3-pre3	-	2	1.19.3 Release Candidate 1	10	12
1.9.3	-	2	1.19.3 Release Candidate 2	10	12
1.9.4	-	2	1.19.3 Release Candidate 3	10	12
16w20a	-	2	1.19.3	10	12
16w21a	-	2	23w03a	11	12
16w21b	-	2	23w04a	11	12
1.10-pre1	-	2	23w05a	11	12
1.10-pre2	-	2	23w06a	12	12
1.10	-	2	23w07a	12	12
1.10.1	-	2	1.19.4 Pre-release 1	12	13
1.10.2	-	2	1.19.4 Pre-release 2	12	13
16w32a	-	3	1.19.4 Pre-release 3	12	13

(续表)

游戏版本名称	数据包版本号	资源包版本号	游戏版本名称	数据包版本号	资源包版本号
16w32b	-	3	1.19.4 Pre-release 4	12	13
16w33a	-	3	1.19.4 Release Candidate 1	12	13
16w35a	-	3	1.19.4 Release Candidate 2	12	13
16w36a	-	3	1.19.4 Release Candidate 3	12	13
16w38a	-	3	1.19.4	12	13
16w39a	-	3	23w12a	13	13
16w39b	-	3	23w13a	13	13
16w39c	-	3	23w14a	13	14
16w40a	-	3	23w16a	14	14
16w41a	-	3	23w17a	14	15
16w42a	-	3	23w18a	15	15
16w43a	-	3	1.20 Pre-release 1	15	15
16w44a	-	3	1.20 Pre-release 2	15	15
1.11-pre1	-	3	1.20 Pre-release 3	15	15
1.11	-	3	1.20 Pre-release 4	15	15
16w50a	-	3	1.20 Pre-release 5	15	15
1.11.1	-	3	1.20 Pre-release 6	15	15
1.11.2	-	3	1.20 Pre-release 7	15	15
17w06a	-	3	1.20 Release Candidate 1	15	15
17w13a	-	3	1.20	15	15
17w13b	-	3	1.20.1 Release Candidate 1	15	15
17w14a	-	3	1.20.1	15	15
17w15a	-	3	23w31a	16	16
17w16a	-	3	23w32a	17	17
17w16b	-	3	23w33a	17	17
17w17a	-	3	23w35a	17	17
17w17b	-	3	1.20.2 Pre-release 1	18	17
17w18a	-	3	1.20.2 Pre-release 2	18	18
17w18b	-	3	1.20.2 Pre-Release 3	18	18
1.12-pre1	-	3	1.20.2 Pre-Release 4	18	18
1.12-pre2	-	3	1.20.2 Release Candidate 1	18	18
1.12-pre3	-	3	1.20.2 Release Candidate 2	18	18
1.12-pre4	-	3	1.20.2	18	18
1.12-pre5	-	3	23w40a	19	18
1.12-pre6	-	3	23w41a	20	18

(续表)

游戏版本名称	数 据 包 版本号	资 源 包 版本号	游戏版本名称	数 据 包 版本号	资 源 包 版本号
1.12-pre7	-	3	23w42a	21	19
1.12	-	3	23w43a	22	20
17w31a	-	3	23w43b	22	20
1.12.1-pre1	-	3	23w44a	23	20
1.12.1	-	3	23w45a	24	21
1.12.2-pre1	-	3	23w46a	25	21
1.12.2-pre2	-	3	1.20.3 Pre-Release 1	26	22
1.12.2	-	3	1.20.3 Pre-Release 2	26	22
17w43a	-	3	1.20.3 Pre-Release 3	26	22
17w43b	-	3	1.20.3 Pre-Release 4	26	22
17w45a	-	3	1.20.3 Release Candidate 1	26	22
17w45b	-	3	1.20.3	26	22
17w46a	-	3	1.20.4 Release Candidate 1	26	22
17w47a	-	3	1.20.4	26	22
17w47b	-	3	23w51a	27	22
17w48a	4	4	23w51b	27	22
17w49a	4	4	24w03a	28	24
17w49b	4	4	24w03b	28	24
17w50a	4	4	24w04a	29	24
18w01a	4	4	24w05a	30	25
18w02a	4	4	24w05b	30	25
18w03a	4	4	24w06a	31	26
18w03b	4	4	24w07a	32	26
18w05a	4	4	24w09a	33	28
18w06a	4	4	24w10a	34	28
18w07a	4	4	24w11a	35	29
18w07b	4	4	24w12a	36	30
18w07c	4	4	24w13a	37	31
18w08a	4	4	24w14a	38	31
18w08b	4	4	1.20.5 Pre-Release 1	39	31
18w09a	4	4	1.20.5 Pre-Release 2	40	31
18w10a	4	4	1.20.5 Pre-Release 3	41	31
18w10b	4	4	1.20.5 Pre-Release 4	41	32
18w10c	4	4	1.20.5 Release Candidate 1	41	32
18w10d	4	4	1.20.5 Release Candidate 2	41	32

(续表)

游戏版本名称	数据包版本号	资源包版本号	游戏版本名称	数据包版本号	资源包版本号
18w11a	4	4	1.20.5 Release Candidate 3	41	32
18w14a	4	4	1.20.5	41	32
18w14b	4	4	1.20.6 Release Candidate 1	41	32
18w15a	4	4	1.20.6	41	32
18w16a	4	4	24w18a	42	33
18w19a	4	4	24w19a	43	33
18w19b	4	4	24w20a	44	33
18w20a	4	4	24w21a	45	34
18w20b	4	4	1.21 Pre-Release 1	46	34
18w20c	4	4	1.21 Pre-Release 2	47	34
18w21a	4	4	1.21 Pre-Release 3	48	34
18w21b	4	4	1.21 Pre-Release 4	48	34
18w22a	4	4	1.21 Release Candidate 1	48	34
18w22b	4	4	1.21	48	34
18w22c	4	4	1.21.1 Release Candidate 1	48	34
1.13-pre1	4	4	1.21.1	48	34
1.13-pre2	4	4	24w33a	49	35
1.13-pre3	4	4	24w34a	50	36
1.13-pre4	4	4	24w35a	51	36
1.13-pre5	4	4	24w36a	52	37
1.13-pre6	4	4	24w37a	53	38
1.13-pre7	4	4	24w38a	54	39
1.13-pre8	4	4	24w39a	55	39
1.13-pre9	4	4	24w40a	56	40
1.13-pre10	4	4	1.21.2 Pre-Release 1	57	41
1.13	4	4	1.21.2 Pre-Release 2	57	41
18w30a	4	4	1.21.2 Pre-Release 3	57	42
18w30b	4	4	1.21.2 Pre-Release 4	57	42
18w31a	4	4	1.21.2 Pre-Release 5	57	42
18w32a	4	4	1.21.2 Release Candidate 1	57	42
18w33a	4	4	1.21.2 Release Candidate 2	57	42
1.13.1-pre1	4	4	1.21.2	57	42
1.13.1-pre2	4	4	1.21.3	57	42
1.13.1	4	4	24w44a	58	43
1.13.2-pre1	4	4	24w45a	59	44

(续表)

游戏版本名称	数 据 包 版本号	资 源 包 版本号	游戏版本名称	数 据 包 版本号	资 源 包 版本号
1.13.2-pre2	4	4	24w46a	60	45
1.13.2	4	4	1.21.4 Pre-Release 1	60	46
18w43a	4	4	1.21.4 Pre-Release 2	61	46
18w43b	4	4	1.21.4 Pre-Release 3	61	46
18w43c	4	4	1.21.4 Release Candidate 1	61	46
18w44a	4	4	1.21.4 Release Candidate 2	61	46
18w45a	4	4	1.21.4 Release Candidate 3	61	46
18w46a	4	4	1.21.4	61	46
18w47a	4	4	25w02a	62	47
18w47b	4	4	25w03a	63	48
18w48a	4	4	25w04a	64	49
18w48b	4	4	25w05a	65	50
18w49a	4	4	25w06a	66	51
18w50a	4	4	25w07a	67	52
19w02a	4	4	25w08a	68	53
19w03a	4	4	25w09a	69	53
19w03b	4	4	25w10a	70	54
19w03c	4	4	1.21.5 Pre-Release 1	70	55
19w04a	4	4	1.21.5 Pre-Release 2	71	55
19w04b	4	4	1.21.5 Pre-Release 3	71	55
19w05a	4	4	1.21.5 Release Candidate 1	71	55
19w06a	4	4	1.21.5 Release Candidate 2	71	55
19w07a	4	4	1.21.5	71	55
19w08a	4	4	25w15a	72	56
19w08b	4	4	25w16a	73	57
19w09a	4	4	25w17a	74	58
19w11a	4	4	25w18a	75	59
19w11b	4	4	25w19a	76	60
19w12a	4	4	25w20a	77	61
19w12b	4	4	25w21a	78	62
19w13a	4	4	1.21.6 Pre-Release 1	79	63
19w13b	4	4	1.21.6 Pre-Release 2	79	63
19w14a	4	4	1.21.6 Pre-Release 3	80	63
19w14b	4	4	1.21.6 Pre-Release 4	80	63
1.14 Pre-Release 1	4	4	1.21.6 Release Candidate 1	80	63

(续表)

游戏版本名称	数据包版本号	资源包版本号	游戏版本名称	数据包版本号	资源包版本号
1.14 Pre-Release 2	4	4	1.21.6	80	63
1.14 Pre-Release 3	4	4	1.21.7 Release Candidate 1	80	63
1.14 Pre-Release 4	4	4	1.21.7 Release Candidate 2	81	64
1.14 Pre-Release 5	4	4	1.21.7	81	64
1.14	4	4	1.21.8 Release Candidate 1	81	64
1.14.1 Pre-Release 1	4	4	1.21.8	81	64
1.14.1 Pre-Release 2	4	4	25w31a	82.0	65.0
1.14.1	4	4	25w32a	83.0	65.1
1.14.2 Pre-Release 1	4	4	25w33a	83.1	65.2
1.14.2 Pre-Release 2	4	4	25w34a	84.0	66.0
1.14.2 Pre-Release 3	4	4	25w34b	84.0	66.0
1.14.2 Pre-Release 4	4	4	25w35a	85.0	67.0
1.14.2	4	4	25w36a	86.0	68.0
1.14.3 Pre-Release 1	4	4	25w36b	86.0	68.0
1.14.3 Pre-Release 2	4	4	25w37a	87.0	69.0
1.14.3 Pre-Release 3	4	4	1.21.9 Pre-Release 1	87.1	69.0
1.14.3 Pre-Release 4	4	4	1.21.9 Pre-Release 2	88.0	69.0
1.14.3	4	4	1.21.9 Pre-Release 3	88.0	69.0
1.14.4 Pre-Release 1	4	4	1.21.9 Pre-Release 4	88.0	69.0
1.14.4 Pre-Release 2	4	4	1.21.9 Release Candidate 1	88.0	69.0
1.14.4 Pre-Release 3	4	4	1.21.9	88.0	69.0
1.14.4 Pre-Release 4	4	4	1.21.10 Release Candidate 1	88.0	69.0
1.14.4 Pre-Release 5	4	4	1.21.10	88.0	69.0
1.14.4 Pre-Release 6	4	4	25w41a	89.0	70.0
1.14.4 Pre-Release 7	4	4	25w42a	90.0	70.1
1.14.4	4	4	25w43a	91.0	71.0
1.14.3 - Combat Test	4	4	25w44a	92.0	72.0
Combat Test 2	4	4	25w45a	93.0	73.0
Combat Test 3	4	4	25w45a Unobfuscated	93.0	73.0
19w34a	4	4	25w45b	93.1	74.0
19w35a	4	4	25w45b Unobfuscated	93.1	743.0
19w36a	4	4	1.21.11 Pre-Release 1	94.0	75.0
19w37a	4	4	1.21.11 Pre-Release 1 Unobfuscated	94.0	75.0
19w38a	4	4	1.21.11 Pre-Release 2	94.0	75.0

(续表)

游戏版本名称	数据包版本号	资源包版本号	游戏版本名称	数据包版本号	资源包版本号
19w38b	4	4	1.21.11 Pre-Release 2 Unobfuscated	94.0	75.0
19w39a	4	4	1.21.11 Pre-Release 3	94.0	75.0
19w40a	4	4	1.21.11 Pre-Release 3 Unobfuscated	94.0	75.0
19w41a	4	4	1.21.11 Pre-Release 4	94.1	75.0
19w42a	4	4	1.21.11 Pre-Release 4 Unobfuscated	94.1	75.0
19w44a	4	4	1.21.11 Pre-Release 5	94.1	75.0
19w45a	4	4	1.21.11 Pre-Release 5 Unobfuscated	94.1	75.0
19w45b	4	4	1.21.11 Release Candidate 1	94.1	75.0
19w46a	4	4	1.21.11 Release Candidate 1 Unobfuscated	94.1	75.0
19w46b	4	4	1.21.11 Release Candidate 2	94.1	75.0
1.15 Pre-release 1	5	5	1.21.11 Release Candidate 2 Unobfuscated	94.1	75.0
1.15 Pre-Release 2	5	5	1.21.11 Release Candidate 3	94.1	75.0
1.15 Pre-release 3	5	5	1.21.11 Release Candidate 3 Unobfuscated	94.1	75.0
1.15 Pre-release 4	5	5	1.21.11	94.1	75.0
1.15 Pre-release 5	5	5	1.21.11 Unobfuscated	94.1	75.0
1.15 Pre-release 6	5	5	26.1 Snapshot 1	95.0	76.0
1.15 Pre-release 7	5	5	26.1 Snapshot 2	96.0	77.0
1.15	5	5	26.1 Snapshot 3	97.0	78.0
1.15.1 Pre-release 1	5	5	26.1 Snapshot 4	97.1	78.1
1.15.1	5	5			

I.2 方块状态

I.3 数据包标签

索引

		词组 (Quotable phrase)	14
A			
安全模式 (Safe mode)	35	D	
		单个词 (Single word)	14
		单精度浮点数 (Float)	14
		端 (Sides)	55
		对象 (Object)	23
B			
崩溃 (Crash)	29		
崩溃报告 (Crash Report)	29		
边界箱 (Boundary box)	68		
扁平化 (The flattening)	8		
标签 (Tag)	12		
布尔值 (Bool)	14, 22		
		二维坐标 (Three-dimensional coordinates)	75
C			
F			
长整数 (Long)	14	方块 (Block)	66
成功次数 (Success)	19	方块更新 (Block update)	68
次要版本号 (Minor versions)	38	方块实体 (Block entity)	67

方块属性 (Block property)	67
方块状态 (Block states)	67
方块坐标 (Block position)	74
封包 (Packet)	55
服务 (器) 端 (Server)	55

G

功能开关 (Feature Flag)	43
功能数据包 (Feature datapack)	43
功能元素 (Feature Element)	43
固有注册表 (Built-in registry)	2

H

哈希值 (Hash value, 散列值)	28
红石刻 (Redstone tick, rt)	66

J

键值对 (Name-value pair)	21
加载标签 (Load ticket)	61
加载等级 (Load level)	60
结果 (Result)	19
计划刻 (Schedule tick)	66
计算等级	60
JSON (JavaScript Object Notation, JavaScript 对象表示法)	21

K

刻 (Tick)	57
客户端 (Client)	55
可写注册表 (Writable registry)	2
控制台命令 (Console command)	12

L

流体状态 (Fluid states)	67
逻辑端 (Logical sides)	55
逻辑服务端 (Logical server)	55
逻辑客户端 (Logical client)	55

M

MC-CMD	12
每刻毫秒数 (Milliseconds per tick, MSPT)	57
每秒刻度数 (Ticks per second, TPS)	57
命令 (Command)	12
命令来源堆叠 (Command source stack)	19
命令历史 (Command history)	15
命令上下文 (Command context)	19
命令源 (Command origin)	19
(赋) 命名空间 ID (Namespaced identifier)	9, 10
命名空间字符串 (Namespaced string)	9
模拟距离 (Simulation distance)	61

N

数组 (Array)	22
随机刻 (Random tick)	65

内置服务器 (Integrated server)	55
---------------------------	----

T

P

贪婪词组 (Greedy phrase)	15
调试棒 (Debug stick)	67

判定箱 (Hitbox)	68
--------------	----

W

Q

切比雪夫距离 (Chebyshev distance)	63
权限等级 (Permission level)	16
区段 (Chunk section)	60
区块 (Chunk)	60
区块刻 (Chunk tick)	65

玩家加载标签 (Player loading ticket)	61
玩家计算标签 (Player simulation ticket)	61
微时序 (Microtiming, 微观延迟)	59
物理端 (Physical sides)	55
物理服务端 (Physical server)	56
物理客户端 (Physical client)	55

X

S

三维坐标 (Three-dimensional coordinates)	75
蛇形命名法 (Snake case)	12
世界指定资源包 (World specific resources)	47
视平线 (Eye level)	68
实体 (Entity)	68
实验性内容 (Experiments)	43
实验性设置 (Experimental settings)	35
双精度浮点数 (Double)	14
数据包 (Data pack)	33
数据包版本号 (Data pack format)	38
数据包标签 (Tags in data packs)	12, 44
数值 (Number)	22

闲置超时 (Idle timeout)	65
斜杠命令 (Slash command)	12
渲染距离 (Render distance)	61

Y

游戏规则 (Game rule)	70
游戏刻 (Game tick, gt)	57
元数据 (Metadata)	36, 48

Z

整数 (Integer)	14
帧率 (Frame per second, FPS)	57
执行朝向 (Execution rotation)	19
执行锚点 (Execution anchor)	19
执行上下文 (Execution context)	19
执行维度 (Execution dimension)	19
执行位置 (Execution position)	19
执行者 (Executor)	19
中心校准 (Center correct)	75
转义字符 (Escape character)	24
专用服务器 (Dedicated server, 独立服务端)	56
注册表 (Registry)	2
主要版本号 (Major versions)	38
字段 (Field)	21
字符串 (String)	14, 22
子区块 (Sub chunk)	60
资源包 (Resource pack)	47
资源包版本号 (Resource pack format)	50
资源标识符 (Resource identifier)	9
资源路径 (Resource location)	9
坐标 (Coordinate)	72

参考文献

- [1] 中文 Minecraft Wiki[EB/OL]. (2026)[2026-01-14]. <https://zh.minecraft.wiki/>.
- [2] DREAMY_BLAZE. 如何合并多个版本的数据包? [J]. Feature, 2025, 1(4).